

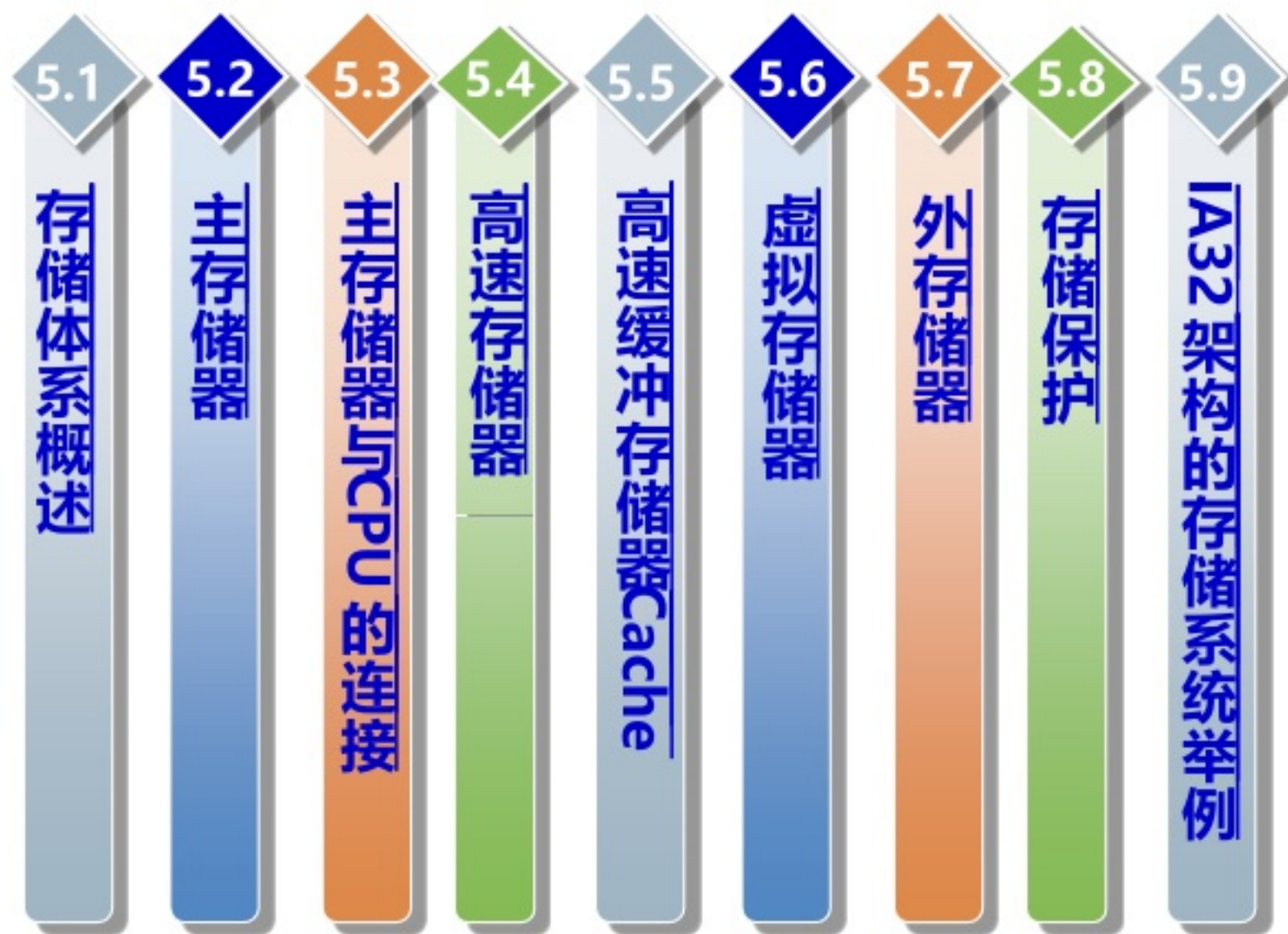
计算机组成原理与系统结构

第5章 存储体系





第5章 存储体系





5.1 存储体系概述

- ❖ 一个二进制位 (bit) 是构成存储器的**最小单位**；字节 (8bits) 是数据存储的**基本单位**。
- ❖ 单元地址是内存单元的**唯一标志**。
- ❖ 存储器具有两种基本的访问操作：读和写。



存储器的分类



主存储器的性能指标



存储器的层次结构





一、存储器的分类

❖ 1、存储器分类

■ (1) 按存储介质分类

- 半导体器件：半导体存储器（RAM、ROM，用作主存）
- 磁性材料：磁表面存储器（磁盘、磁带，用作辅存）
- 光介质：光盘存储器（用作辅存）

■ (2) 按存取方式分类

- 随机存取存储器RAM（Random Access Memory）：存储器中任何存储单元的内容都能按地址被随机存取，且存取时间和存储单元的物理位置无关（主存）
- 顺序存取存储器SAM（Sequentially Addressed Memory）：存取时间和存储单元的物理位置有关（磁盘、磁带）
- 相联存储器CAM（Content Addressed Memory）：按内容来检索存储器单元，访问该单元的另一部分内容。



一、存储器的分类

❖ 1、存储器分类

■ (3) 按存储器的读写功能分类

- 只读存储器ROM (Read Only Memory) : 特指只能读出、不可改写的存储器; 有些ROM可改写, 因历史原因及功能特点仍称为ROM; 大多是随机存取。
- 读写存储器 (Read Write Memory) : 即可读出、也可写入。

■ (4) 按信息的可保存性分类

- 永久记忆的存储器: 又称非易失性 (nonvolatile) 存储器, 在断电后还能保存信息 (辅存、ROM)
- 非永久记忆的存储器: 又称易失性 (volatile) 存储器, 在断电后信息丢失 (主存中的RAM)



一、存储器的分类

❖ 1、存储器分类

■ (5) 按在计算机系统中的作用分类

- **主存储器**：又称**内存**，为主机的一部分，用于存放系统当前正在执行的数据和程序，属于临时存储器。
- **辅助存储器**：又称**外存**，为外部设备，用于存放暂不用的数据和程序，属于永久存储器。



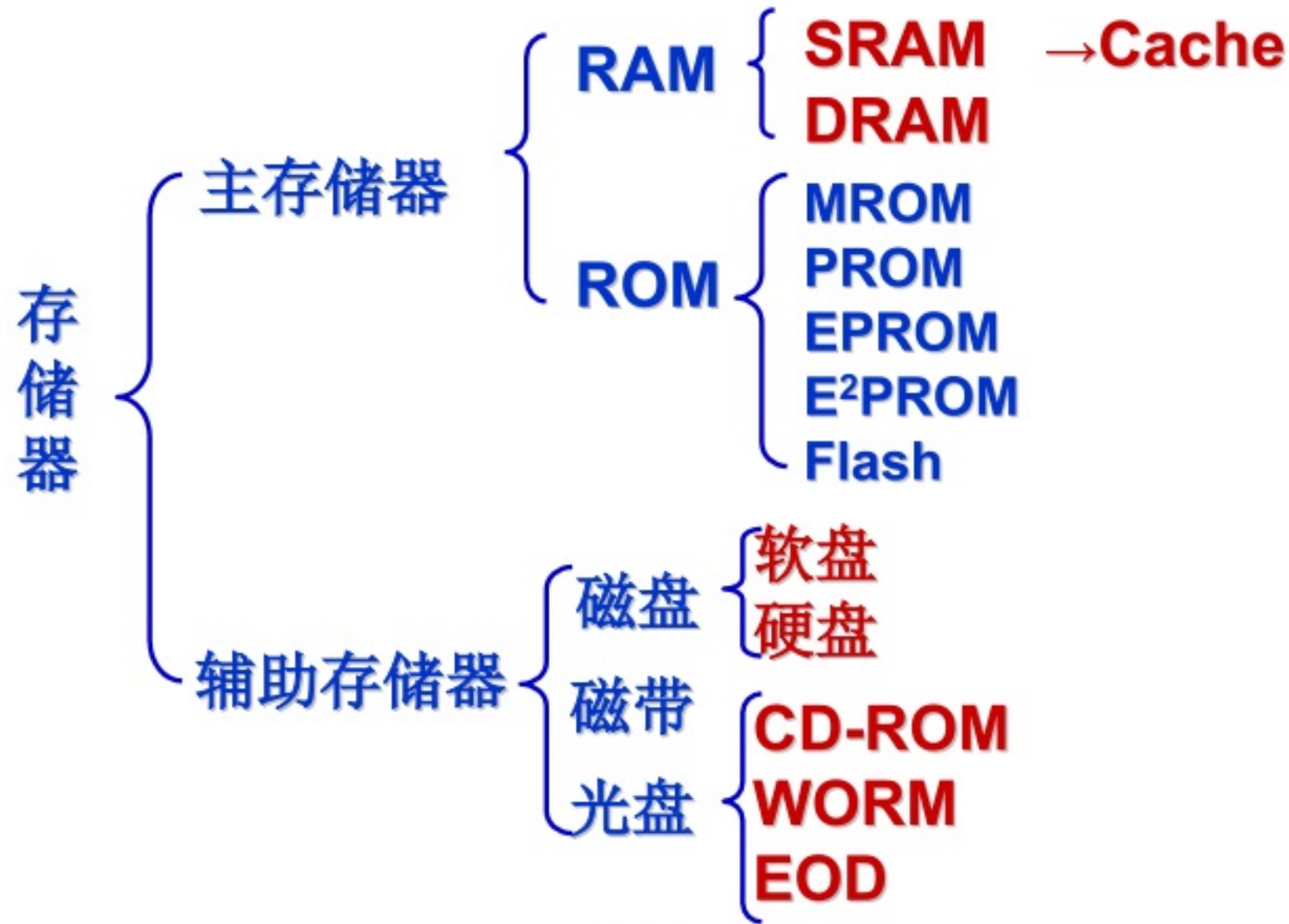
一、存储器的分类

❖ 2、主存储器分类

- **主存的地位：**在现代计算机中，主存储器处于全机的中心地位。
- **主存的类型：**要求为随机存取、快速
 - 随机读写存储器 (RAM)
 - 只读存储器 (ROM)
 - 掩膜式只读存储器 (MROM)
 - 可编程只读存储器(PROM)
 - 可擦除可编程的只读存储器(EPROM)
 - 电可擦除的可编程的只读存储器(E²PROM)
 - 闪存(Flash Memory)：在E²PROM上发展起来的永久性存储器



存储器分类综述





二、主存储器的性能指标

❖ 1、存储容量：

- 指存储器可容纳的二进制信息量
- 单位：字节或位

❖ 存储器芯片的存储容量 = 存储单元个数 × 每存储单元的位数

- 量化单位：

千kilo

• $1K = 2^{10}$

• $1P = 2^{50}$

Peta

兆mega

$1M = 2^{20}$

$1E = 2^{60}$

Exa

千兆giga

$1G = 2^{30}$

$1Z = 2^{70}$

太tera

$1T = 2^{40}$

$1Y = 2^{80}$



二、主存储器的性能指标

❖ 2、存储速度：由以下3个方法来衡量。

- **存取时间** (Memory Access Time) : 指从启动一次存储器操作到完成该操作所需的全部时间。
 - 存取时间愈短, 其性能愈好。通常存取时间用纳秒 ($ns = 10^{-9}s$) 为单位。
- **存储周期** (Memory Cycle Time) : 指存储器进行连续两次独立的存储器操作所需的最小间隔时间。
 - 通常存取周期 T_C 大于存取时间 t_A , 即 $T_C \geq t_A$ 。
- **存储器带宽**: 是单位时间里存储器所能存取的最大信息量, 存储器带宽的计量单位通常是位/秒 (bps) 或字节/秒, 它是衡量数据传输速率的重要技术指标。



二、主存储器的性能指标

❖ **3、存储器的价格：**用每位的价格来衡量。

- 设存储器容量为 S ，总价格为 C ，则位价为 C/S (分/位)。
- 它不仅包含了存储元件的价格，还包括为该存储器操作服务的外围电路的价格。

❖ **4、可靠性：**指存储器正常工作（正确存取）的性能。

❖ **5、功耗：**存储器工作的耗电量。

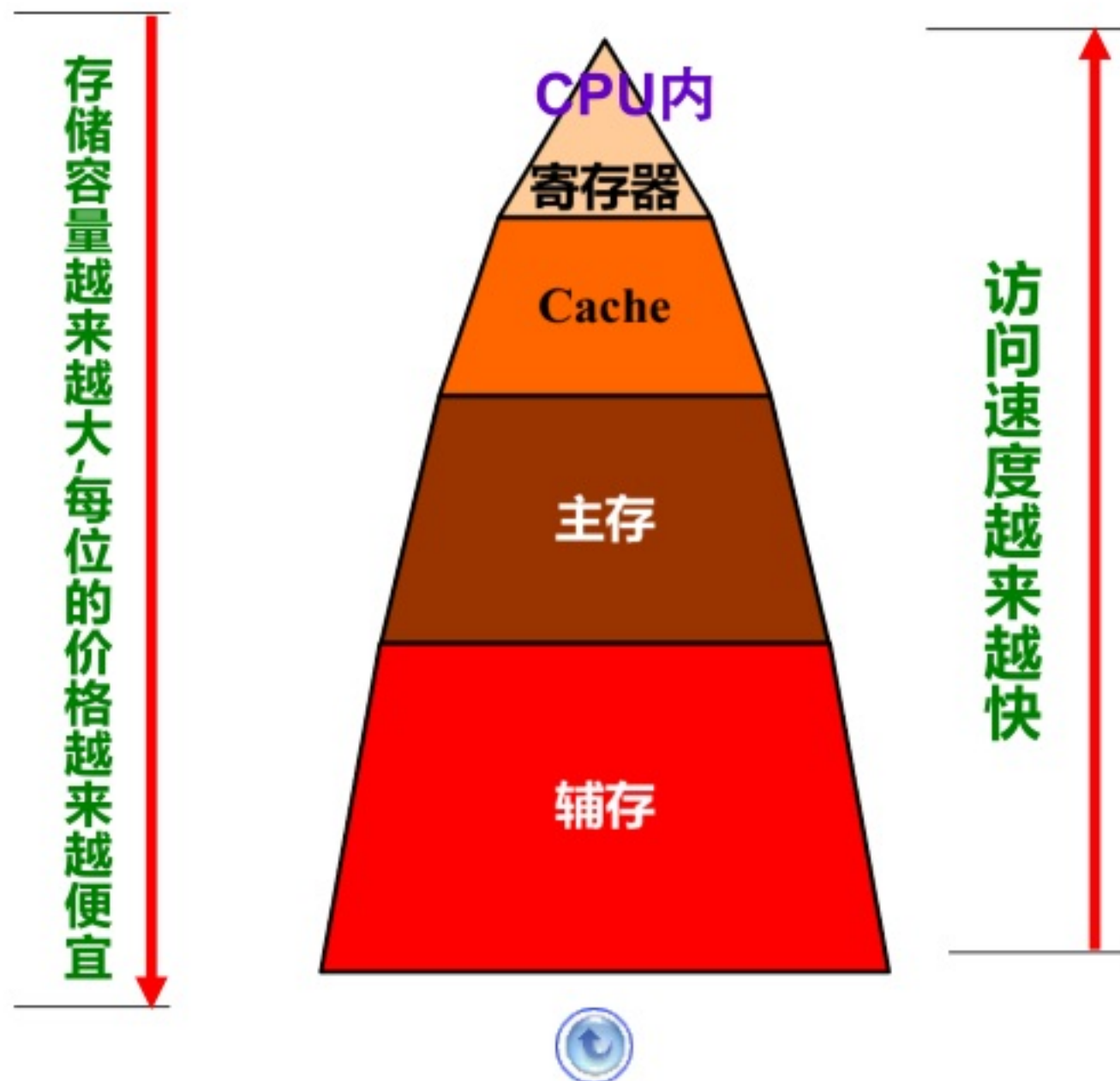
❖ **存储容量、速度和价格的关系：**

- 速度快的存储器往往价格较高，容量也较小。
- 容量、速度和价格三个指标是相互制约的。





三、存储器的层次结构





5.2 主存储器

❖主存储器特点:

- ❑ ①可以被CPU直接存取（访问）
- ❑ ②一般由半导体材质构成
- ❑ ③**随机存取**：读写任意存储单元所用时间是相同的，与单元地址无关
- ❑ ④**与辅存相比，速度快，价格高，容量小**

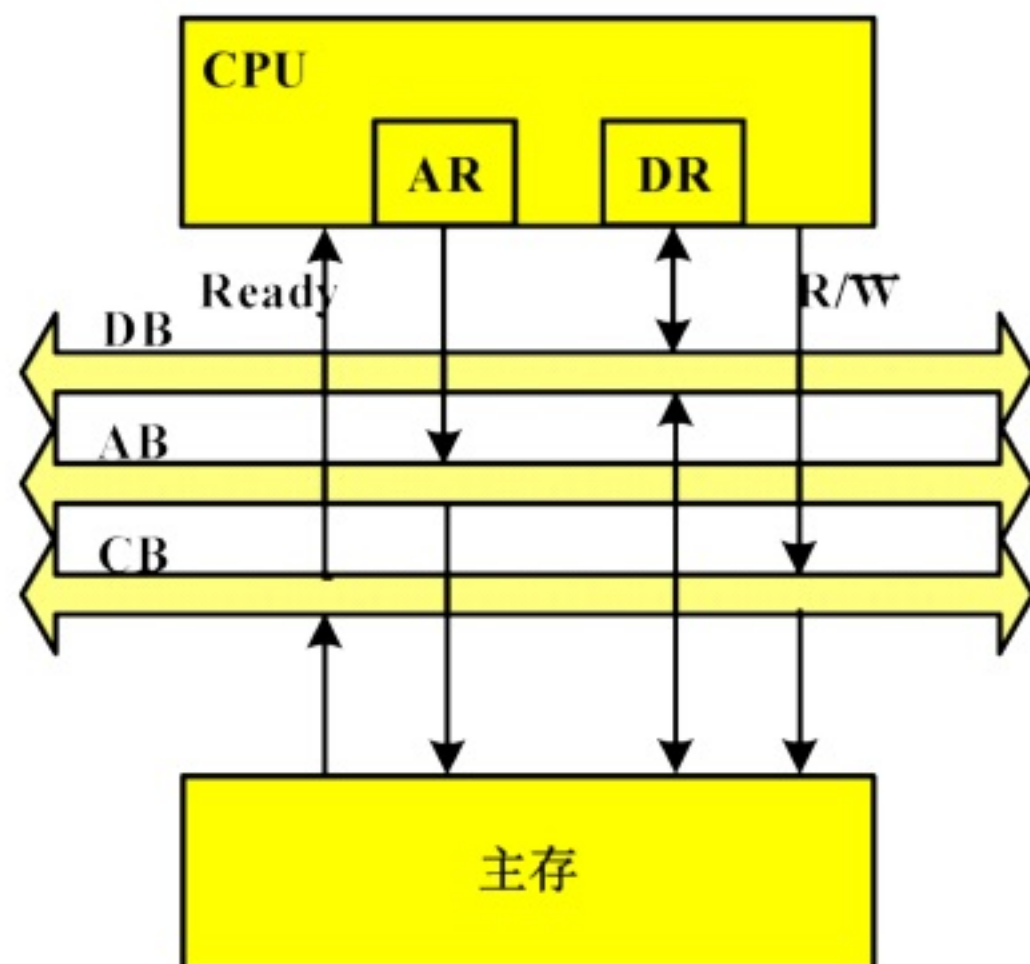
❖**AR** (Address Register) : 地址寄存器

❖**DR** (Data Register) : 数据寄存器

❖如果只用于存放**主存地址**，则：

❖**MAR**: Memory Address Register, 存储器地址寄存器

❖**MDR**: Memory Data Register, 存储器数据寄存器





5.2 主存储器

❖ CPU访问主存：主存的操作；

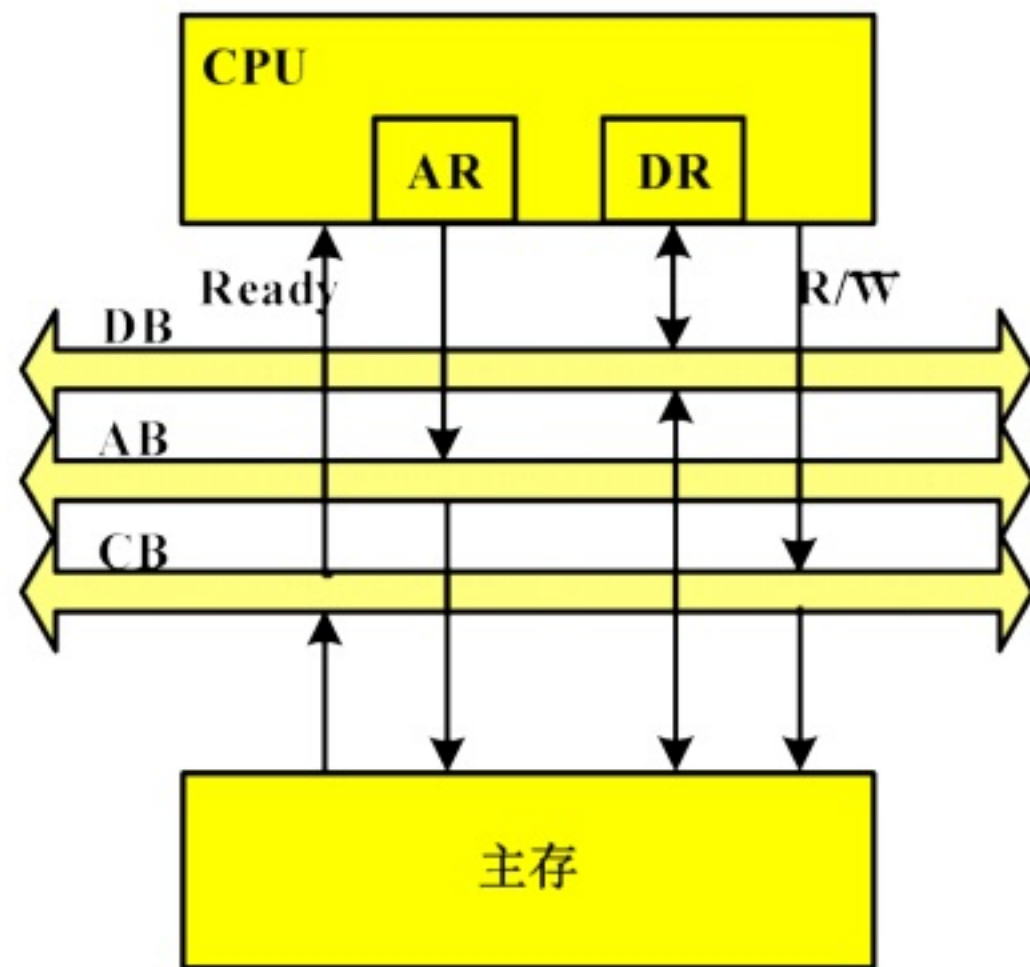
■ CPU读存储器操作：

- 送地址到AB
- 发读信号
- 从DB上接收数据到目的部件

■ CPU写存储器操作：

- 送地址到AB，送数据到DB
- 发写信号
- 等待写操作完成

- 异步存储器：CPU须等待Ready=1后，才能从DB上读数据或者撤销数据





5.2 主存储器

❖ 主存储器按其功能可分为RAM和ROM。



随机读写存储器RAM



只读存储器ROM

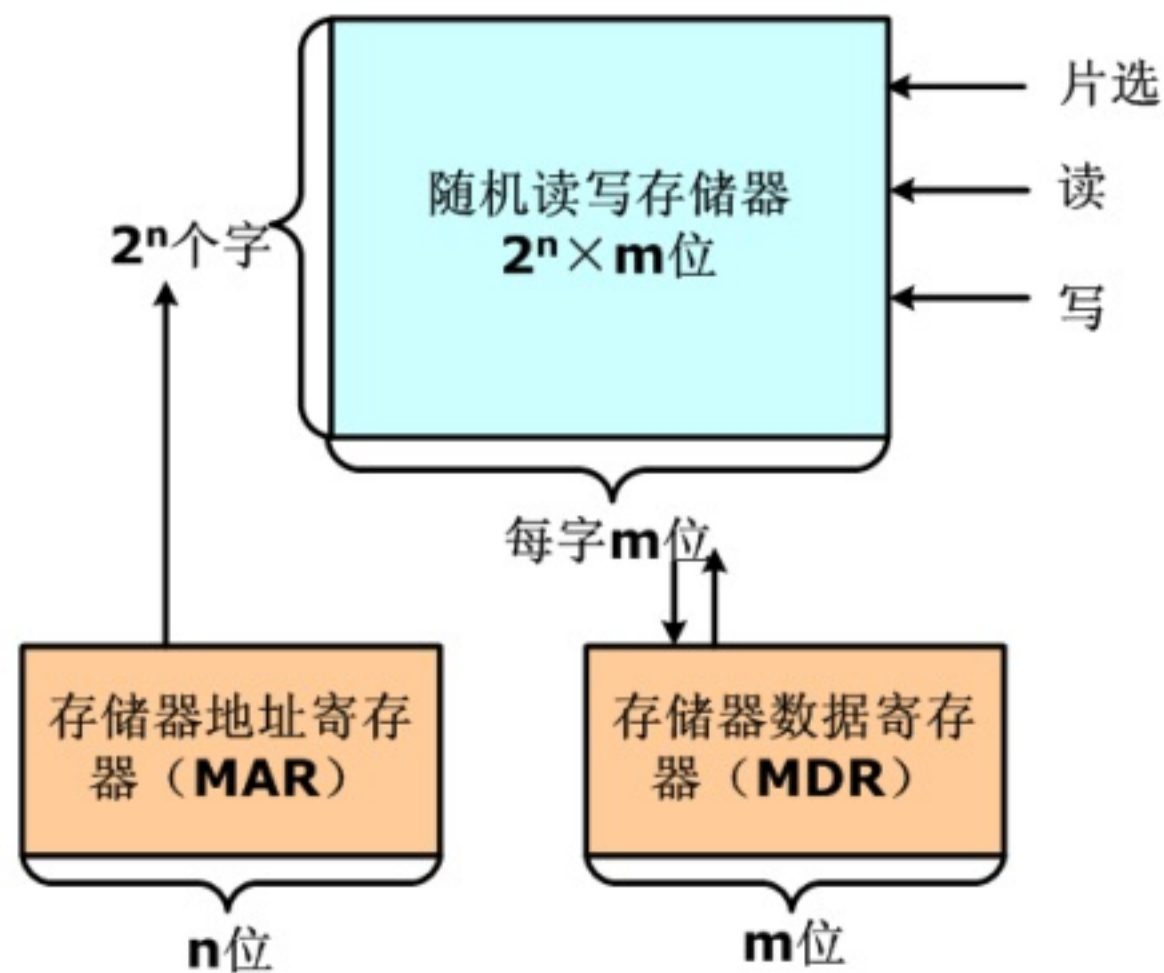


高性能的主存储器





一、随机读写存储器RAM





一、随机读写存储器RAM

1

静态存储器 (SRAM)

2

动态存储器 (DRAM)

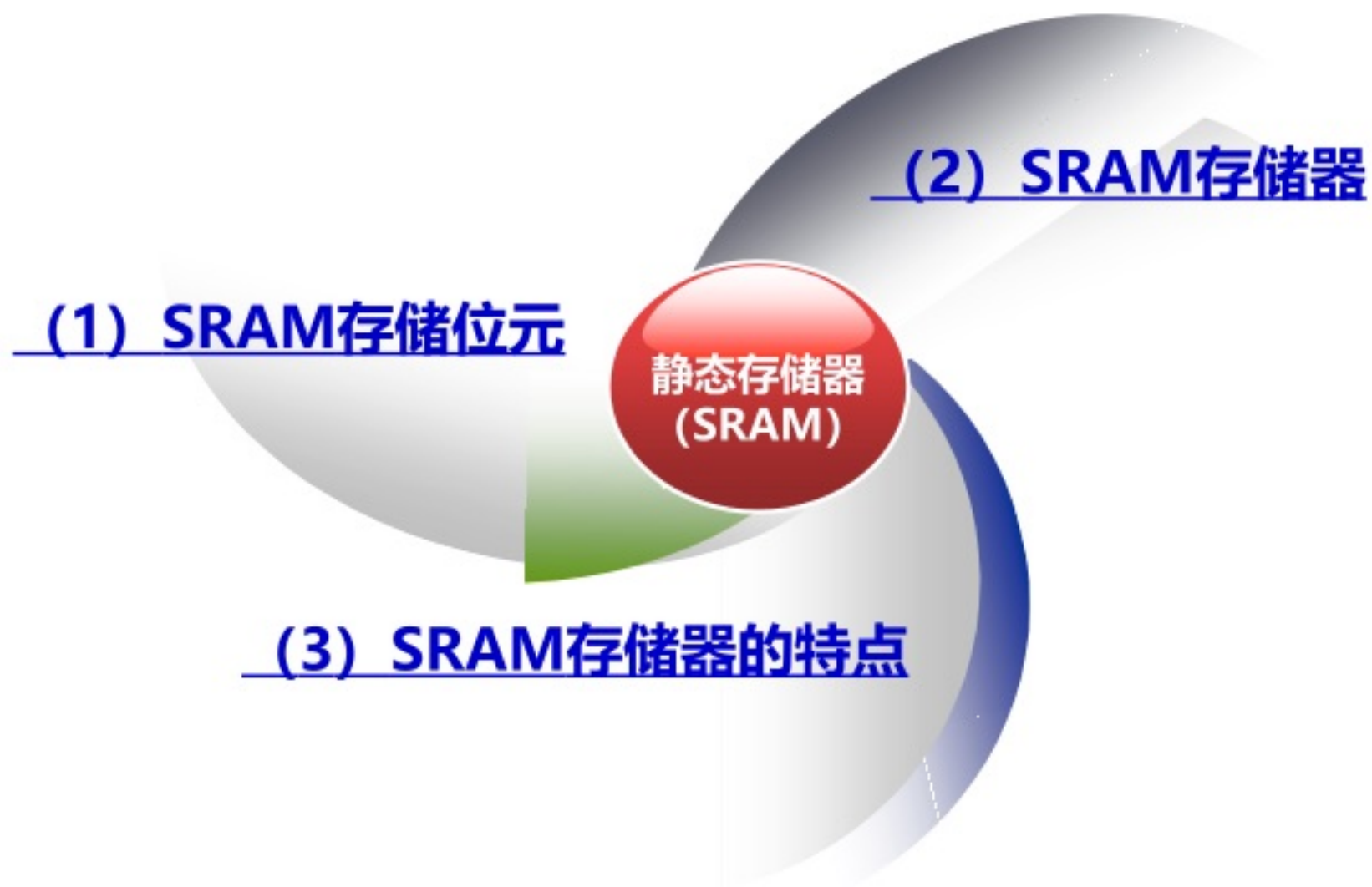
3

SRAM和DRAM的对比





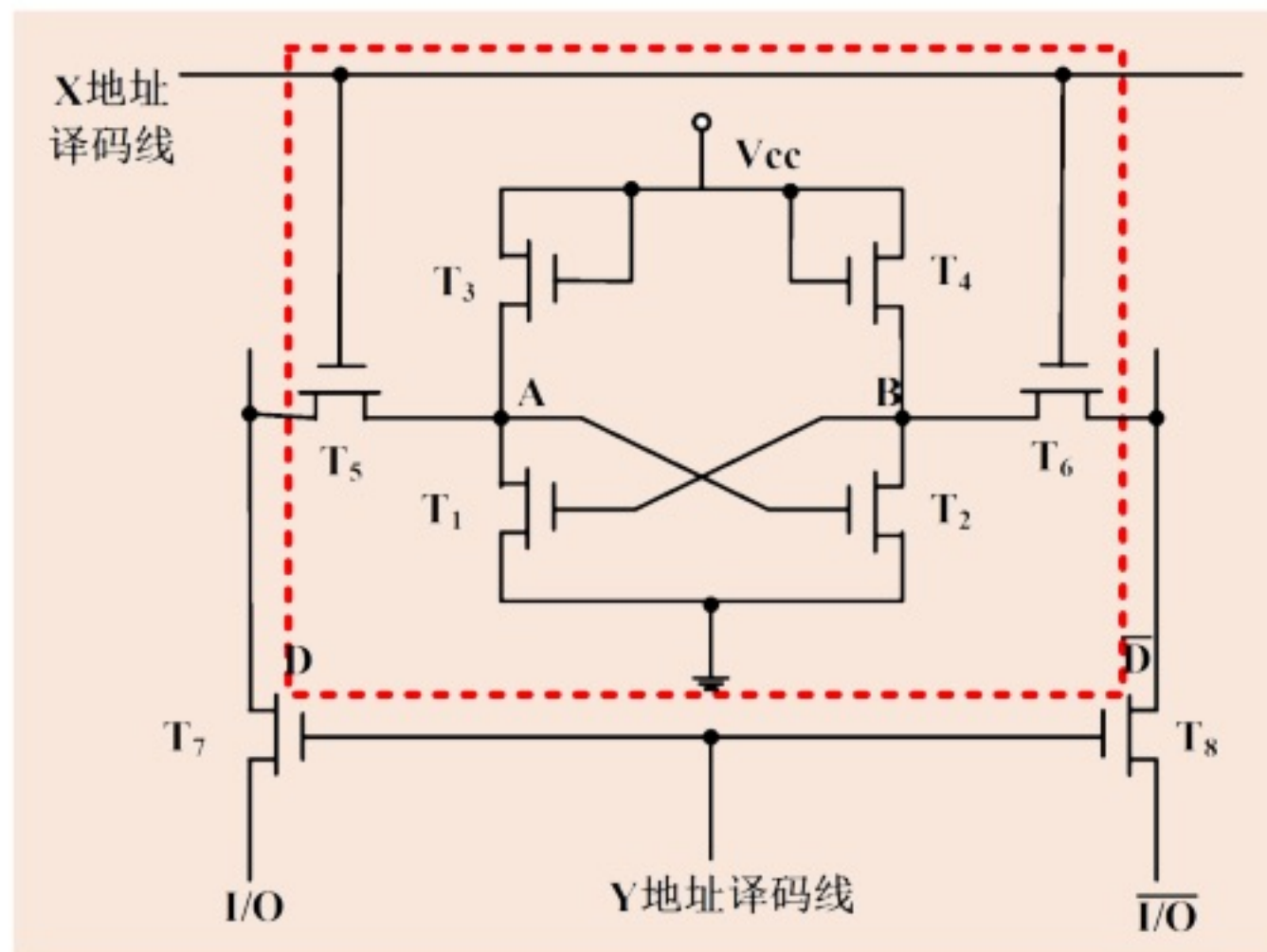
1、静态存储器 (SRAM)



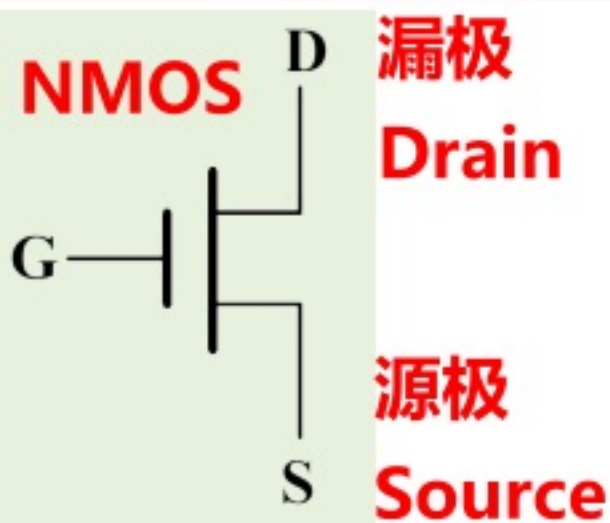


(1) SRAM存储位元

六管NMOS静态存储器结构

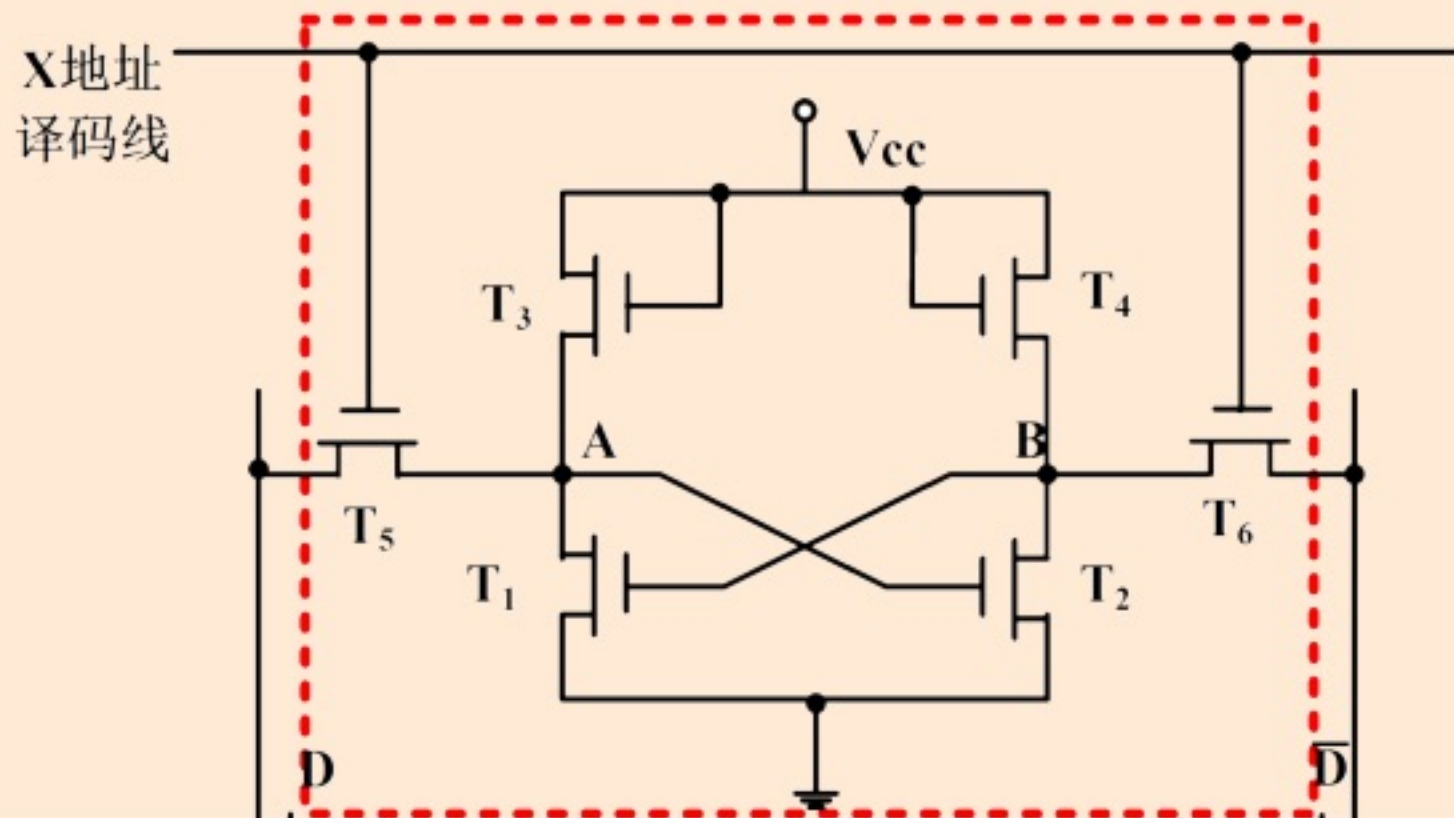


栅极
Gate



- ❖ $V_{GS} \geq V_{TN} (+2V)$ 时, NMOS管导通, D和S之间内阻很低, $V_{DS} \approx 0$, 相当于一个**闭合的开关**
- ❖ $V_{GS} < V_{TN} (+2V)$ 时, NMOS管截止, D和S之间内阻很高, 相当于一个**断开的开关**

(1) SRAM存储位元



❖ T1、T2: 工作管

❖ T3、T4: 负载管

❖ T5、T6: 门控管

❖ “1” 状态: T1截止, T2导通

▪ A点= H B点= L

❖ “0” 状态: T2截止, T1导通

▪ A点= L B点= H

❖ 平时: X译码线=L; T5、T6截止, D线与A、 $\bar{D}$ 线与B点隔离

❖ 读: X译码线=H; T5、T6导通, $V_A \rightarrow D$ 线、 $V_B \rightarrow \bar{D}$ 线

▪ ? : 读 “0” D线=L、 $\bar{D}$ 线=H

▪ ? : 读 “1” D线=H、 $\bar{D}$ 线=L

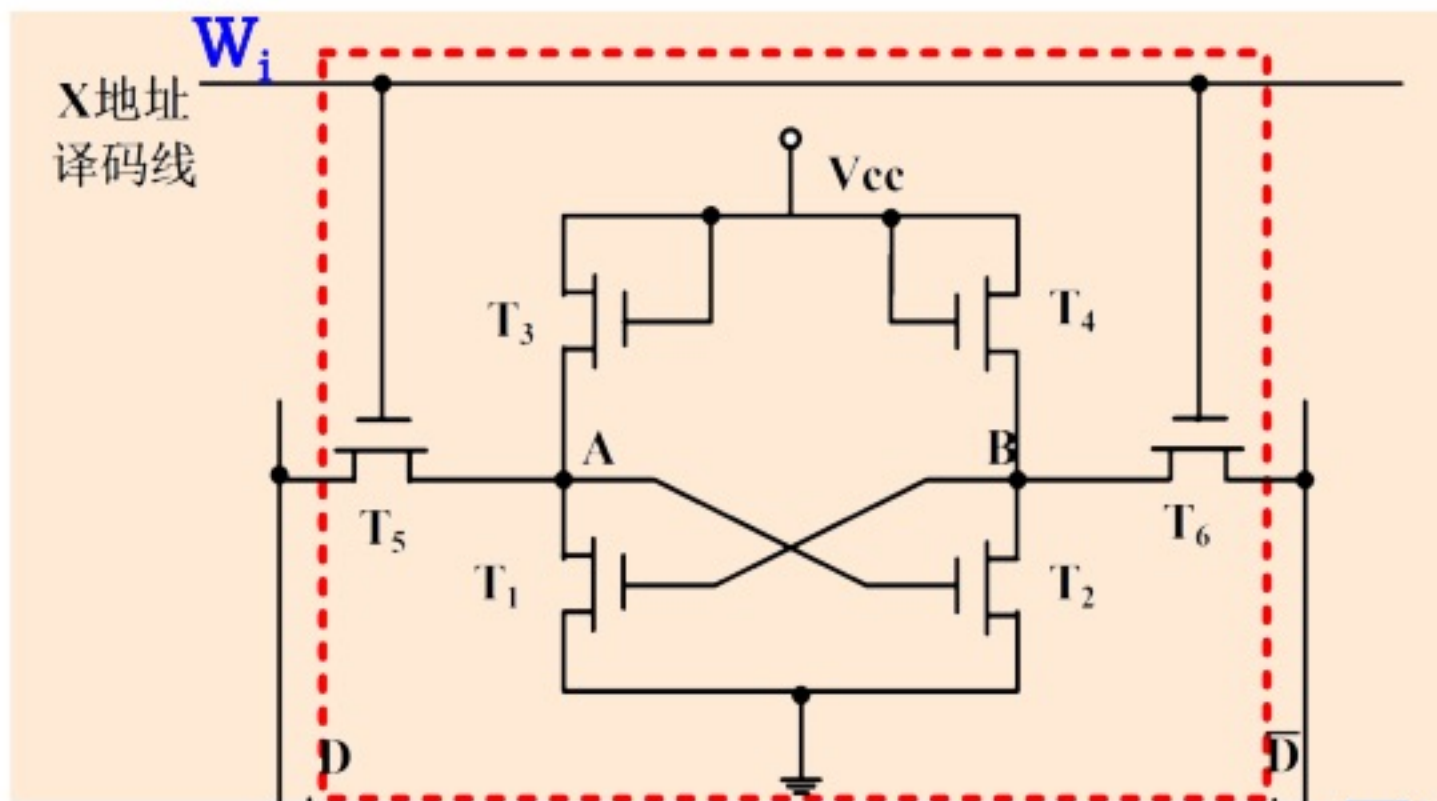
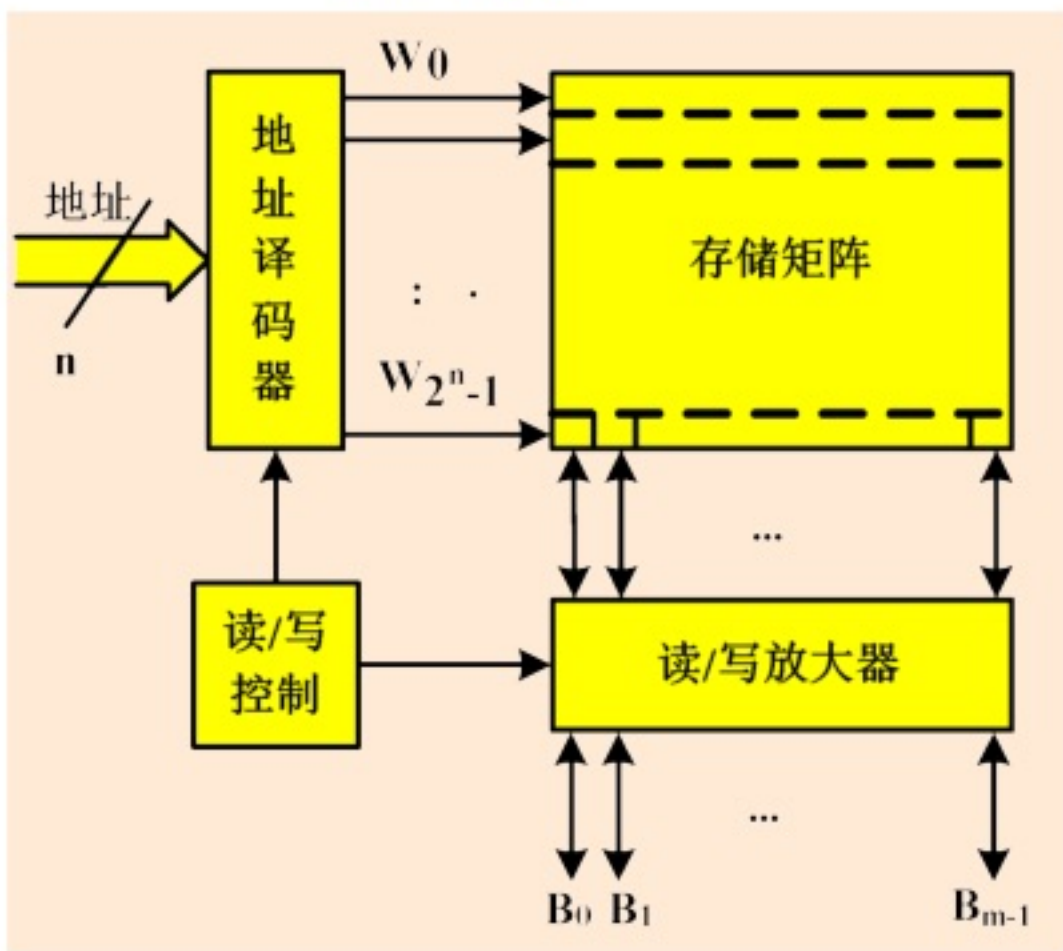
❖ 写: X地址译码线=H; T5、T6导通, D线 $\rightarrow$ A、 $\bar{D}$ 线 $\rightarrow$ B点

▪ 写 “0” ? D线=L、 $\bar{D}$ 线=H

▪ 写 “1” ? D线=H、 $\bar{D}$ 线=L

- ❖ ①存储位元→存储矩阵；②地址译码器

- ### ❖ Wi: 地址译码器输出的字选择线

 $2^n \times m$ 位



(2) SRAM存储器

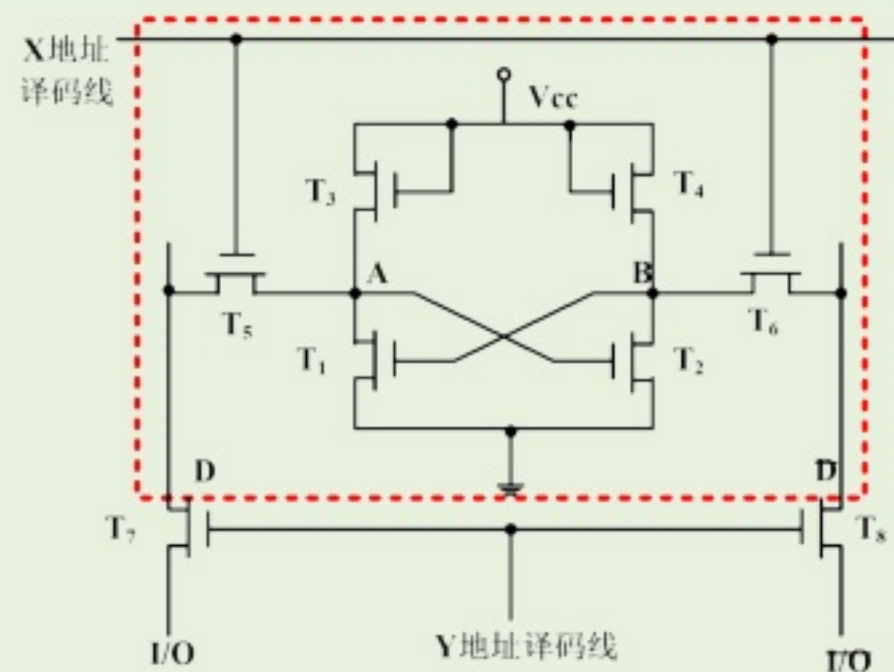
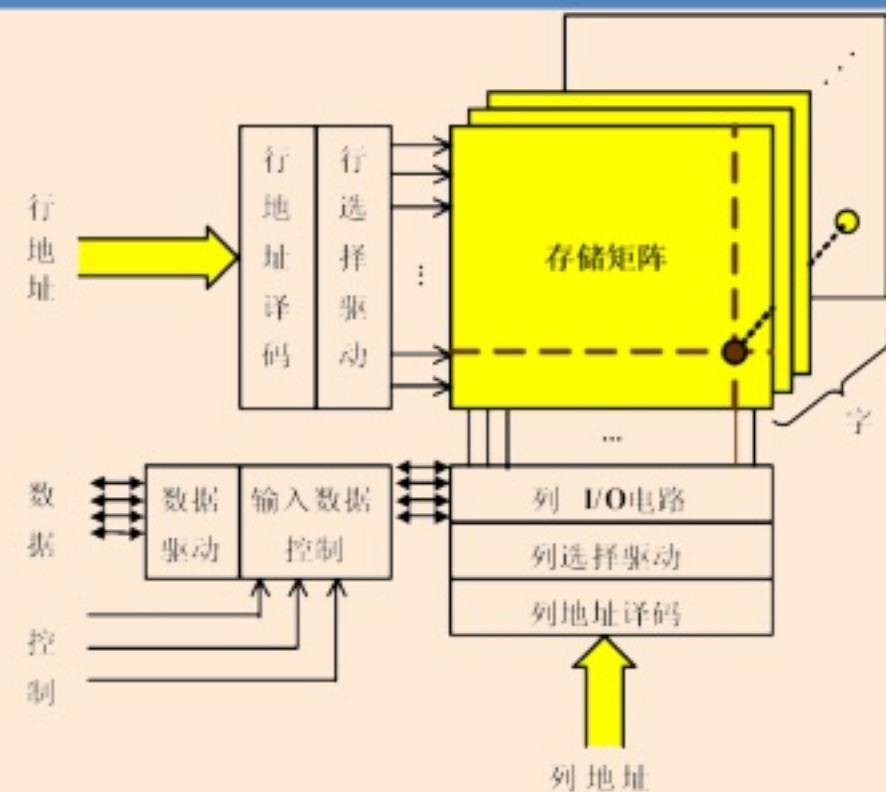
❖ 地址译码方式:

- ①线性译码方式:
- 存储矩阵是一维数组结构
- n 位地址线, 一维译码后, 有 2^n 根选择线
- 假设64个单元:
 - 地址线: $n=6$
 - 地址译码器: 6-64线译码器 ($n-2^n$)
 - 地址译码器输出的地址选择线有: 64根 (2^n)



(2) SRAM存储器

- ②双向译码方式:
- 存储矩阵是二维数组结构(阵列)
- n位地址分为行、列地址分别译码, 有 $2^{n/2+1}$ 根选择线
- 假设64个单元: 8×8 的存储阵列
 - 地址线 $n=6$
 - 行地址: 3位, 列地址: 3位
 - 行列地址译码器: 均为3-8译码器
 - 地址选择线: $2 \times 8 = 16$ 根
 - 选择线大大减少

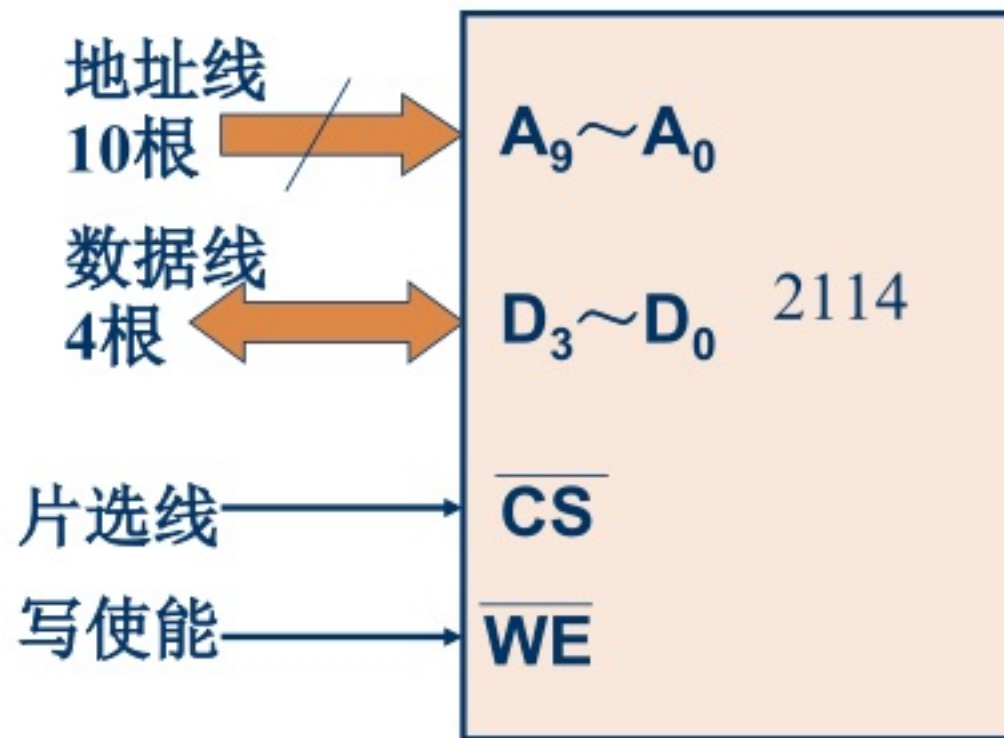




2114 SRAM存储器

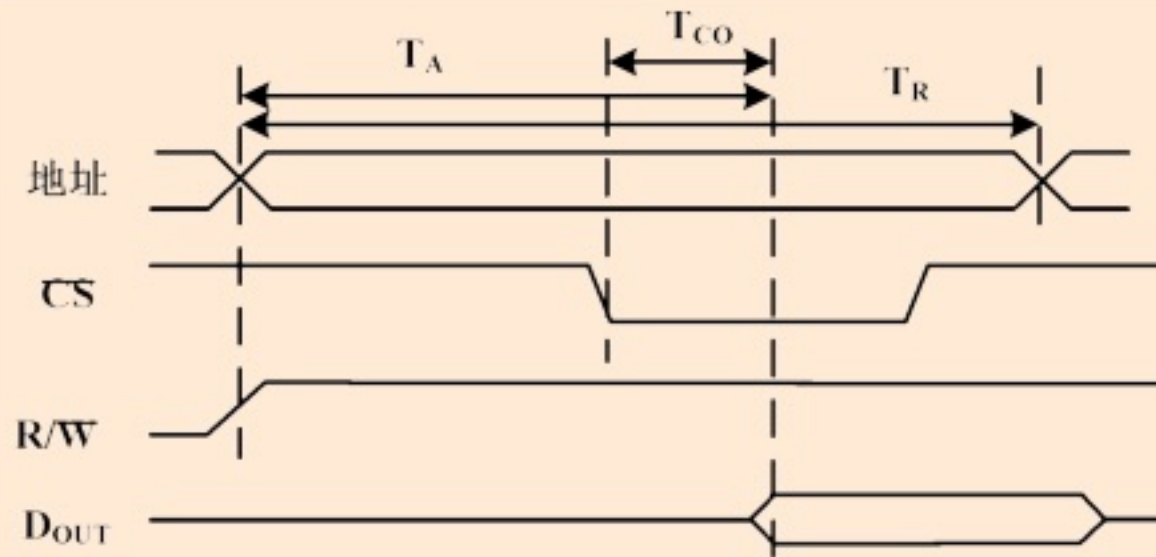
- ❖ 1K×4位
- ❖ 行地址：6位
- ❖ 列地址：4位
- ❖ 阵列：64×16×4

$\overline{CS}$	$\overline{WE}$	功能
H	×	无操作, D3-D0高阻抗
L	H	读操作, D3-D0为输出
L	L	写操作, D3-D0为输入

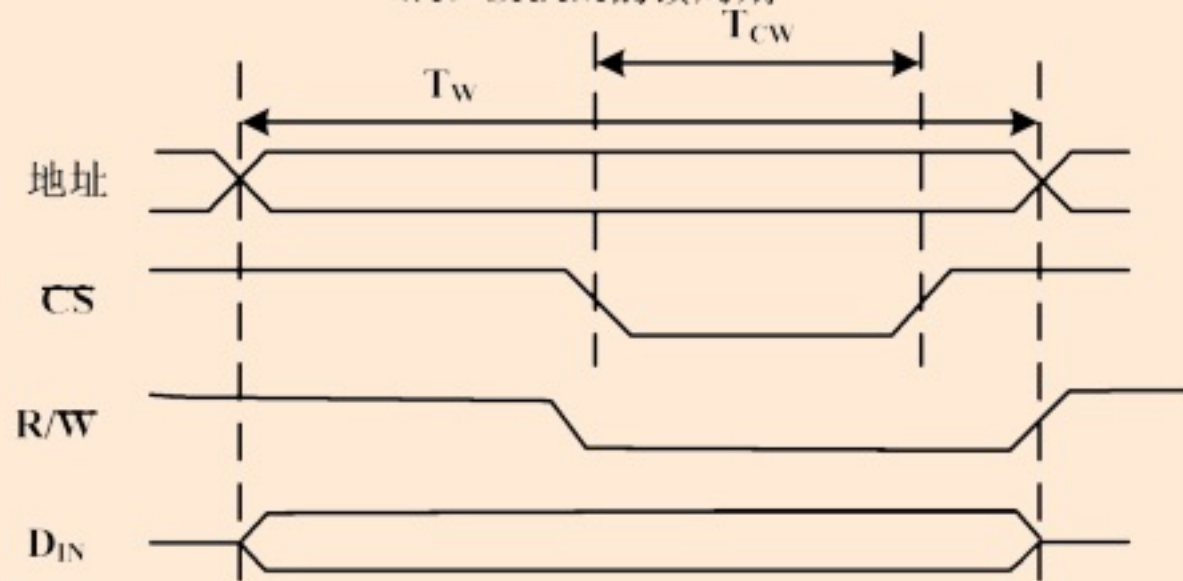




SRAM的读写时序



(A) SRAM的读周期



(B) SRAM的写周期

读操作:

地址有效

读信号有效
片选有效

数据输出

写操作:

地址有效
数据有效

写信号有效
片选有效





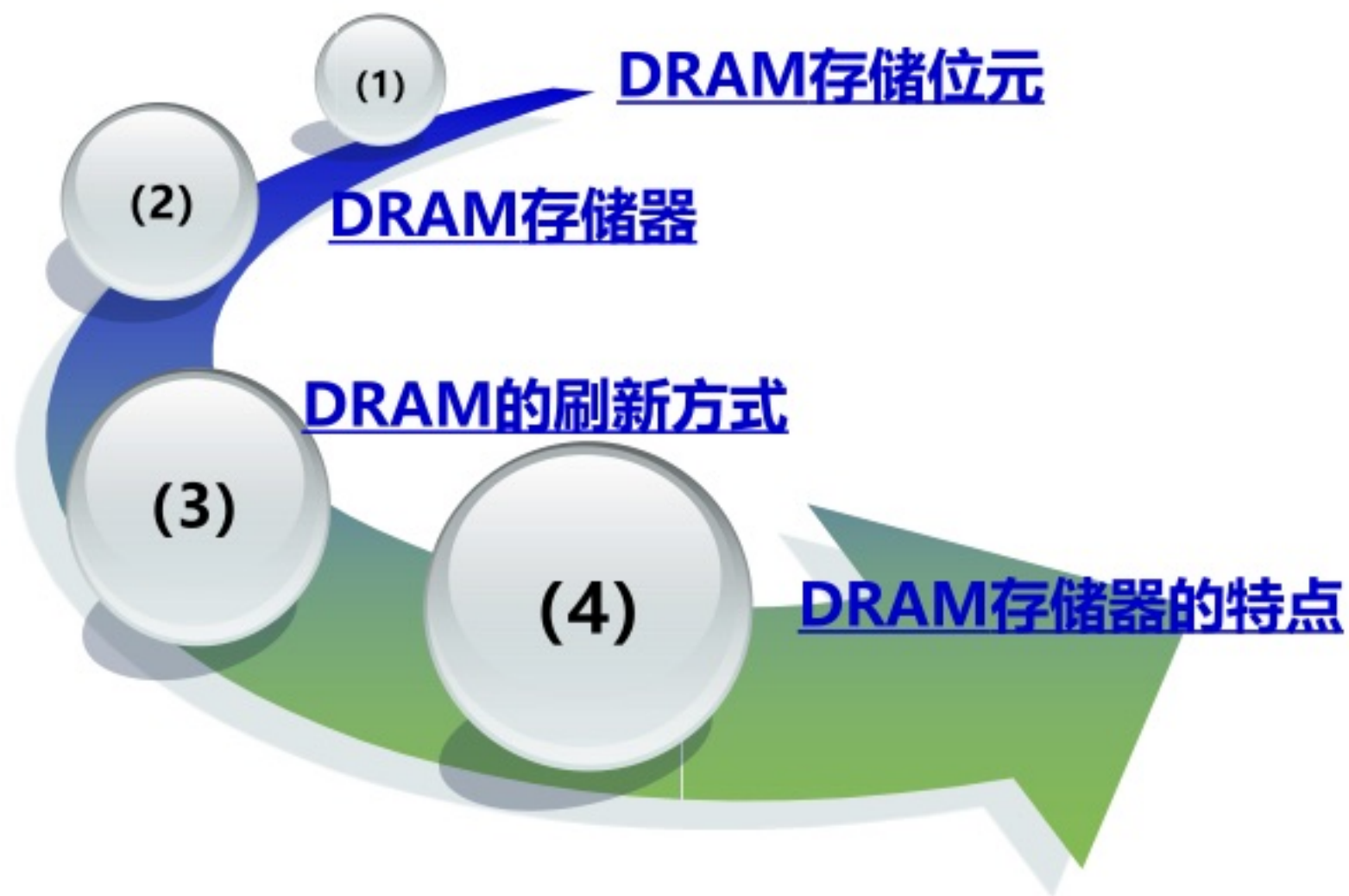
(3) SRAM存储器的特点

- ❖ 使用双稳态触发器表示0和1代码。
- ❖ 电源不掉电的情况下，信息稳定保持（静态）。
- ❖ 存取速度快，集成度低（容量小），价格高。
- ❖ 常用作高速缓冲存储器Cache。





2、动态存储器 (DRAM)





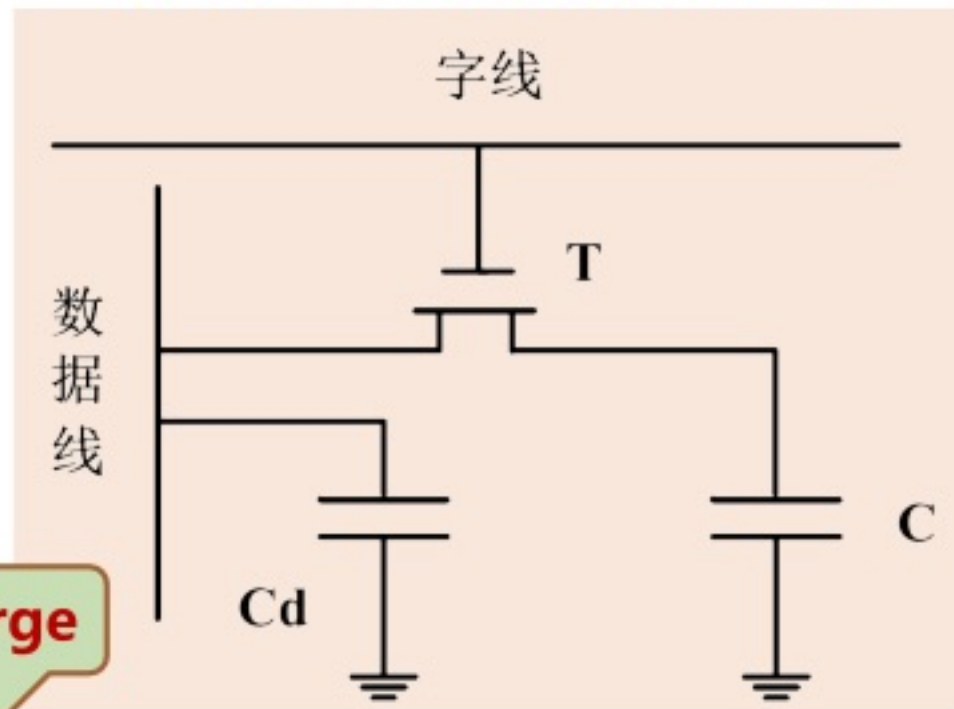
(1) DRAM存储位元

- ❖ “1”状态：电容C上有电荷
- ❖ “0”状态：电容C上无电荷
- ❖ 平时：字线=L，T截止，数据线与C隔离
- ❖ 读：字线=H，T导通
 - 读“0”：C上无电荷流入数据线
 - 读“1”：C放电，电荷流入数据线

破坏性读出

precharge

单管MOS动态存储器结构



- ❖ 写：字线=H，T导通
 - 写“0”：数据线=L，C若有电荷则放电
 - 写“1”：数据线=H，对C充电
- ❖ 再生：读出后信息可能被破坏，需要重写。
- ❖ 刷新：经过一段时间后，信息可能丢失，需要重写。





(2) DRAM存储器

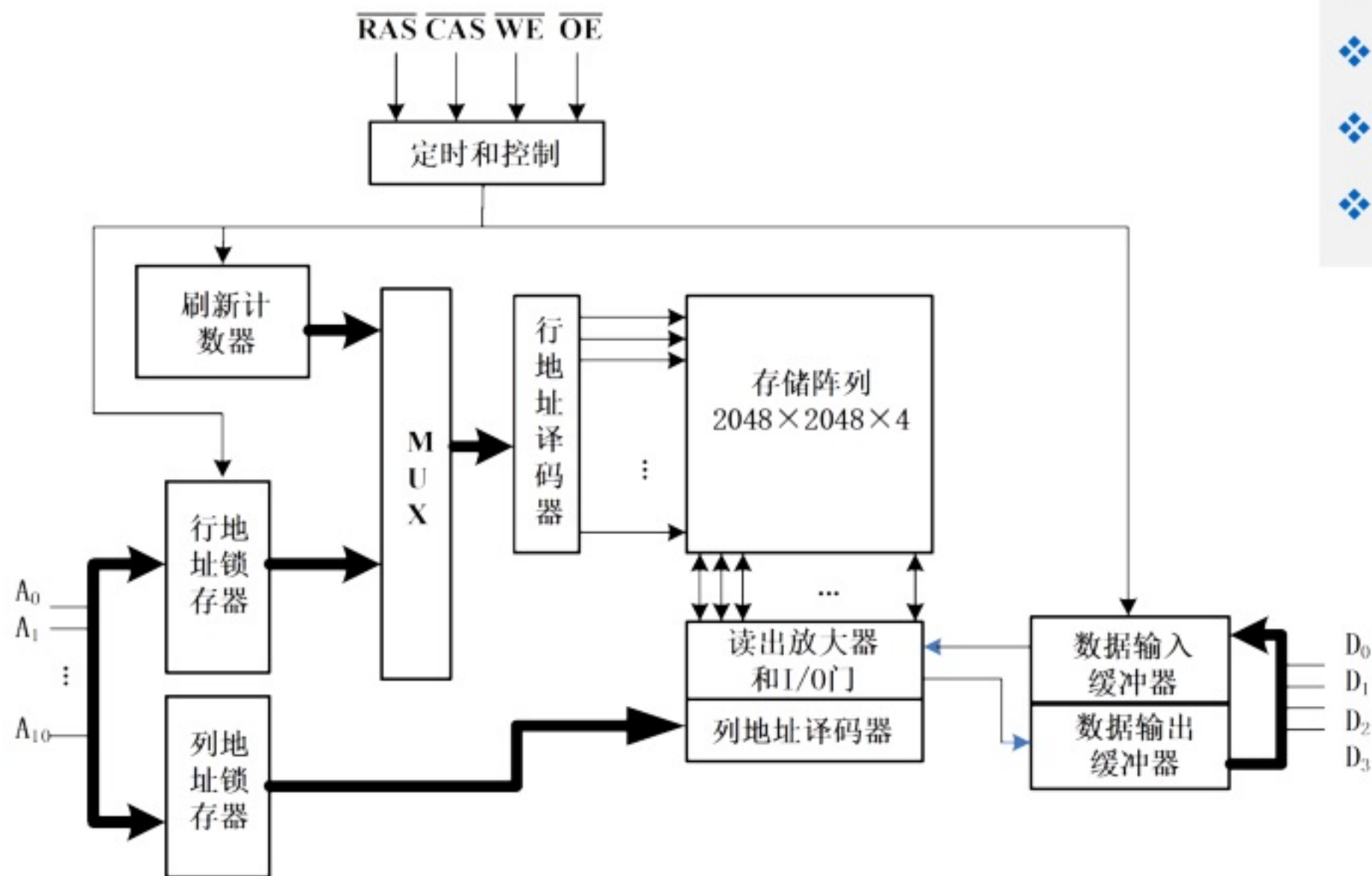
❖ DRAM的位单元扩展成存储器，与SRAM不同的地方：

- 采用双向译码 容量大
- 芯片地址引脚 A_i
 - 约为存储器地址码位数的一半
 - 行、列地址分时送入
 - 如何区分地址引脚上的地址是行地址还是列地址？
- 控制引脚：使用行地址选通信号 $\overline{RAS}$ 和列地址选通信号 $\overline{CAS}$ 来区分行列地址
 - 先送行地址，再送列地址；即 $\overline{RAS}$ 先有效， $\overline{CAS}$ 后有效
- 含刷新和再生电路



(2) DRAM存储器

4M×4位的DRAM

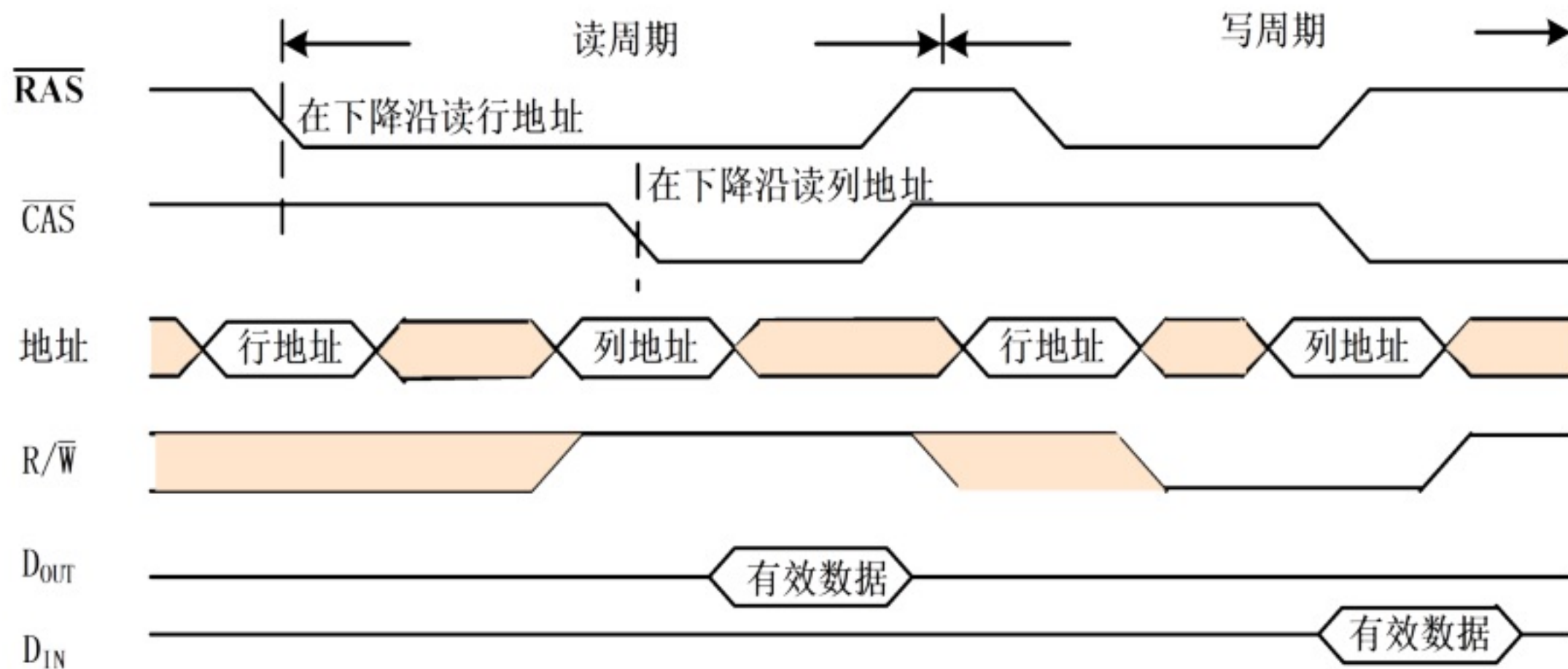


- ❖ 存储器单元地址几位?
- ❖ 芯片地址线几位?
- ❖ 为何只有一半?
- ❖ 刷新计数器几位?



DRAM的读/写过程

- ❖ 行地址选通信号 $\overline{RAS}$ 的下降沿时送行地址，列地址选通信号 $\overline{CAS\#}$ 的下降沿时送列地址，且必须在 $\overline{RAS}$ 有效后， $\overline{CAS}$ 才能有效。





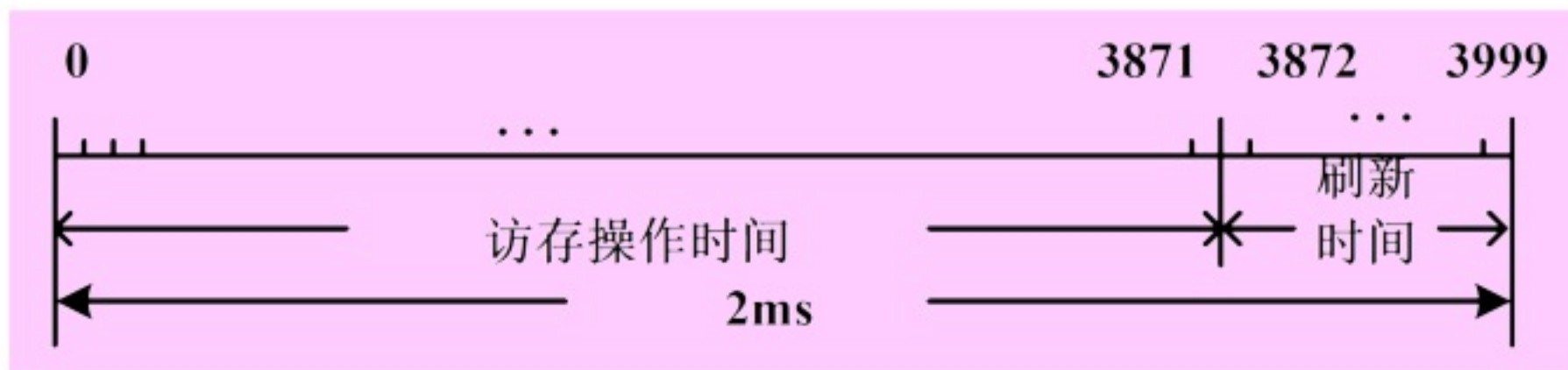
(3) DRAM的刷新方式

- ❖ **刷新操作：**即是按行来执行内部的读操作。
 - 由刷新计数器产生行地址，选择当前要刷新的行；
 - 读即刷新，刷新一行所需时间即是一个存储周期；读出的数据不送到外部总线。
- ❖ **单元刷新间隔时间 τ ：**DRAM允许的最大信息保持时间；一般 $\tau=2ms$ 。
 - 现在公认的时间是 $\tau=64ms$
- ❖ **刷新周期 T_{fresh} ：**从上一次刷新结束到下一次对整个DRAM全部刷新一遍为止的时间间隔；
 - 最优： $T_{fresh} = \tau = 2ms$
- ❖ **刷新行数：**单个芯片的单个矩阵的行数。
 - 对于内部包含多个存储矩阵的芯片：各个矩阵的同一行是被同时刷新的。
 - 对于多个芯片连接构成的DRAM：DRAM控制器每次选中所有芯片的同一行进行刷新
- ❖ **刷新方式：**集中式刷新、分散式刷新和异步式刷新。



①集中式刷新

- ❖ 例：64K×1位DRAM芯片中，存储电路由4个独立的128×128的存储矩阵组成。设存储器存储周期为500ns，单元刷新间隔是2ms。



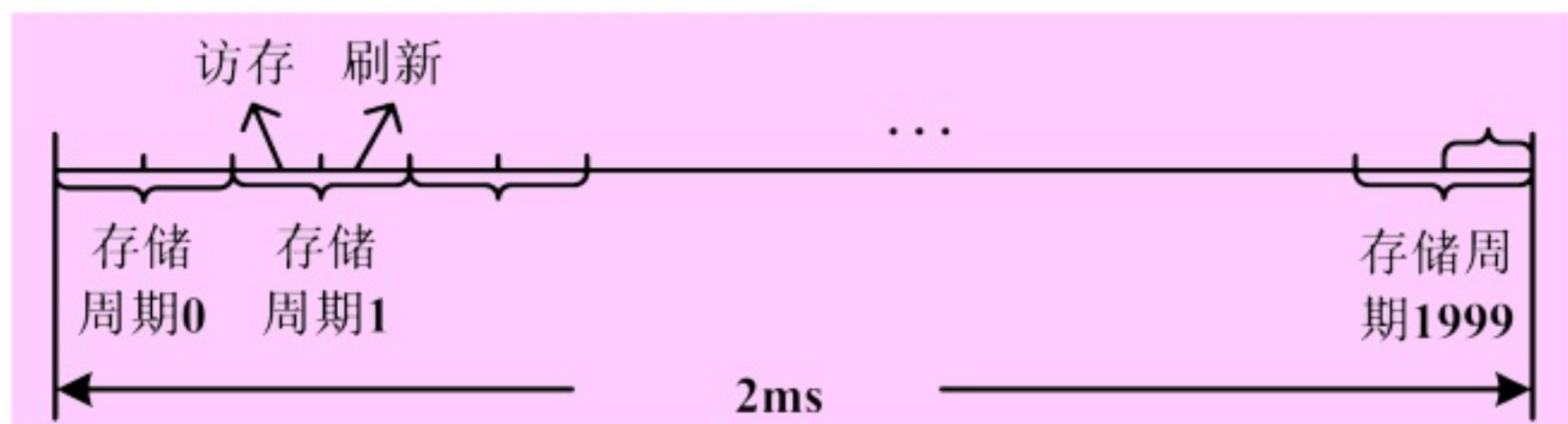
- ❖ 在2ms单元刷新间隔时间内，集中对128行刷新一遍，其余时间用于访问操作
- ❖ **集中刷新时间：** $128 \times 500\text{ns} = 64\mu\text{s}$
- ❖ **死时间：** 在内部刷新时间（64μs）内，**不允许访存。**



②分散式刷新

❖ 在任何一个存储周期内，分为访存和刷新两个子周期。

- 访存时间内，供CPU和其他主设备访问。
- 在刷新时间内，对DRAM的某一行刷新。



❖ 系统存储周期为存储器存储周期的两倍，即 $500\text{ns} \times 2 = 1\mu\text{s}$ 。

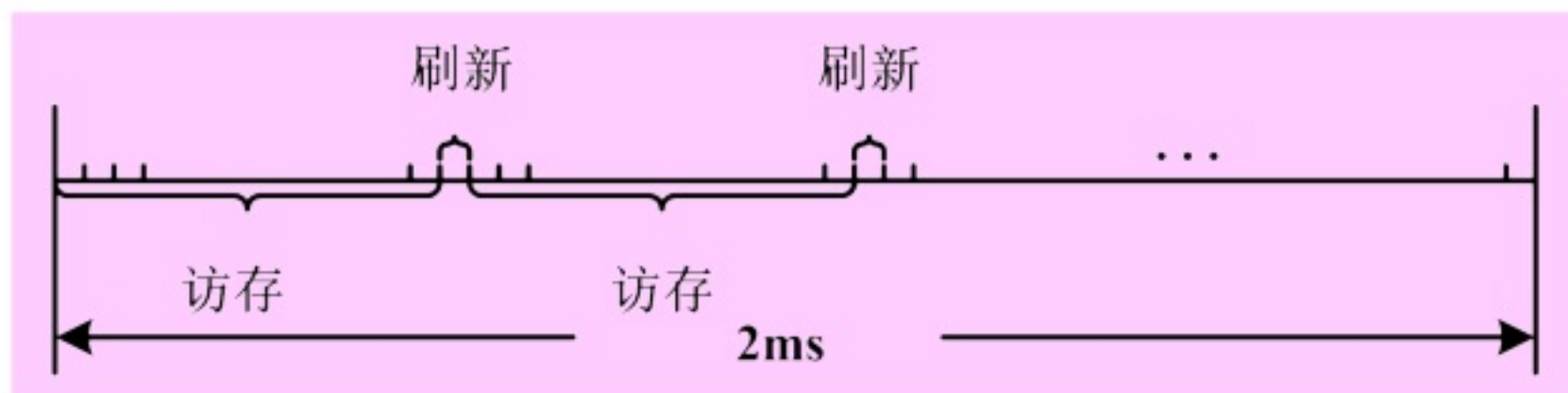
❖ 刷新周期缩短，为 $128 \times 1\mu\text{s} = 128\mu\text{s}$ 。

❖ 但是：在2ms的单元刷新间隔时间内，对DRAM刷新了 $2\text{ms} \div 128\mu\text{s} = 15$ 遍。



③异步式刷新

- ❖ 异步刷新采取折中的办法，在2ms内分散地把各行刷新一遍。
- ❖ 避免了分散式刷新中不必要的多次刷新，提高了整机速度；同时又解决了集中式刷新中“死区”时间过长的时间问题。



- ❖ 刷新电路：每隔 $15.625\mu\text{s}$ 产生一个刷新信号，刷新一行
- ❖ 刷新信号的周期： $2\text{ms}/128=15.625\mu\text{s}$





(4) DRAM存储器的特点

- ❖ 使用半导体器件中分布电容上有无电荷来表示0和1代码。
- ❖ **电源不掉电**的情况下，信息也会丢失，因此需要不断**刷新**。
- ❖ 存取速度慢，集成度高（容量大），价格低。
- ❖ 常用作内存条。





3、SRAM和DRAM的对比

比较内容	SRAM	DRAM
存储信息0和1的方式	双稳态触发器	极间电容上的电荷
电源不掉电时	信息稳定（静态）	信息会丢失（动态）
刷新	不需要	需要
集成度	低	高
容量	小	大
价格	高	低
速度	快	慢
适用场合	Cache	主存
易失性	是	是





二、只读存储器ROM

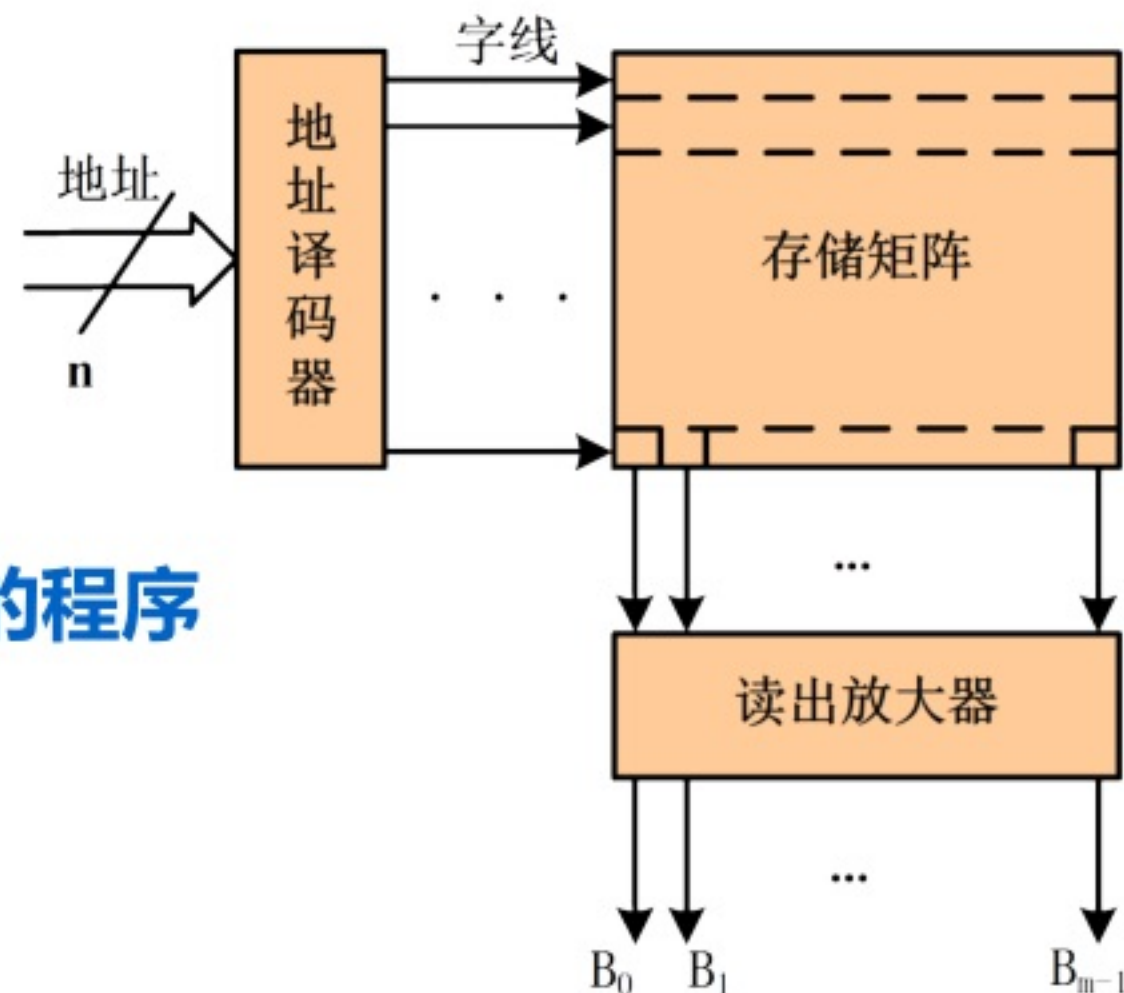
❖ ROM (Read-only Memory)

❖ 可以作为主存

❖ 一般结构：

- 随机存取

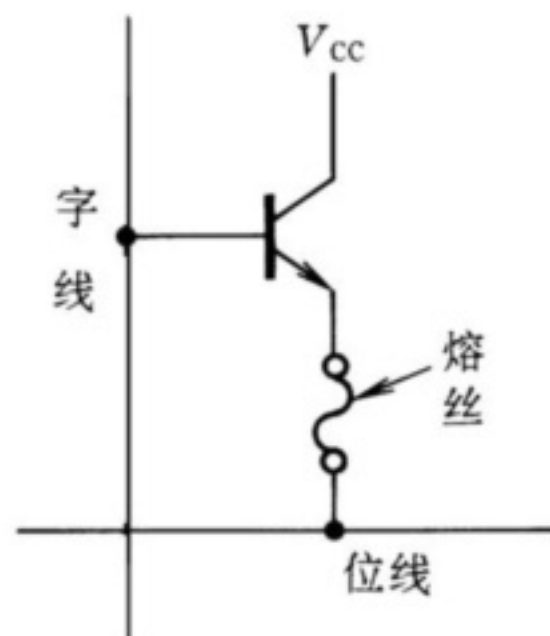
❖ 固件 (firmware) : 存储在ROM中的程序





二、只读存储器ROM

- ❖ ①MROM (Masked-ROM) : 掩膜式ROM
 - 工厂定制
- ❖ ②PROM (Programmable-ROM) : 可编程ROM
 - 熔丝型, 一次可编程, 出厂时默认全1
 - 用户对其编程: 用烧写工具写入0
- ❖ ③EPROM (Erasable-PROM) : 可擦除可编程ROM
 - 紫外线擦除: 擦除器
 - 大电流烧写编程: 烧写器, 软件+硬件
 - 脱机改写: 需要拆卸下芯片, 擦除、编程后再安装回去



EPROM存储器
27C64(8K×8bits)



二、只读存储器ROM

❖ ④E²PROM (Electrically-EPROM) : 电可擦除可编程ROM

- 电擦除、电改写
- 在线改写：在板卡上直接改写
- 编程电压24V，读出电压5V

E²PROM
28C64
(8K×8bits)

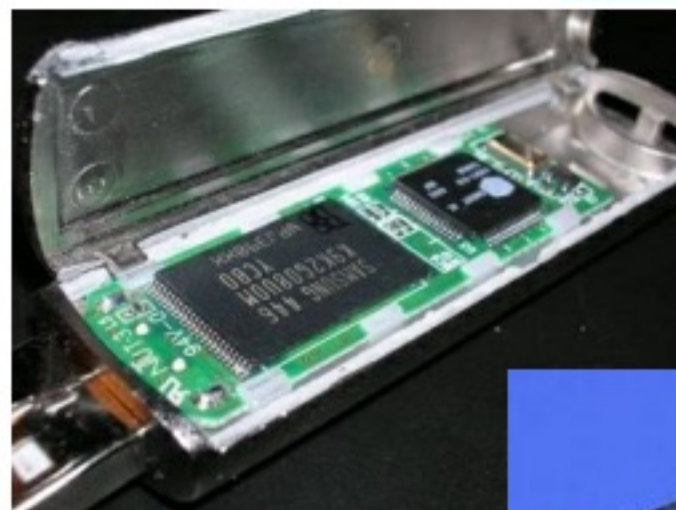


❖ ⑤Flash Memory: 闪存

- 基于E²PROM原理改造，擦除速度很快
- 写入方法不同，写入前需要擦除：按块改写

❖ 分为NOR、NAND两种类型

- NOR Flash: 可以用做存放BIOS与启动程序/引导程序的固件，内存的一部分
- NAND Flash: U盘、SSD、移动硬盘





几种非易失性存储器的比较

存储器	类别	擦除方式	能否单字节修改	写机制
MROM	只读	不允许	否	掩膜位写
PROM	写一次读多次	不允许	否	电信号
EPROM	写多次读多次	紫外线擦除	否	电信号 脱机改写
E ² PROM	写多次读多次	电擦除	能	电信号 在线改写
Flash Memory	写多次读多次	电擦除	否	电信号 在线改写





三、高性能的主存储器

- ❖ 集成度越来越高、容量越来越大、速度越来越快
- ❖ EDO RAM：“扩展数据输出RAM”（1996-1999）
 - 在完成当前内存周期前即可开始下一内存周期操作，所以在大量存取操作时，可以大大地缩短存取时间
- ❖ EDRAM：增强型DRAM
 - 芯片内集成了一小块Cache（SRAM），保存最后一次读操作所在行的全部内容
 - 刷新操作能够与Cache读操作并行进行
- ❖ CDRAM：带Cache的DRAM
 - 包含的Cache比EDRAM更大，既能够作为真正的Cache使用，又可以用作支持串行存取数据块的缓冲器



三、高性能的主存储器

❖ RDRAM (Rambus DRAM) :1990年-1992-1996

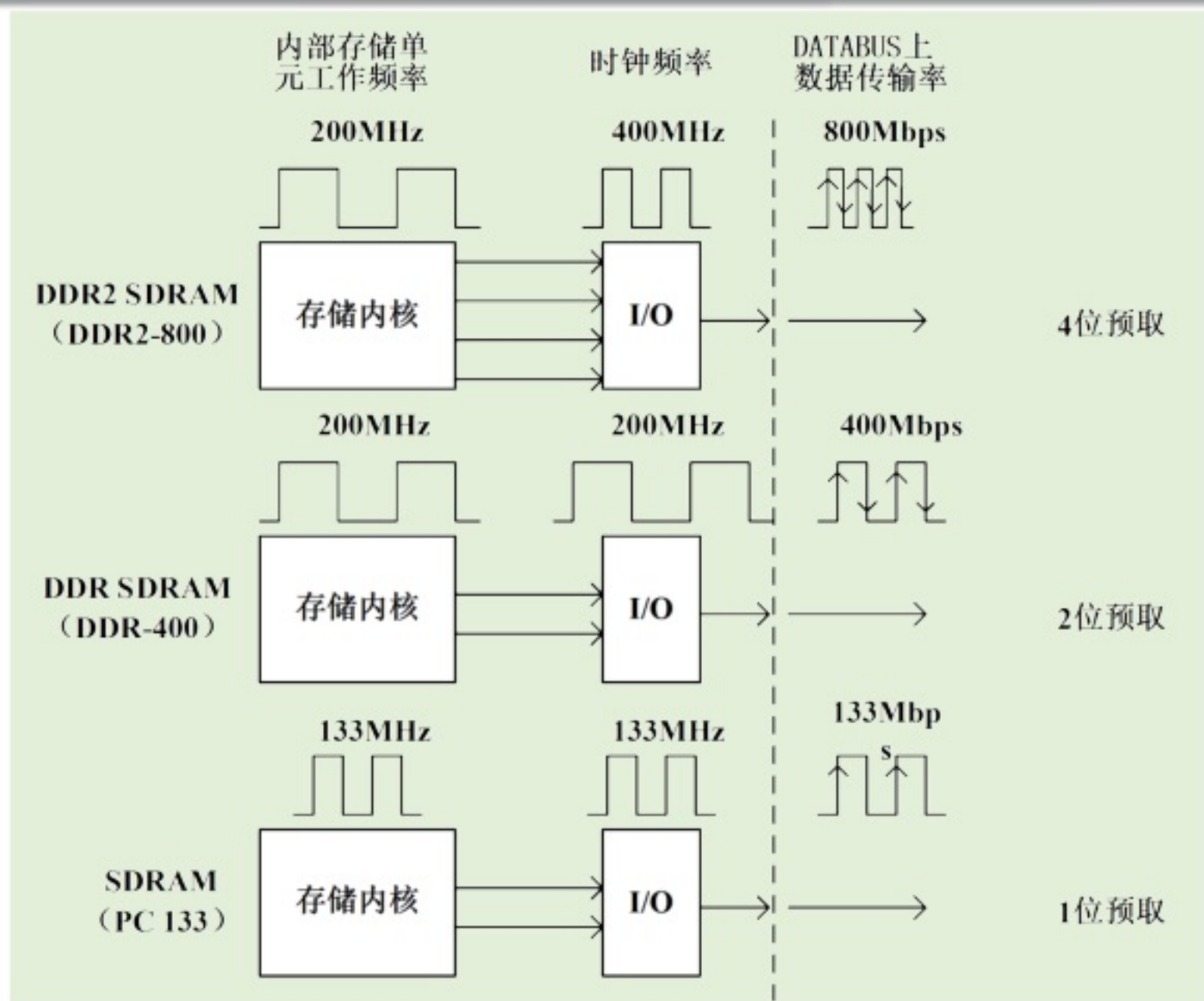
- 采用垂直封装，所有引脚都在一边，使得存储器的装配非常紧凑。
- 它与CPU之间传送数据是通过专用的RDRAM高速总线进行的，总线最多能寻址320块RDRAM芯片，传输率可达到500Mb/s

❖ DDR SDRAM (双倍速率SDRAM)，简称DDR。(1997-)

- 能在时钟触发沿的上、下沿进行数据传输
- DDR1支持预取2位，DDR2支持预取4位
- 制造成本比SDRAM略高一些，远小于Rambus的价格



SDRAM、DDR1-400和DDR2-533三者比较





5.3 主存储器与CPU的连接

- 一 背景知识——存储芯片简介
- 二 存储器容量扩展的三种方法
- 三 主存储器与CPU的连接





一、背景知识——存储芯片简介

❖ ①SRAM存储芯片的引脚封装

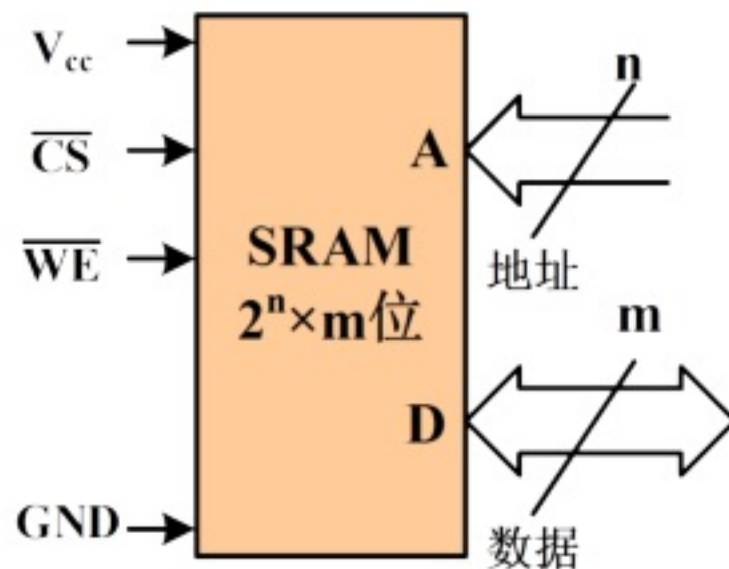
❖ 地址不复用

❖ 芯片容量： $2^n \times m$ 位

- 地址： n 位
- 数据： m 位
- 片选： $\overline{CS}$
- 写使能： $\overline{WE}$
- 电源和地

❖ 6116芯片： $2K \times 8$ 位

- 地址：11位
- 数据：8位
- 片选： $\overline{CS}$
- 写使能： $\overline{WE}$
- 输出使能： $\overline{OE}$
- 电源和地



$\overline{CS}$	$\overline{WE}$	$\overline{OE}$	功能
H	x	x	无操作, I/O线高阻抗
L	H	L	读操作, I/O线为输出
L	L	x	写操作, I/O线为输入



一、背景知识——存储芯片简介

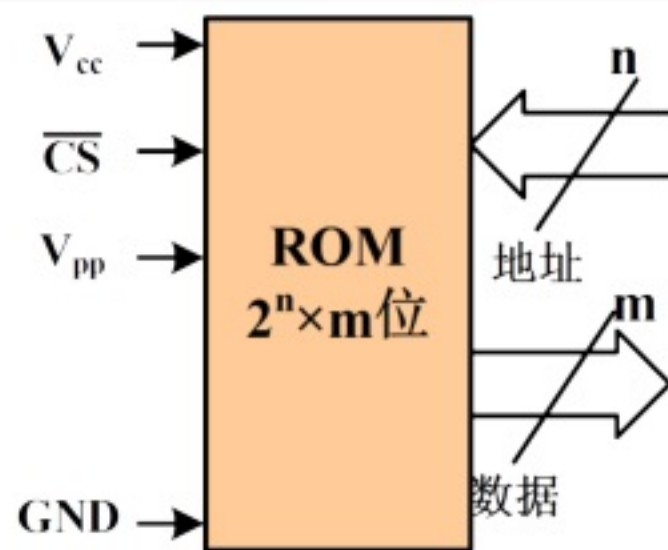
❖ ②ROM存储芯片的引脚封装

❖ 随机存取的ROM：地址不复用

- Flash构成的SSD：磁盘寻址

❖ 芯片容量： $2^n \times m$ 位

- 地址： n 位
- 数据： m 位
- 片选： $\overline{CS}$
- 电源 V_{cc} ：工作电源 (3V/5V)
- 电源 V_{pp} ：编程电源 (12V/24V)



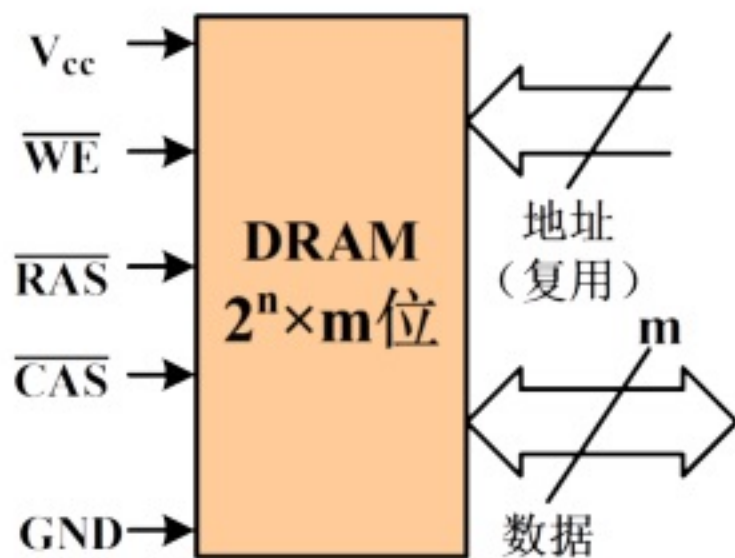
❖ M28C64芯片：8K×8位 EEPROM

- 地址(in)：13位
- 数据(inout)：8位
- 片选(in)： $\overline{CS}$
- 写使能(in)： $\overline{WE}$
- 输出使能(in)： $\overline{OE}$
- 就绪/忙状态(out)： $R/\overline{B}$



一、背景知识——存储芯片简介

❖ ③ DRAM存储芯片的引脚封装



❖ 地址复用

❖ 芯片容量: $2^n \times m$ 位

- 地址: 约为 $n/2$ 位
- 数据: m 位
- 行地址选通: $\overline{RAS}$
- 列地址选通: $\overline{CAS}$
- 写使能: $\overline{WE}$





二、存储器容量扩展的三种方法

❖ 问题1：为什么要扩展？

- 容量不够大：例如 DRAM芯片颗粒→内存条
 - 存储单元的位数不够：位扩展
 - 存储单元的字数不够：字扩展

❖ 以SRAM和ROM为例介绍扩展方法

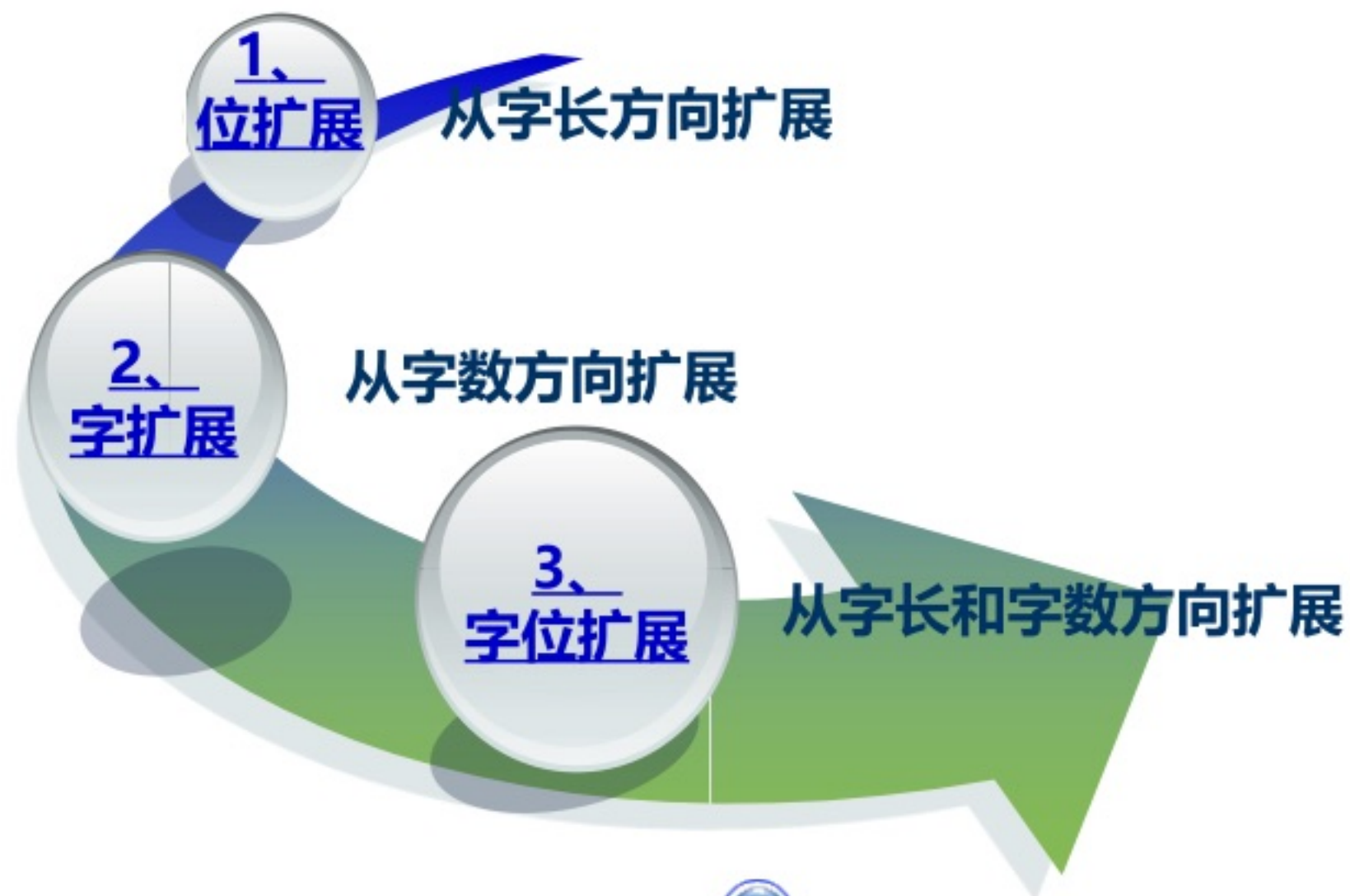
❖ 问题2：为什么以SRAM和ROM为例介绍？

- 地址不复用，相对简单；
- DRAM比较复杂：DRAM的扩展，通常由DRAM控制器来负责连接，并控制实现刷新



二、存储器容量扩展的三种方法

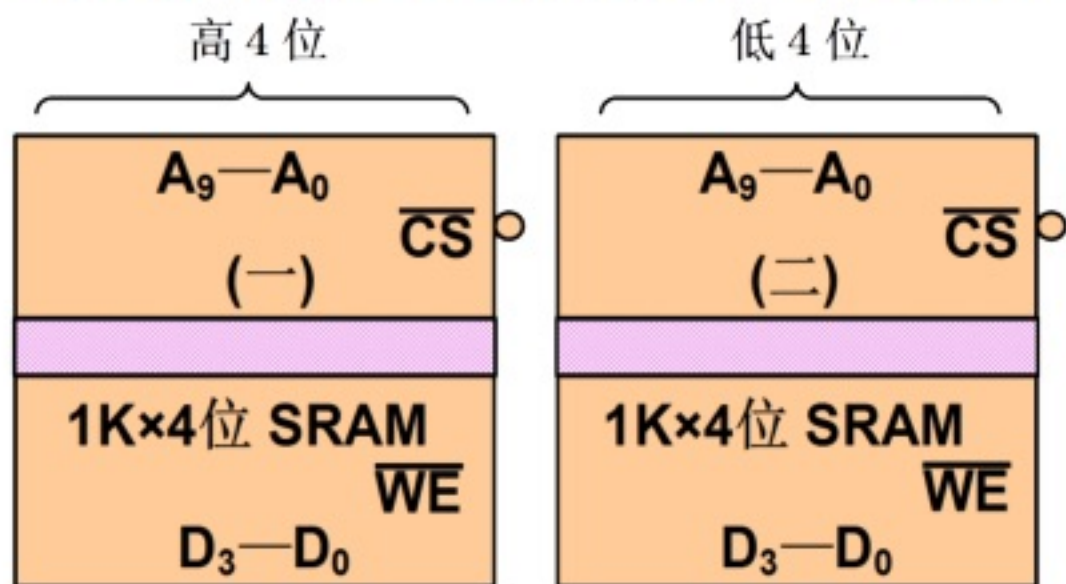
❖ 以SRAM和ROM为例介绍





1、位扩展

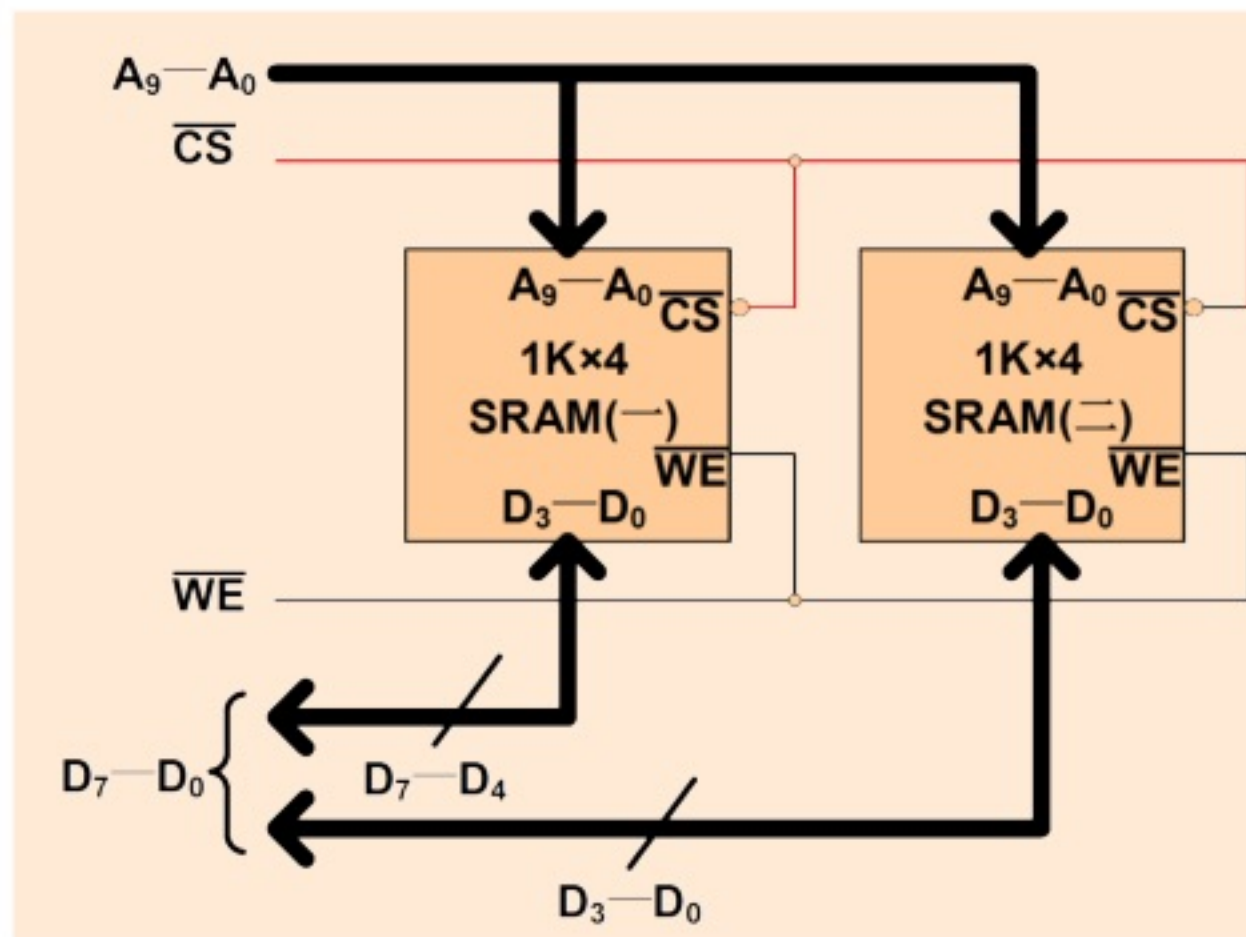
❖ 要求：用1K×4位的SRAM芯片 → 1K×8位的SRAM存储器



❖ 容量 = $2^{10} \times 8$ 位

❖ 读地址为000H的存储单元的内容？

❖ 读地址为3FFH的存储单元的内容？





1、位扩展

❖ 各芯片连接要点:

- ①地址线A: 连接在一起;
- ②读写控制信号 $\overline{WE}$: 连接在一起;
- ③片选信号 $\overline{CS}$: 连接在一起;
- ④数据线D: 并联后, 作为存储器外部数据总线

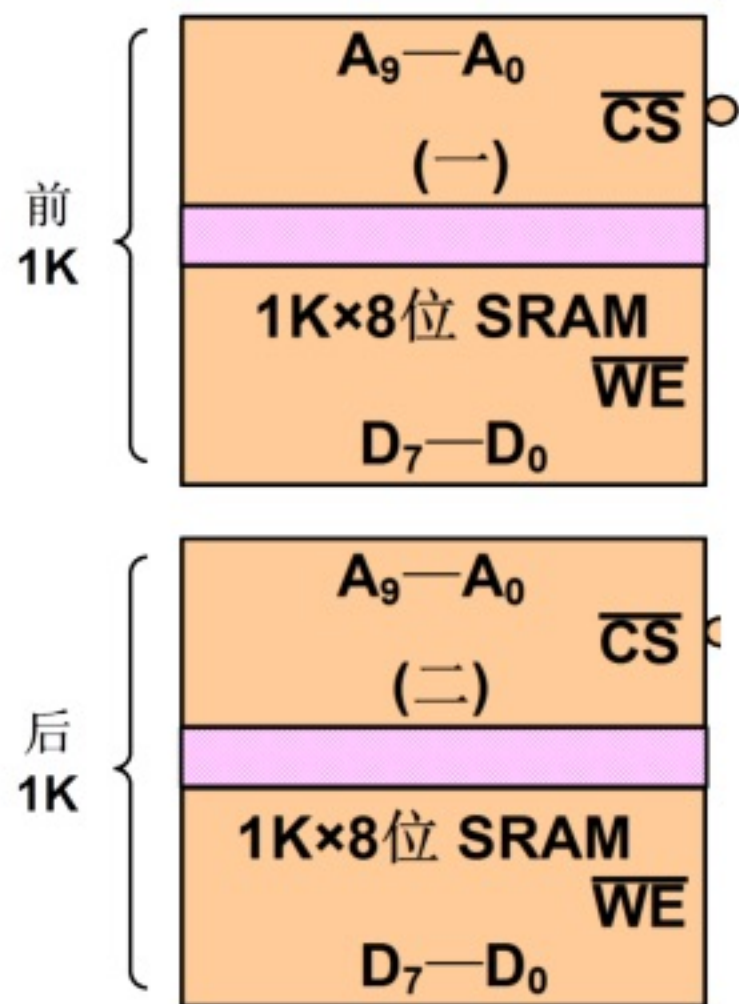
❖ 即: 位并联





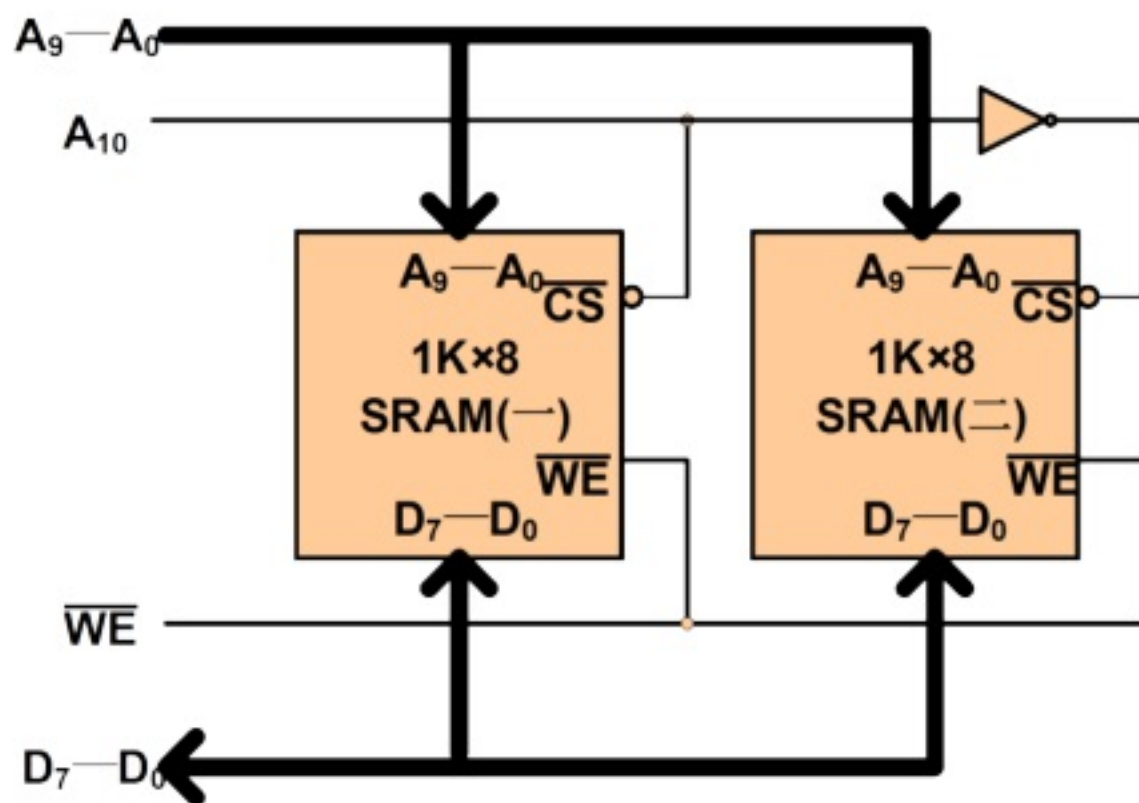
2、字扩展

❖要求：用1K×8位的SRAM芯片 → 2K×8位的SRAM存储器



❖分析地址：

- A_{10} 用于选择芯片
- $A_9 \sim A_0$ 用于选择芯片内的某一存储单元



A_{10}	A_9	$\sim$	A_0	
0	0	$\sim$	0	前 1K
$\vdots$	$\vdots$	$\vdots$	$\vdots$	
0	1	$\sim$	1	
1	0	$\sim$	0	后 1K
$\vdots$	$\vdots$	$\vdots$	$\vdots$	
1	1	$\sim$	1	

- ❖容量 = $2^{11} \times 8$ 位
- ❖读地址为000H的存储单元
- ❖读地址为400H的存储单元



2、字扩展

❖ 各芯片连接要点:

- ①数据线D: 连接在一起;
- ②地址线A: 连接在一起, 作为存储器地址线的低位;
- ③读写控制信号 $\overline{WE}$: 连接在一起;
- ④片选信号 $\overline{CS}$: 由存储器地址线剩余高位, 通过译码器来产生。

❖ 即: 地址串联





3、字位扩展

❖ 需扩展的存储器容量为 $M \times N$ 位，已有芯片的容量为 $L \times K$ 位 ($L < M, K < N$)

芯片个数 =

$$\frac{M \times N}{L \times K}$$

❖ 分 M/L 组，每组 N/K 个芯片

❖ 每组： N/K 个芯片进行位扩展

❖ M/L 组：进行字扩展



例：1K×4位SRAM→4K×8位

❖ ①芯片个数：

- $4K \times 8\text{位} / 1K \times 4\text{位} = 4 \times 2 = 8\text{片}$
- 2片1组，共4组

❖ ②地址、数据线分析：

- 存储器地址：12位 $A_{11}-A_0$ ($\because 4K = 2^{12}$)
- 芯片地址：10位 A_9-A_0 ($\because 1K = 2^{10}$)
- 数据线：存储器8位 D_7-D_0 ，芯片4位 D_3-D_0

❖ ③每2片并联成为1组（1K×8位），8片共4组

- 每组2块芯片的地址线 A_9-A_0 （10根）连接在一起
- 每组2块芯片的数据线 D_3-D_0 （4根）并联为8位 D_7-D_0
- 每组2块芯片的片选信号连接在一起、读写信号连接在一起

❖ ④4组进行地址串联

- 4组芯片的数据线（8根） D_7-D_0 连接在一起，作为存储器的数据线 D_7-D_0
- 4组芯片的地址线（10根）连接在一起，作为存储器的地址线低10位 A_9-A_0
- 存储器高2位地址（ $A_{11}A_{10}$ ）经过2:4译码器，输出的4根线分别作为4组的片选信号
- 4组芯片的读写信号连接在一起



例：1K×4位SRAM→4K×8位

❖ ⑤与CPU连接

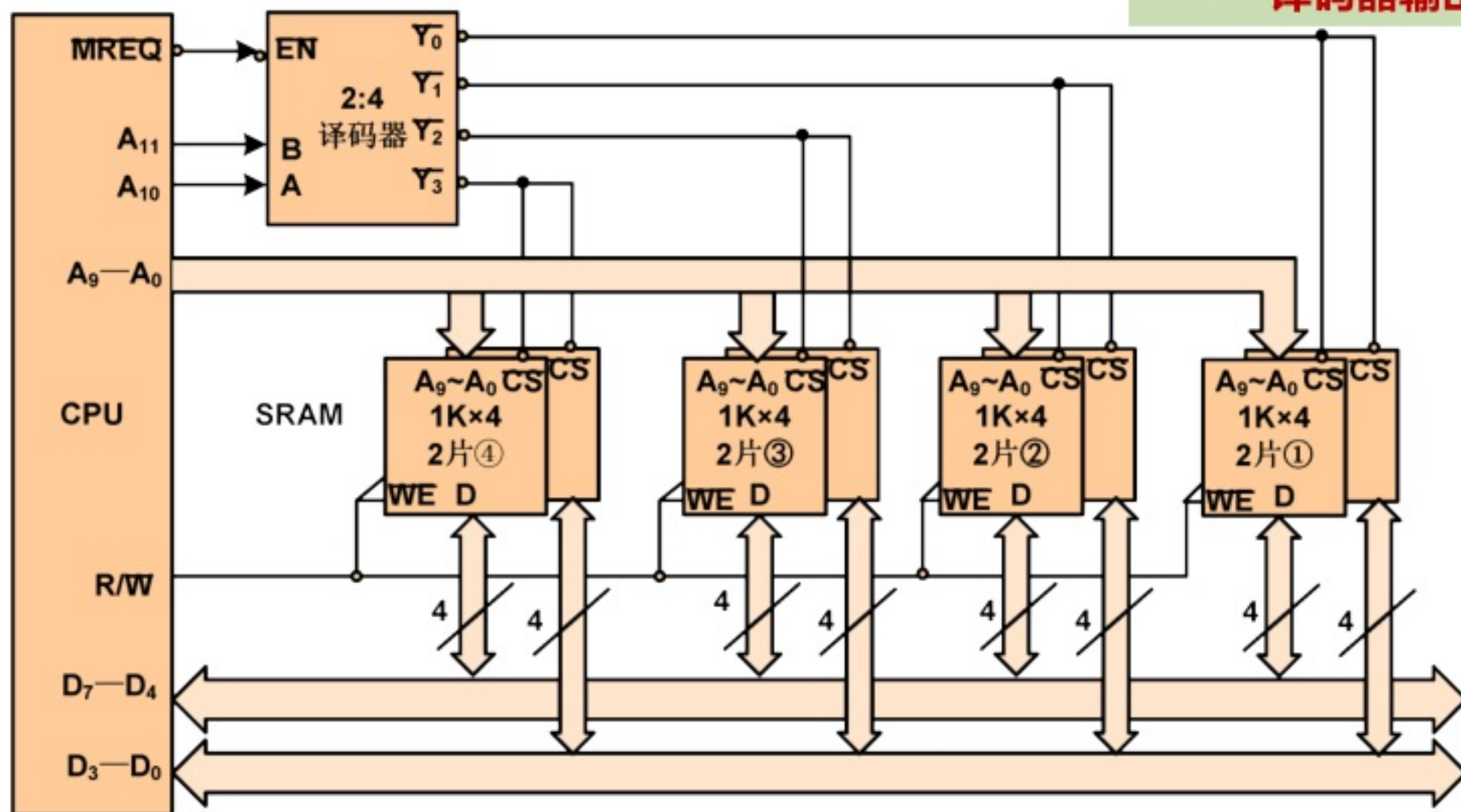
- CPU通常有：
 - 等数量的**数据线**；
 - 读写控制信号 $R/\overline{W}$ (=1: 读, =0: 写)；
 - 存储器访问请求信号 $\overline{MREQ}$ (=0: 访问存储器, 此时AB上是存储器单元地址; =1: 不访问存储器)；
 - **地址线**: 可能多于存储器的地址线, 因为CPU可以访问的存储器地址空间, 往往多于真正的物理主存。
- 数据线、读写控制信号线: 直接连接
- 地址线: 一样多, 直接连接;
- $\overline{MREQ}$: 作为**2:4译码器的使能控制端**, $\overline{MREQ} = 0$, 则译码器工作, 某组的 $\overline{CS} = 0$, 该组被选中工作, 由低10位地址码选中其中的某个存储单元进行读写工作; 如果 $\overline{MREQ} = 1$, 译码器不工作, 则4组的 $\overline{CS}$ 均=1, 都不工作, 即存储器不工作。



例：1K×4位SRAM→4K×8位

❖ 2:4译码器的特点：

- 使能端为低电平有效；
- 译码器输出也是低电平有效





三、主存储器与CPU的连接

- ❖ 1、根据CPU芯片提供的地址线数目，确定CPU访存的地址范围，并写出相应的二进制地址码；
- ❖ 2、根据地址范围的容量，确定各种类型存储器芯片的数目和扩展方法；
- ❖ 3、分配CPU地址线。**CPU地址线的低位**（数量 = 存储芯片的地址线数量）直接连接存储芯片的地址线；**CPU高位地址线**皆参与形成存储芯片的**片选信号**；
- ❖ 4、连接数据线、R/W#等其他信号线，MREQ#信号一般可用作地址译码器的使能信号。
- ❖ 需要说明的是，主存的扩展及与CPU连接在做法上并不唯一，应该具体问题具体分析



例5 - 1

- ❖ 设CPU有16根地址线，8根数据线，并用MREQ#作访存控制信号（低电平有效），用R/W#作读/写控制信号（高电平为读，低电平为写）。现有下列存储芯片：1K*4位SRAM；4K*8位SRAM；8K*8位SRAM；2K*8位ROM；4K*8位ROM；8K*8位ROM；及3：8译码器和各种门电路。
- ❖ 要求：主存的地址空间满足下述条件：最小8K地址为系统程序区（ROM区），与其相邻的16K地址为用户程序区（RAM区），最大4K地址空间为系统程序区（ROM区）。
- ❖ 请画出存储芯片的片选逻辑，存储芯片的种类、片数
- ❖ 画出CPU与存储器的连接图。



首先根据地址范围写出相应的二进制地址码

A ₁₅	A ₁₄	A ₁₃	A ₁₂	A ₁₁	A ₁₀	A ₉	A ₈	A ₇	A ₆	A ₅	A ₄	A ₃	A ₂	A ₁	A ₀	
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	} 最小8K 系统区
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	
0	0	0	1	1	1	1	1	1	1	1	1	1	1	1	1	} 相邻 16K 用户程 序区
0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	} 相邻 16K 用户程 序区
0	0	1	1	1	1	1	1	1	1	1	1	1	1	1	1	
0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	} 相邻 16K 用户程 序区
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	
0	1	0	1	1	1	1	1	1	1	1	1	1	1	1	1	} 相邻 16K 用户程 序区
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	
1	1	1	1	0	0	0	0	0	0	0	0	0	0	0	0	} 最大4K 系统区
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	
1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	} 最大4K 系统区
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	



❖ 第二步：选择芯片

- 最小8K系统程序区 $\leftarrow$ 8K*8位ROM, 1片
- 16K用户程序区 $\leftarrow$ 8K*8位SRAM, 2片;
- 4K系统程序工作区 $\leftarrow$ 4K*8位ROM, 1片。

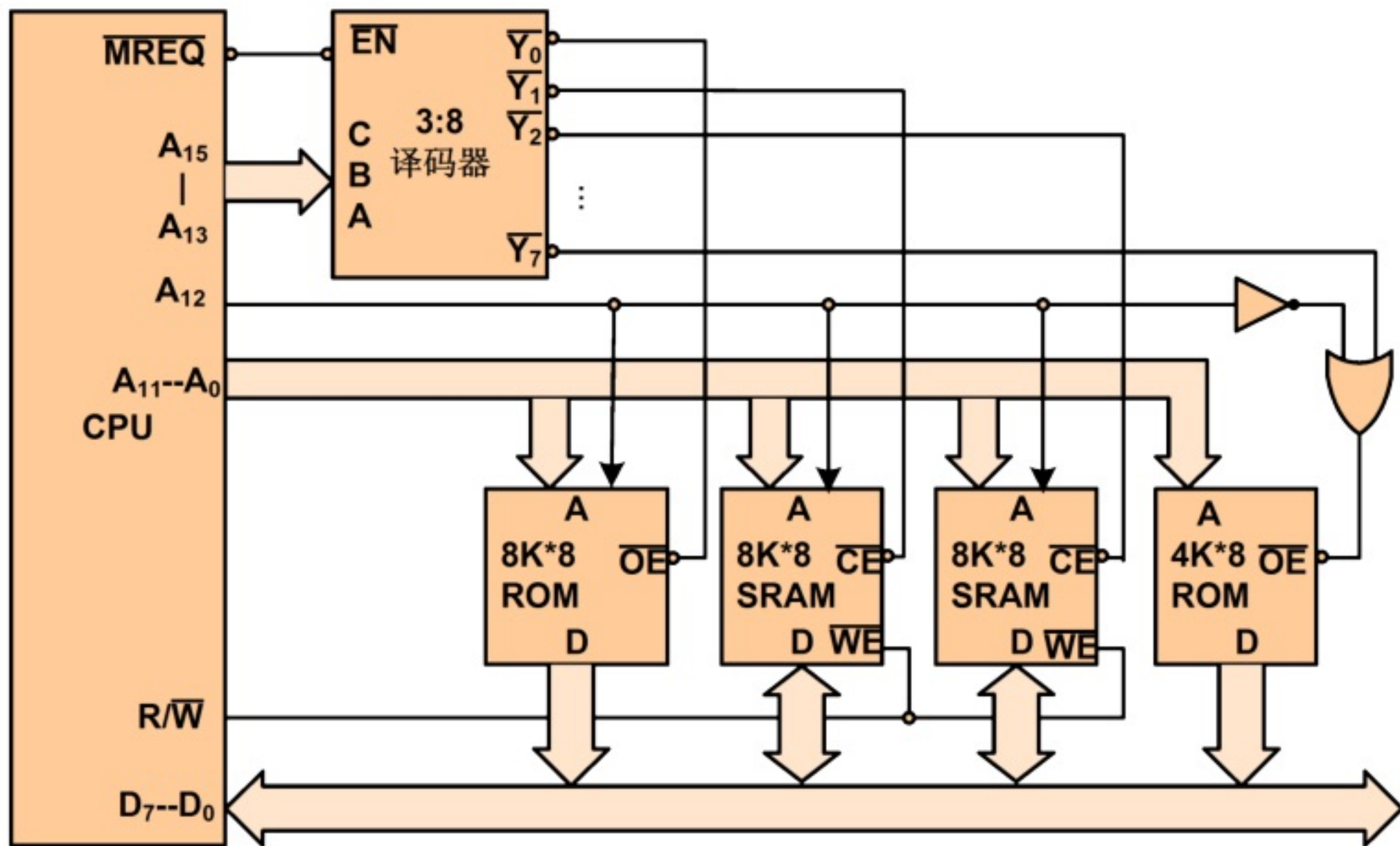
❖ 第三步，分配CPU地址线。

- CPU的低13位地址线 $A_{12} \sim A_0$ 与1片8K*8位ROM和两片8K*8位SRAM芯片提供的地址线相连；将CPU的低12位地址线 $A_{11} \sim A_0$ 与1片4K*8位ROM芯片提供的地址线相连。

❖ 第四步，译码产生片选信号。



连接图



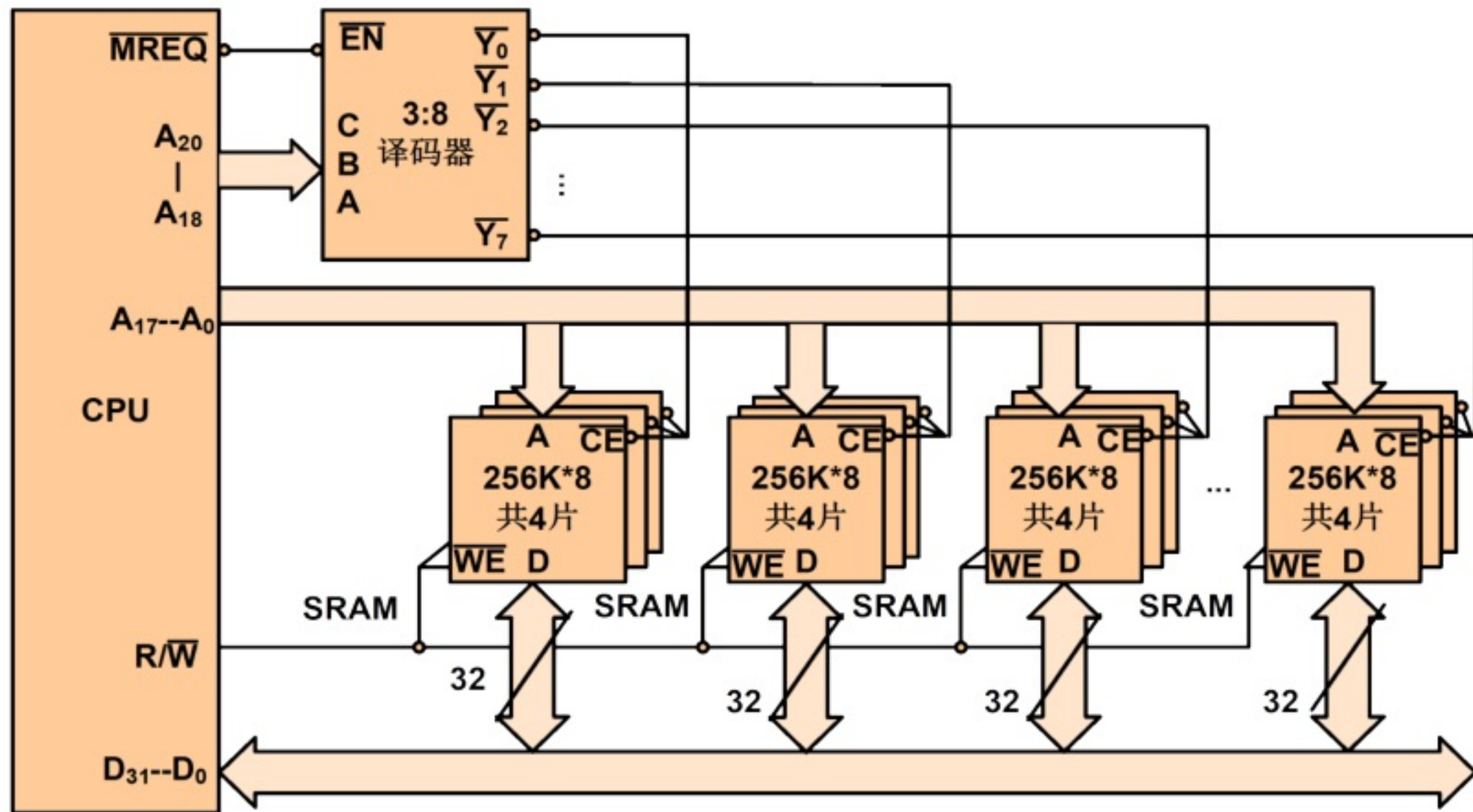


例5 - 2

- ❖ 设有若干片256K×8位的SRAM芯片，问如何构成2048K×32位的存储器？需要多少片RAM芯片？该存储器需要多少根地址线？画出该存储器与CPU连接的结构图，设CPU的接口信号有地址信号、数据信号、控制信号MREQ#和R/W#。
- ❖ 解：采用字位扩展的方法。
- ❖ SRAM芯片个数： $2048K/256K \times 32/8 = 32$ 片
- ❖ 每4片一组进行位扩展，共8组芯片进行字扩展
- ❖ 片选：该存储器需要21条地址线 $A_{20} \sim A_0$ ，其中高3位用于芯片选择接到74LS138芯片的CBA，低18位接到存储器芯片地址。
- ❖ MREQ#：作为译码器的使能信号。



连接图





5.4 高速存储器

- ❖ **解决问题：**弥补CPU与主存速度上的差异。
- ❖ **从存储器角度，解决问题的有效途径：**
 - 主存采用更高速的技术来缩短存储器的读出时间，或加长存储器的字长
 - 采用并行操作的**多端口存储器**
 - 在CPU和主存之间加入一个**高速缓冲存储器（Cache）**，以缩短读出时间
 - 在每个存储器周期中存取几个字（**多体交叉存储**）



双端口存储器



多体交叉存储器



相联存储器



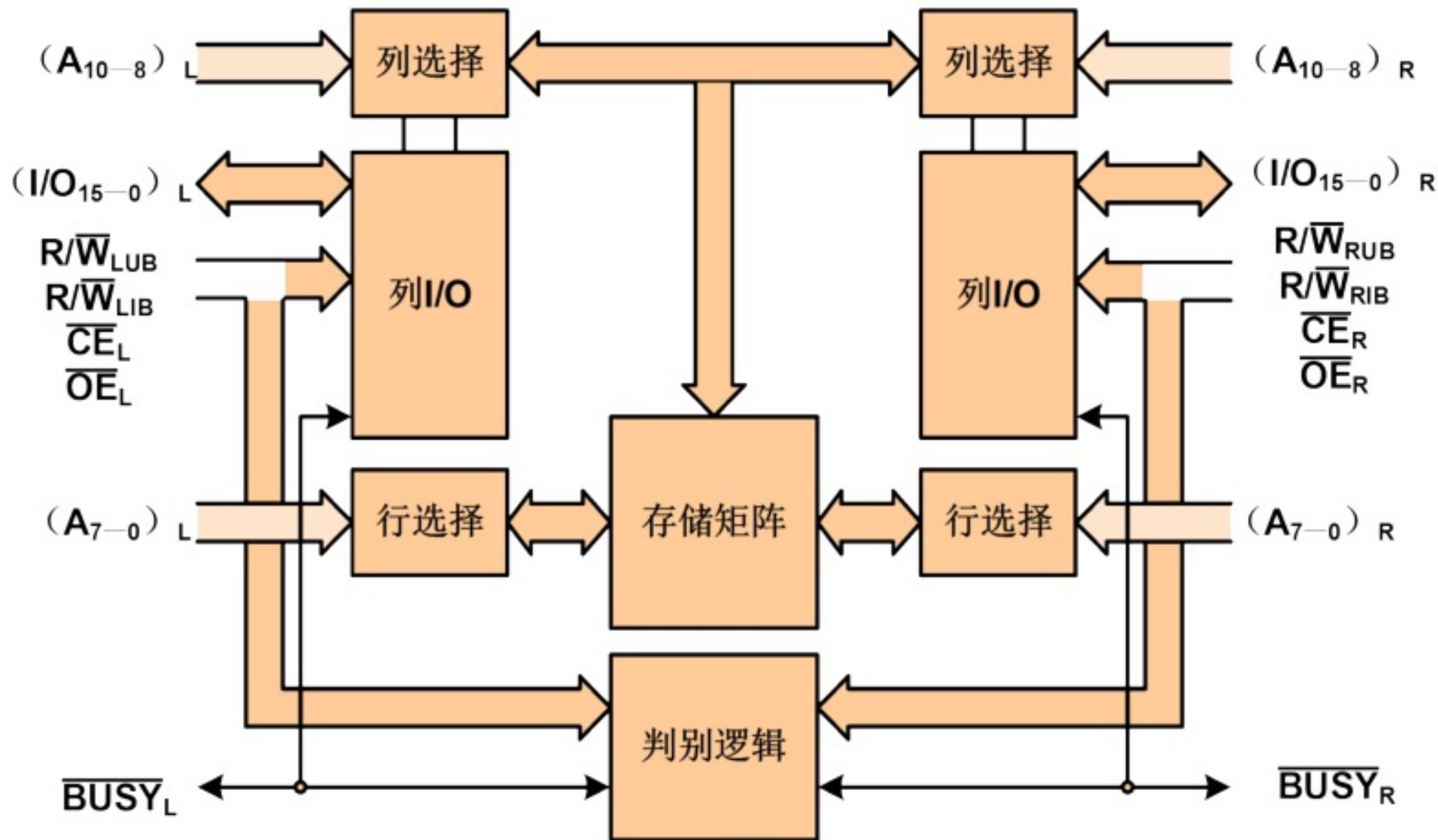


一、双端口存储器

- ❖ **特点：**同一个存储器具有两组相互独立的读写控制线路，允许两个独立的CPU或控制器同时异步地访问存储单元，是一种高速工作的存储器。其最大的特点是存储数据共享。
- ❖ **结构特点：**具有左右两个端口，每一个端口都有自己的数据线、地址线、控制信号线。
- ❖ **访问冲突：**当左端口和右端口的地址不相同时，在两个端口上同时进行读写操作，不会发生冲突。若左、右端口同时访问相同的存储单元，则会发生读写冲突。
- ❖ **解决方法：**判断逻辑决定对哪个端口优先进行读写操作，而暂时关闭另一个被延迟的端口，即置其忙信号 $\overline{BUSY}=0$ 。



2K×16位双端口存储器IDT7133的逻辑框图





二、多体交叉存储器

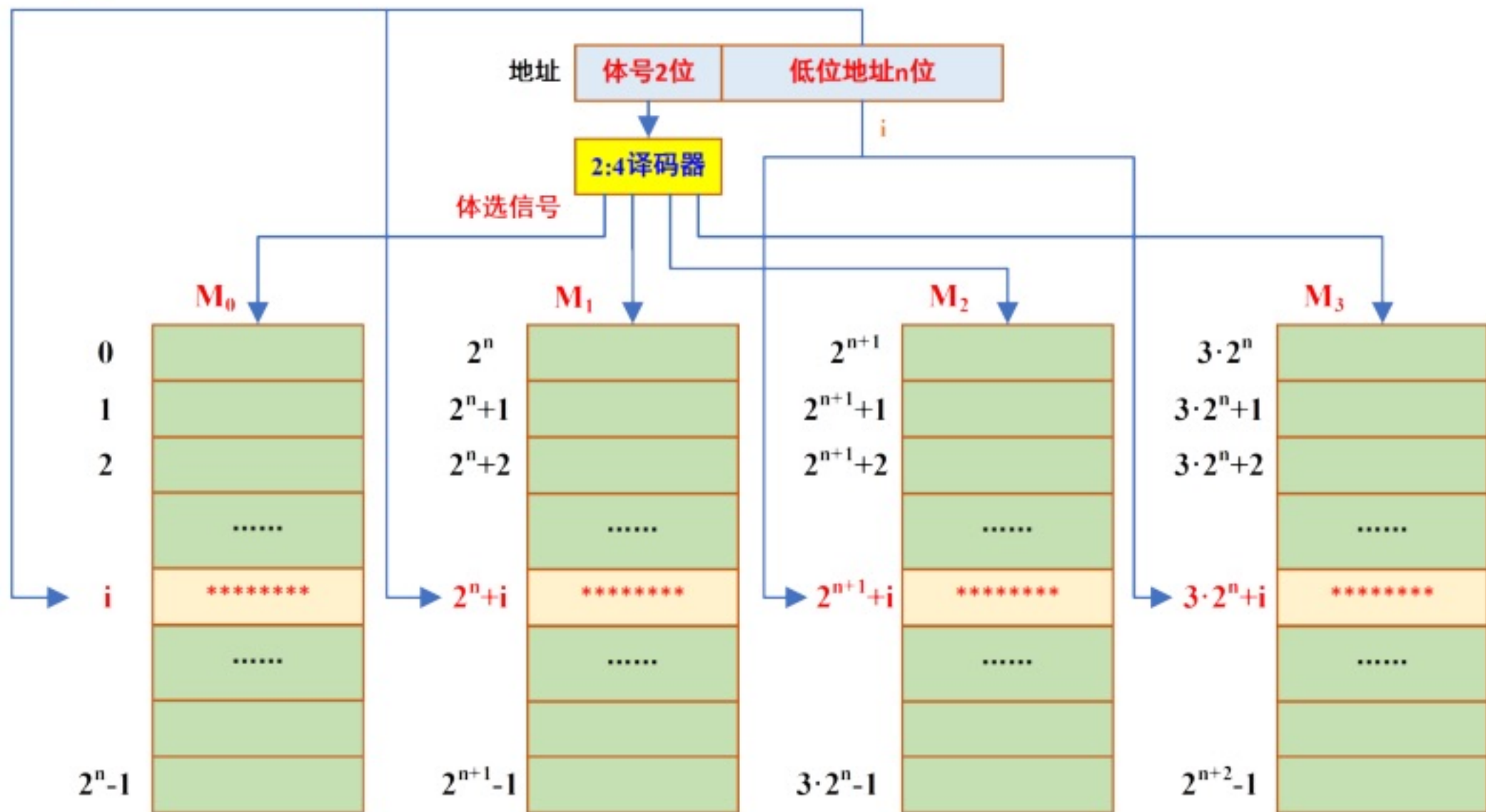
- ❖ **特点：**通过改进主存的组织方式，在不改变存储器存取周期的情况下，提高存储器的带宽。
- ❖ **结构特点：**多体交叉存储器由M个的存储体（或称存储模块）组成，每个存储体有相同的容量和存取速度，又有各自独立的地址寄存器、地址译码器、读写电路和驱动电路。
- ❖ **编址方法：**交叉编址，即任何两个相邻地址的物理单元不属于同一个存储体，一般在相邻的存储体中；同一个存储体内的地址都是不连续的。



顺序编址

❖ 4体

❖ 顺序编址：高位地址通过译码器产生体选信号

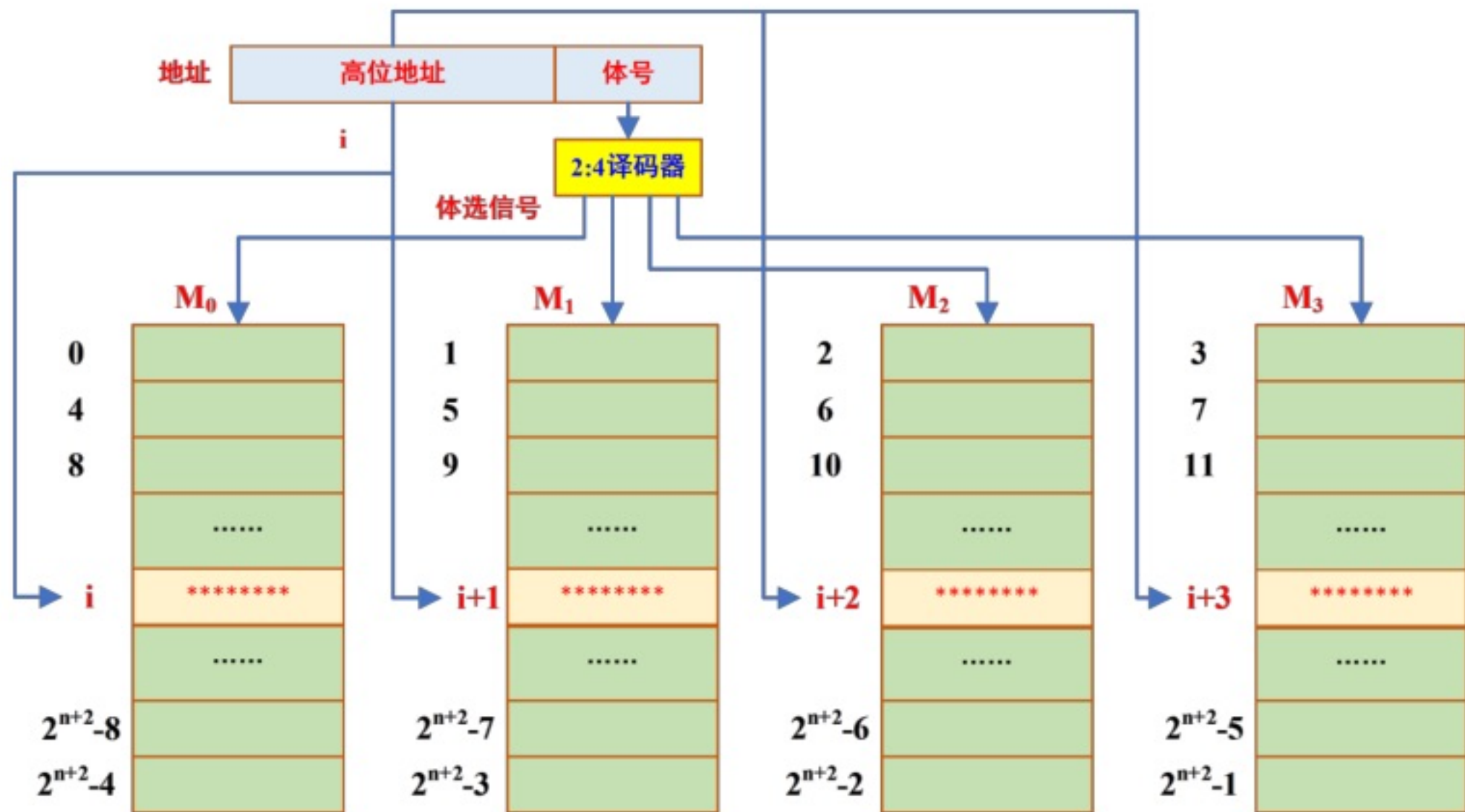




交叉编址

❖ 4体

❖ 交叉编址: 低位地址通过译码器产生体选信号

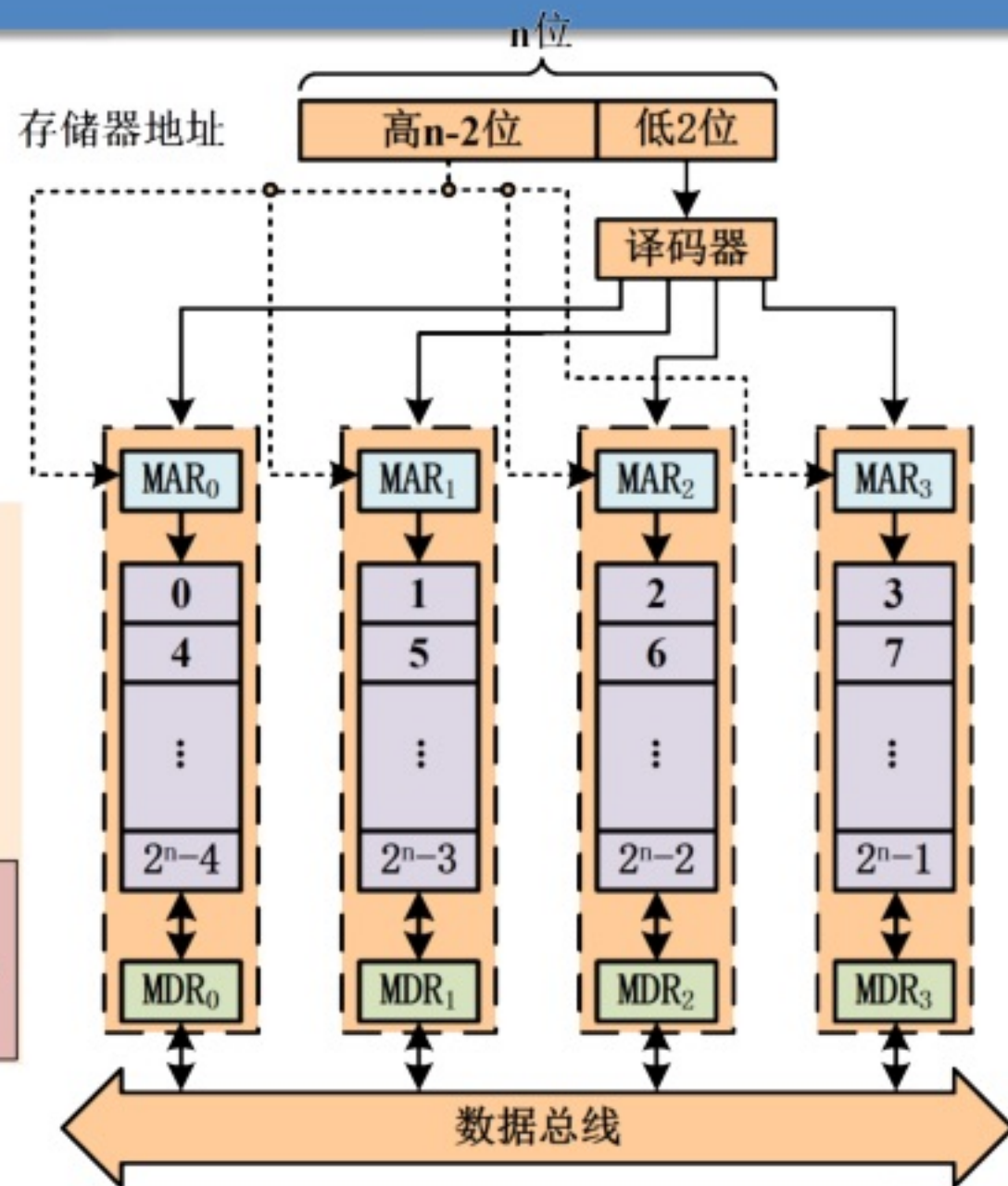
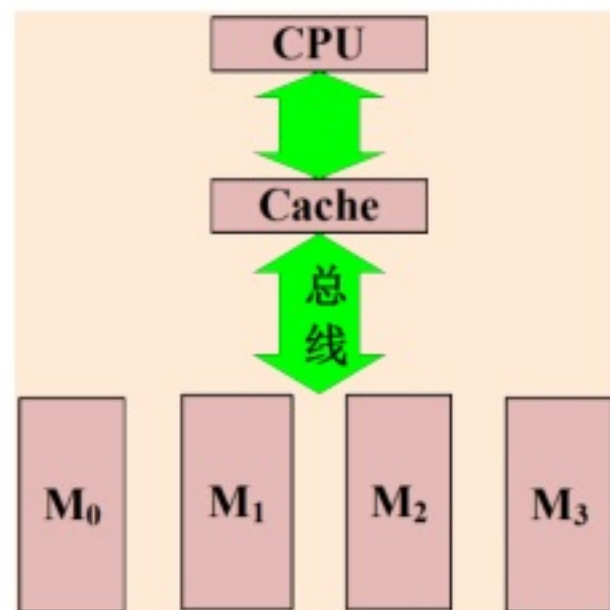


4体交叉存储器连接

❖ 访问:

- CPU送出 m 个地址(分属各体, 以避免访问冲突)
- 间隔 $\tau = T/m$ 启动访问
- 采用流水线形式, 分时使用数据总线进行信息传递

❖ 每个存储体的存储周期 T 没变,
但是当CPU连续访问一个字块时,
可以大大提高存储器的带宽



❖ 当 n 很大时, 带宽约是原来的 m 倍





三、相联存储器（associative memory）

❖ 特点：按**内容**进行访问的存储器

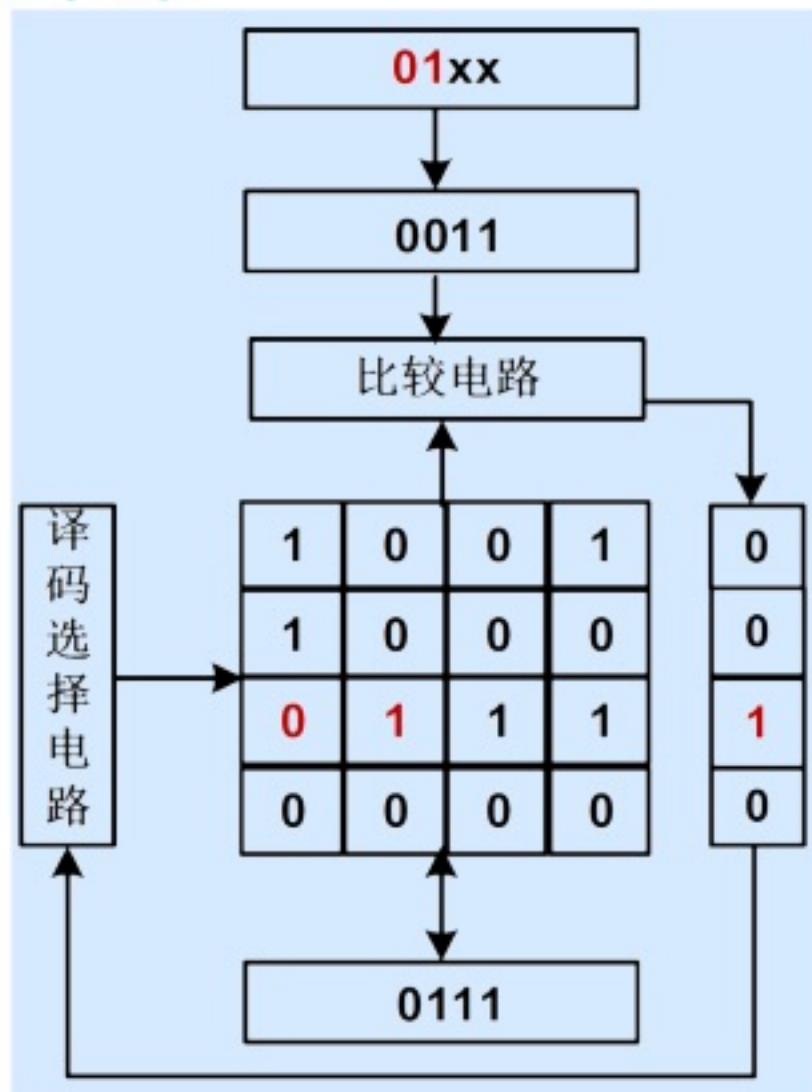
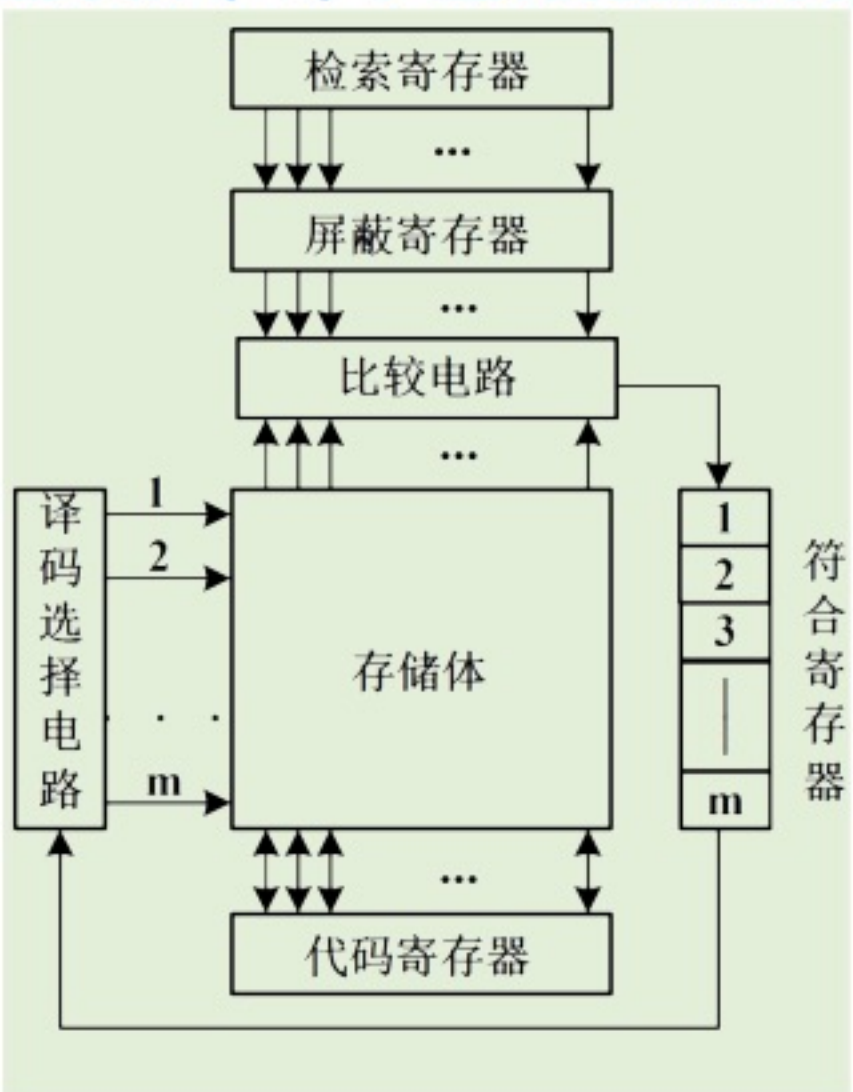
- 用**关键字**去检索存储器中部分字段，对包含关键字的**存储单元**进行读和写
- **全部由硬件实现其功能**；容量比较小
- **适用于快速查询的场合**，如Cache的目录、虚拟存储器中页表、段表的快表



三、相联存储器 (associative memory)

❖ 构成：存储体+查询比较电路

- 检索寄存器：待检索的内容，即关键字Key
- 屏蔽寄存器：屏蔽不要检索的字段 (=1)，选择要检索的字段 (=0)
- 符合寄存器：关键字字段内容=检索的关键字Key ? =, 则为1
- 代码寄存器：输出关键字Key内容相符的单元的所有内容 或者其他字段的值Value





5.5 高速缓冲存储器Cache

主存与Cache的地址映射方式

Cache的
基本原理

替换算法

Cache的
多层次设计

写策略





一、Cache的基本原理



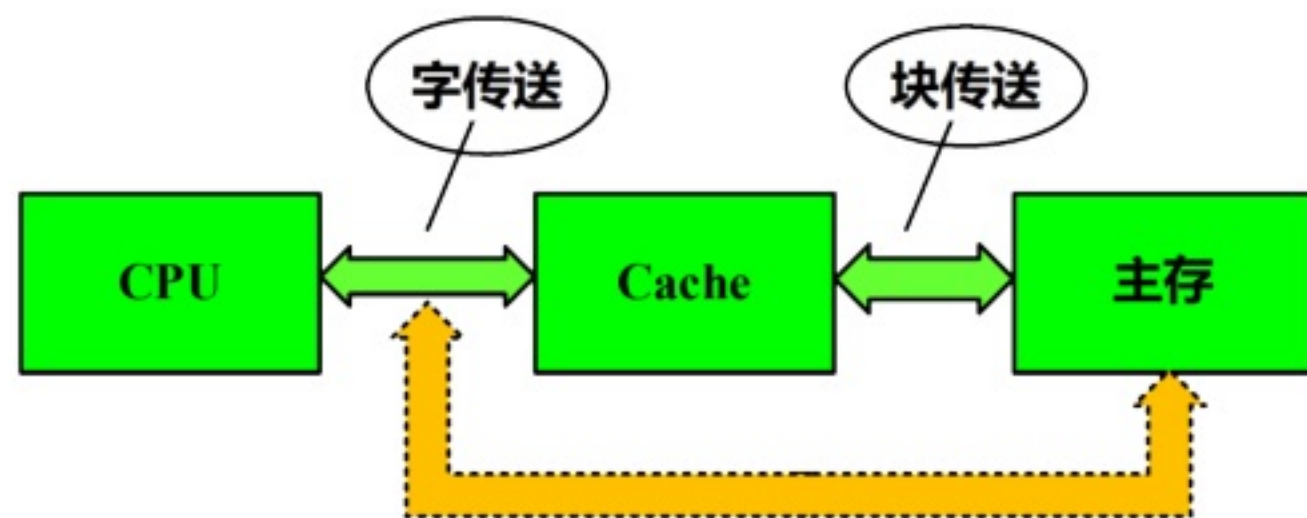


1、Cache的特点

- ❖ 位于CPU和主存之间的一个高速小容量的存储器，一般由SRAM构成
- ❖ 功用：弥补CPU和主存之间的速度差异，提高CPU访问主存的平均速度
- ❖ 对程序员透明：功能均由硬件实现
- ❖ 内容：经常访问的主存热数据的副本
- ❖ 设置Cache，就能快吗？
- ❖ 理论基础：程序访问的局部性原理——在一个较短的时间间隔内，程序访问的地址往往集中在一个不大的范围内。
 - 时间局部性：若某个地址被访问，则在不久的将来仍有可能被再次访问
 - 空间局部性：若某个地址被访问，则与其相邻的地址单元也很可能被访问

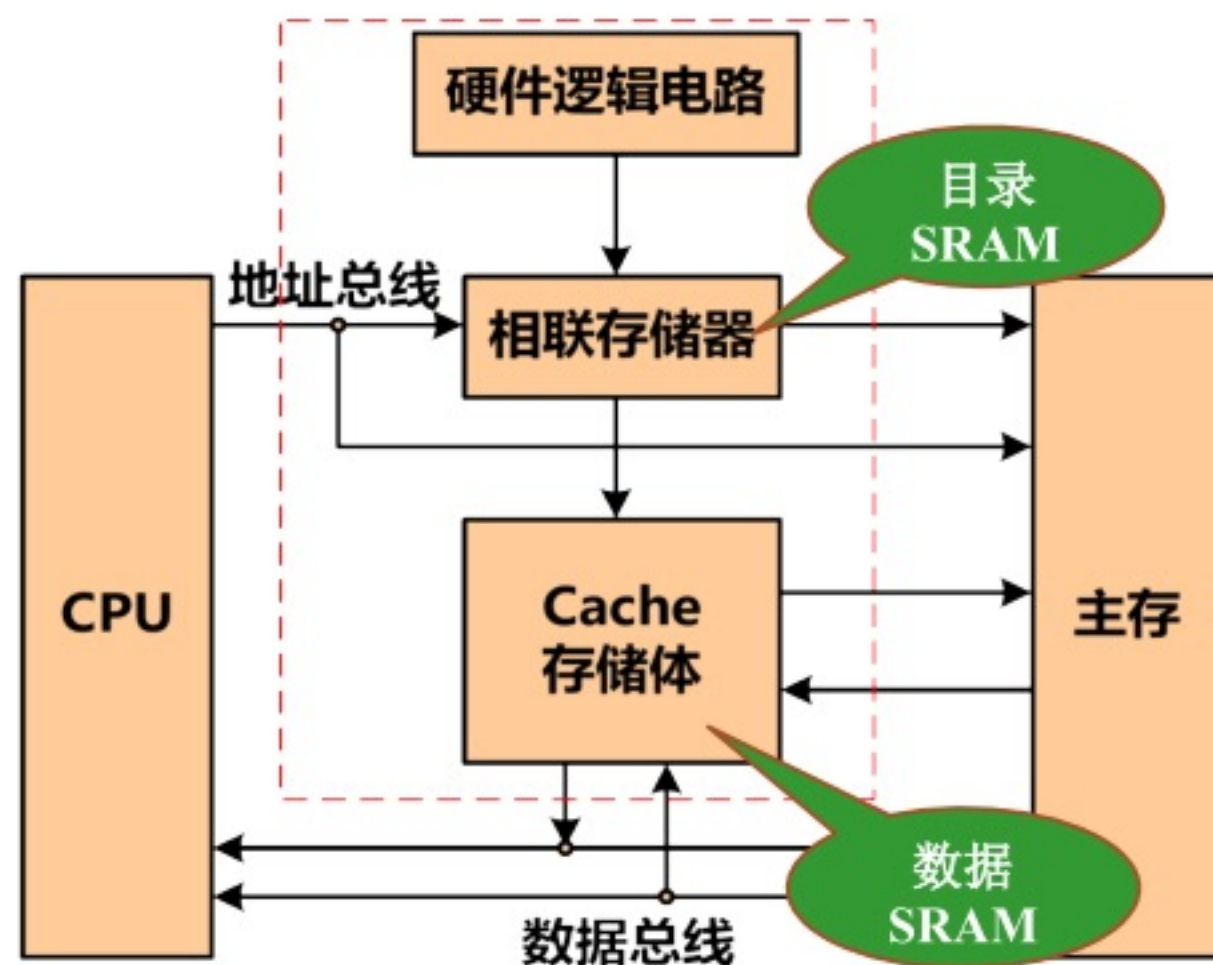
2、Cache的工作原理

- ❖ Cache的速度比主存快5 - 10倍。
- ❖ CPU访问主存时，依据主存地址，先从Cache寻找数据
 - 命中：从Cache读取数据
 - 不命中：从主存将该地址所在的整块数据装入Cache



Cache、主存与CPU的关系

Cache的组成原理图





Cache的读操作

地址映射

❖ 依据主存地址，Cache控制逻辑判断这个字是否在Cache中

hit

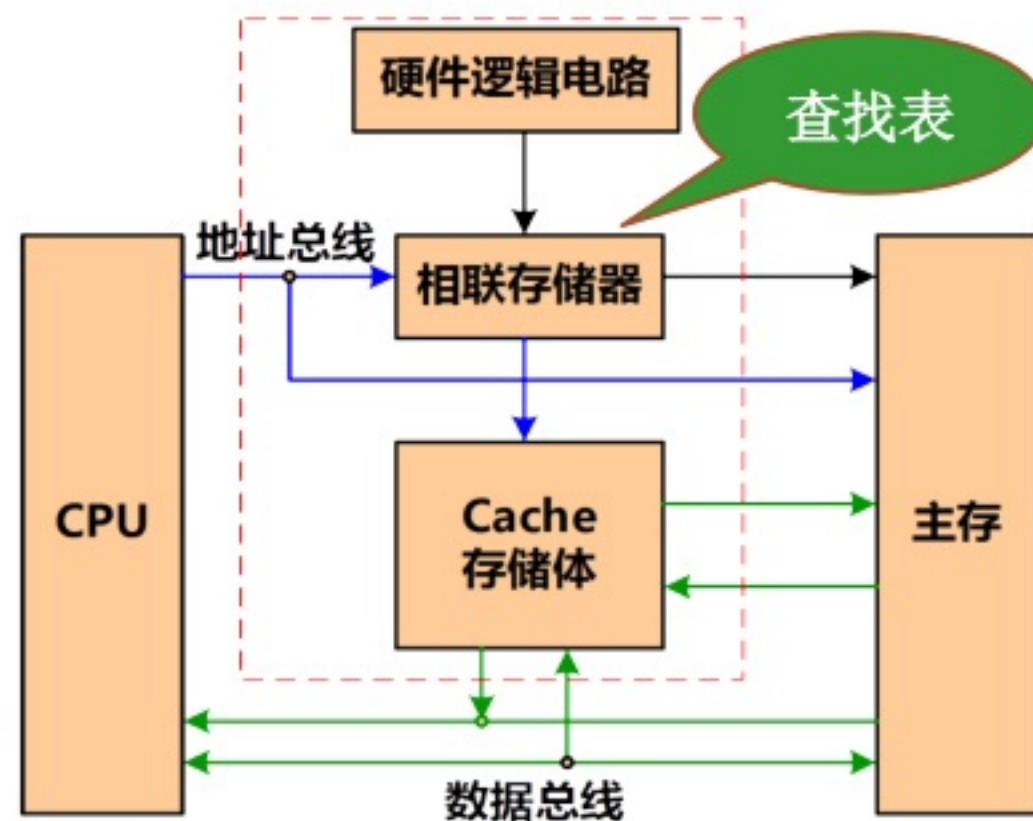
命中：该字在Cache中

miss

不命中：该字不在Cache

❖ 命中/不命中、读/写操作：

- 读命中：立即从Cache读出送给CPU；
- 读不命中：两种解决方法
 - A) ①主存中该字所在的数据块 复制到 Cache中；
②从Cache，将该字传送给 CPU
 - B) 把此字从主存 读出 送到CPU，
同时，把包含这个字的数据块从主存中读出送到Cache中





Cache的写操作

❖ 命中/不命中、读/写操作:

写不命中:

allocation

- **写分配:** 将主存该字块装入Cache, 该字写入Cache

non-allocation

- **非写分配:** 不调入Cache, 直接将该字写入主存中

写命中: 两种方法进行处理

write-through

- **写贯穿方法:** 同时对Cache和主存进行写操作

write-back

- **写回:** 只写Cache, 不写主存; 仅当此Cache块被替换时, 才将该块写入主存

写策略:

- **写缺失处理:** 写分配 or 非写分配
- **保持一致性:** 写回 or 写贯穿





3、Cache的命中率

❖ **命中率h**: CPU访问主存数据时, 命中Cache的次数, 占全部访问次数的比率

▪ 失效率: 不命中Cache的次数, 占全部访问次数的比率

▪ N: CPU发出的访存请求总次数

▪ N_c : 命中Cache的次数

$$h = \frac{N_c}{N}$$

❖ **命中率的大小**, 取决于程序的行为、Cache的容量、组织方式、块大小。

❖ t_c : Cache的访问时间

$$t_a = ht_c + (1 - h) \times (t_m + t_c)$$

❖ t_m : 主存的访问时间

$$t_a = ht_c + (1 - h) t_m$$

❖ t_a : Cache/主存系统的**平均访问时间**

❖ Cache/主存系统的访问效率e:

$$e = \frac{t_c}{t_a}$$





二、主存与Cache的地址映射方式

❖ **讨论的问题：** Cache中数据如何组织，以便对应到主存。

- 如何根据主存地址，判断Cache有无命中，并变换为Cache的地址。

❖ **有三种地址映射方式：**

1

直接映射

2

全相联映射

3

组相联映射

❖ **行：** Cache的数据块

❖ **块：** 主存的数据块

❖ **行与块等长**

❖ **主存容量：** 2^m 块

❖ **Cache容量：** 2^c 行

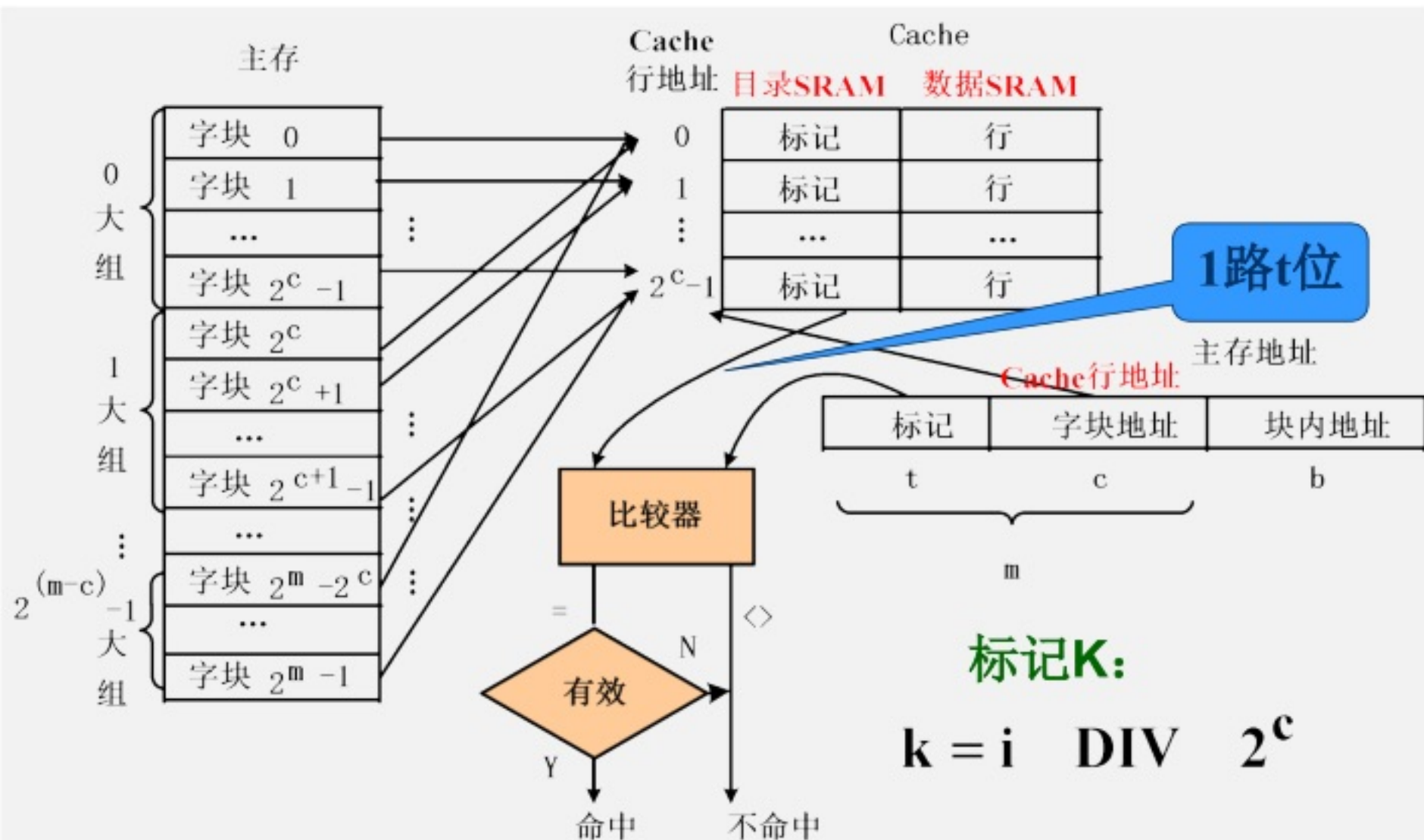
❖ **每个字块：** 2^b 字



1、直接映射(direct mapped)

- ❖ 主存容量: 2^m 块
- ❖ Cache容量: 2^c 行
- ❖ 每个字块: 2^b 字

- ❖ 多对一的映射关系: 主存的第*i*块指定映射到Cache的第*j*行
 $j = i \bmod 2^c$





直接映射的特点

- ❖ **优点：**映射方式简单，易实现；映射速度快，无须查表
 - **数据查找：**利用主存地址中的Cache行地址/行索引(index)字段，**直接定位**到Cache行，然后对比该行的标记位(tag)→是否命中
 - 目录SRAM，可以和数据SRAM一起存放
 - 冲突后，直接淘汰
- ❖ **缺点：**容易冲突，Cache命中率低。
- ❖ **适合：**大容量Cache



直接映象法举例

❖ 例如：主存4GB（按字节编址），Cache 512KB，块大小16B；
读主存234589ABH的过程（假设Cache空白）。

❖ 主存划分格式

- 主存地址： 32 位
- 块大小16B= 2^4 B，又是按字节编址的，所以块内地址： 4 位
- Cache分为： $512\text{KB} \div 16\text{B} = 2^{15}$ 行（块）
- Cache行地址： 15 位
- 高位标记： $32 - 15 - 4 = 13$ 位

标记	Cache行（块）地址	块内地址
13位	15位	4位



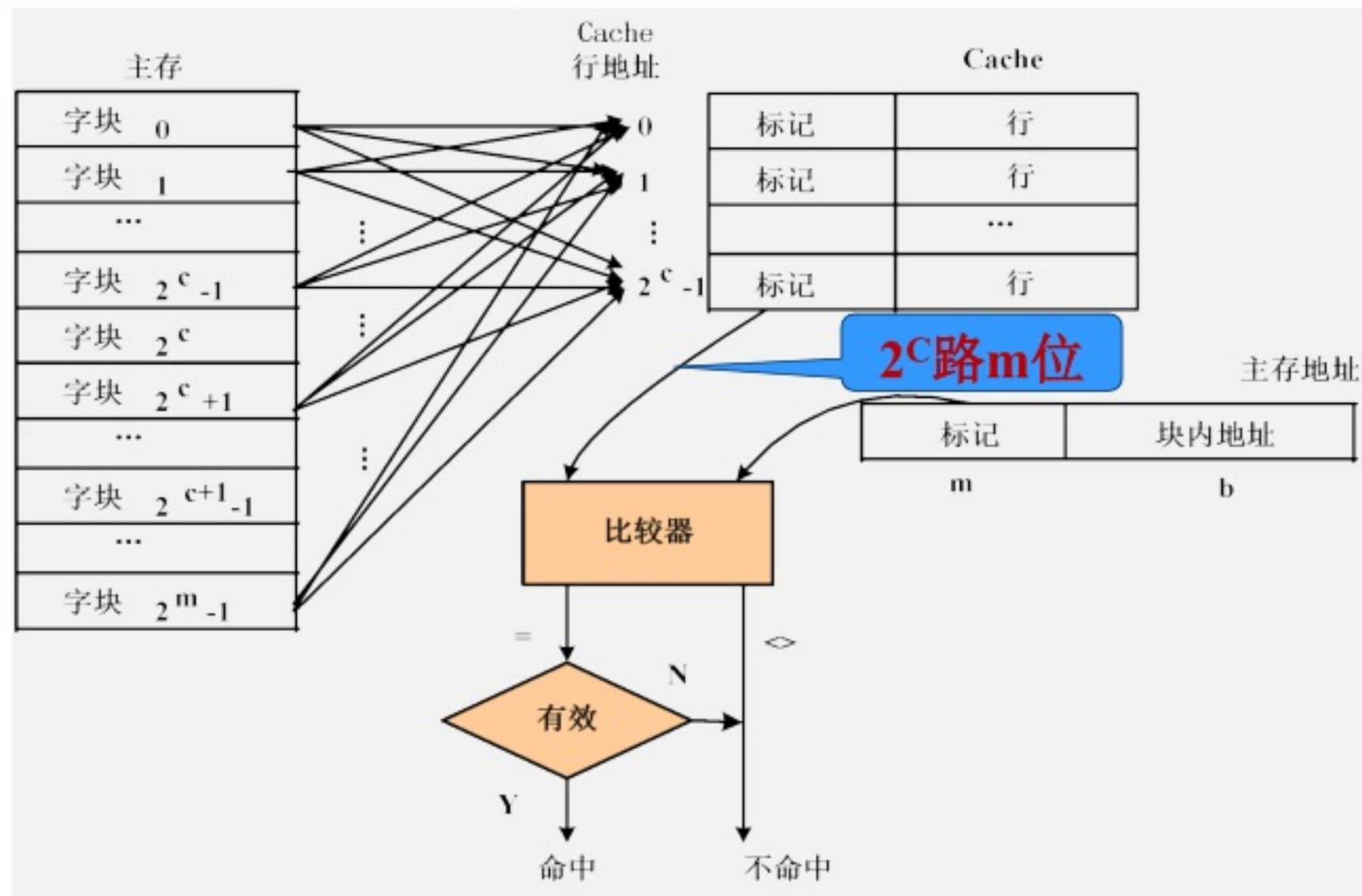


2、全相联映射 (fully-associated)

- ❖ 主存容量: 2^m 块
- ❖ Cache容量: 2^c 行
- ❖ 每个字块: 2^b 字

❖ 多对多的映射关系:

- 主存的**任何一块**均可以映射到Cache的**任何一行**





2、全相联映射

❖ **优点：**映射机制灵活，命中率高

- 任意映射
- Cache全部装满后才会出现块冲突，块冲突的概率低，利用率高

❖ **缺点：**

- 比较器使用多路并发比较（相联存储器），电路和淘汰算法复杂

❖ **适合于小容量的Cache**



全相联映射法举例

❖ 例如：主存4GB（按字节编址），Cache 8KB，块大小16B；读主存234589ABH的过程（假设Cache空白）。

❖ 主存划分格式

- 主存地址：32 位
- 块大小16B= 2^4 B，又是按字节编址的，所以块内地址：4 位
- 高位标记：32-4=28 位

标记	块内地址
28位	4位
0010 0011 0100 0101 1000 1001 1010	1011

❖ 目录SRAM多大？

❖ 512行* (1+28) 位

❖ 几路、多少位的比较器？

❖ 512路、28位的并发比较





3、组相联映射 (set-associated)

- ❖ 将Cache的行分成 2^{c-r} 组，每组 2^r 行
- ❖ 主存字块 i 存放到Cache中的固定组（第 j 组），但可以映射到该组的任何一行

$$j = (i \bmod 2^{c-r}) \times 2^r + k \quad 0 \leq k \leq 2^r - 1$$

❖ 优点:

- 大大增加了映射的灵活性，主存块可映射到Cache的 2^r 块，提高了命中率
- 每次比较只是进行 2^r 路比较， r 较小时，硬件开销不是很大
- 组相联映射通常采用2路/4路/8路/16路比较，即取 $r = 1/2/3/4$



组相联映射法举例

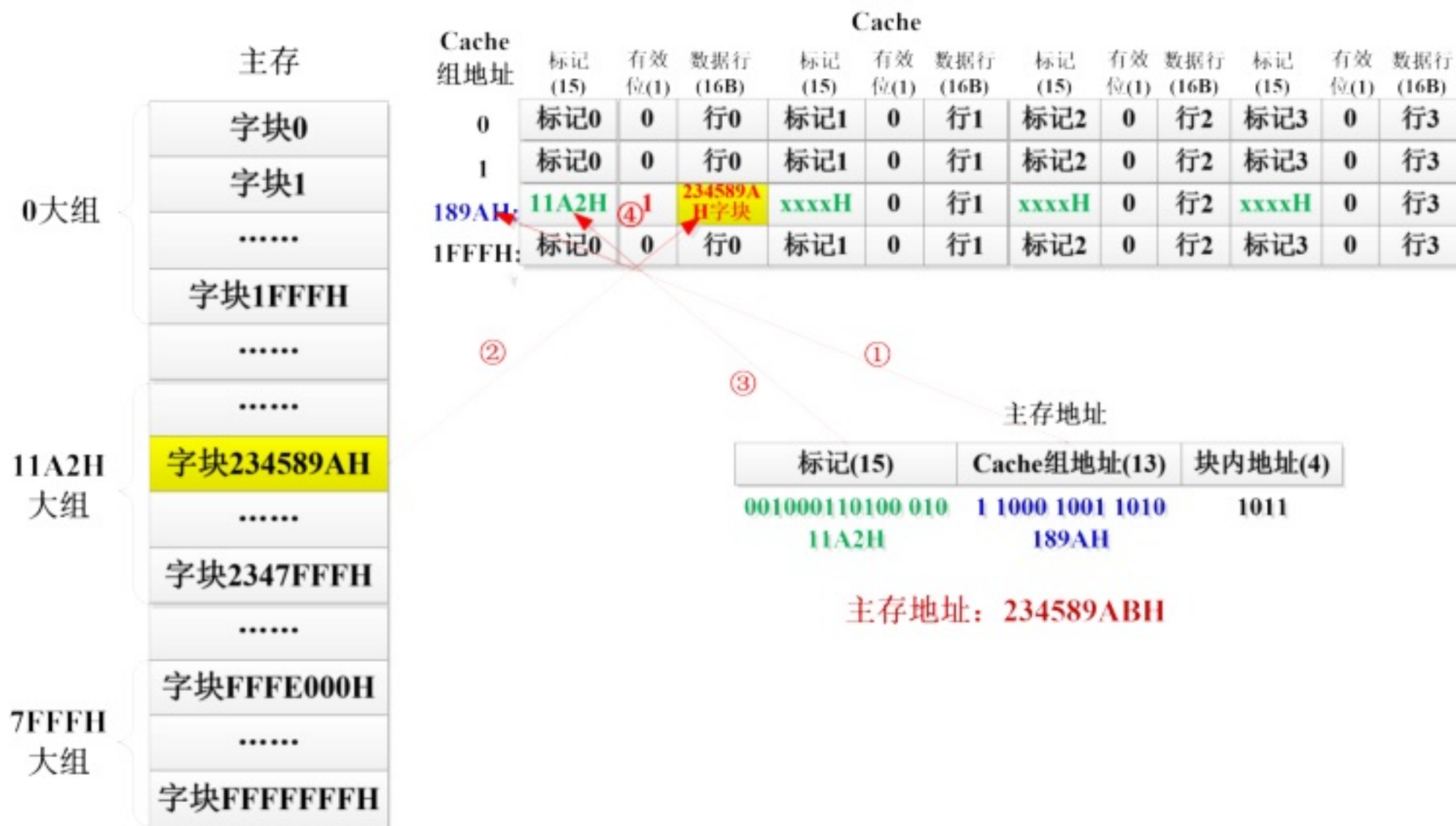
- ❖ 例如：主存4GB(按字节编址)，Cache 512KB，块大小16B；4路组相联，读主存234589ABH的过程(假设Cache空白)。
- ❖ 主存划分格式
 - 主存地址： 32 位
 - 块大小16B= 2^4 B，又是按字节编址的，故块内地址： 4 位
 - Cache分为： $512\text{KB} \div 16\text{B} = 2^{15}$ 行 (块)
 - 4路组相联，即4块/组，Cache分为： $2^{15} \div 4 = 2^{13}$ 组
 - Cache组地址： 13 位
 - 高位标记： $32 - 13 - 4 = 15$ 位

标记	Cache组地址	块内地址
15位	13位	4位
0010 0011 0100 010	1 1000 1001 1010	1011



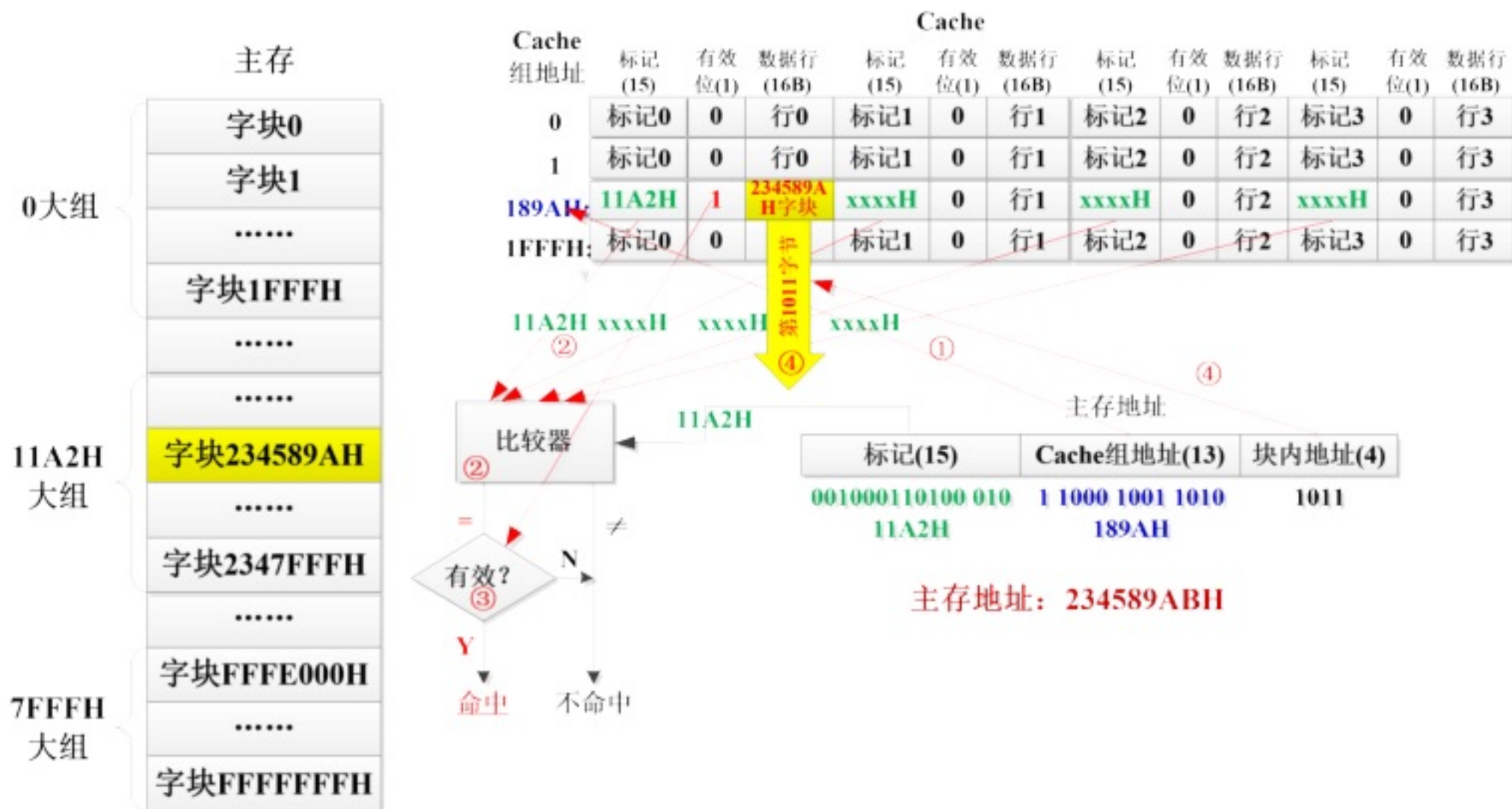


组相联映象法举例(装入Cache)





组相联映象法举例(第二次访问命中)





三、替换算法

❖ 1、随机替换算法

❖ 2、先进先出算法 (FIFO, First-in First-out)

❖ 3、最近最少使用算法 (LRU, Least-Recently-Used)

- 统计哪一行是近段时间使用次数最少的Cache行，需替换时就将它替换出去
 - 为每个Cache行设置一个计数器（最大值是每组的行数-1）
 - Cache每命中一次，命中行的计数器被清零，其他行的计数器加1
 - 需要替换时，将计数器值最大的行替换出去





四、写策略

❖ 写策略需要解决两个问题：

- **当写命中时，写入Cache后，主存要不要写？**
 - 写贯穿 (Write-through) : 写Cache+写主存
 - 写回 (Write-back) : 只写Cache, 不写主存
- **当写不命中时，要不要把数据装入Cache？**
 - 写分配 (Write-allocate) : 要装入Cache;
 - 非写分配 (non-Write-allocate) : 不要装入Cache;

❖ 一般采用组合：

- **写贯穿+非写分配：**
 - 写命中时：同时写Cache和写主存
 - 写缺失时：只写主存，不调入Cache
- **写回+写分配：**
 - 写命中时：只写Cache, 不写主存
 - 写缺失时：字块调入Cache, 只写Cache



四、写策略

❖ 写回策略 (Write Back)

- **操作:** 只写Cache, 不写主存; 仅当此Cache行被换出时, 相应的主存块内容才被修改
- **缺点:** Cache和主存对应的数据块, 数据可能会不一致; 硬件与控制较为复杂;
- **额外硬件:** 为每个Cache行设置一个**修改位(modified)/脏位(dirty)**
 - =1: 本Cache行已经被CPU修改过, 与主存不一致
 - =0: 本Cache行没有被CPU修改过, 与主存一致
 - 该行被替换时, 检查其修改位, =1, 则需要先写回主存





五、Cache的多层次设计

❖ 设计Cache主要考虑五个问题：

- 第一，容量
- 第二，Cache中行的大小
- 第三，Cache的组织（地址映射方式）
- 第四，指令和数据共用同一个Cache还是分享不同Cache
- 第五，Cache的层次



五、Cache的多层次设计

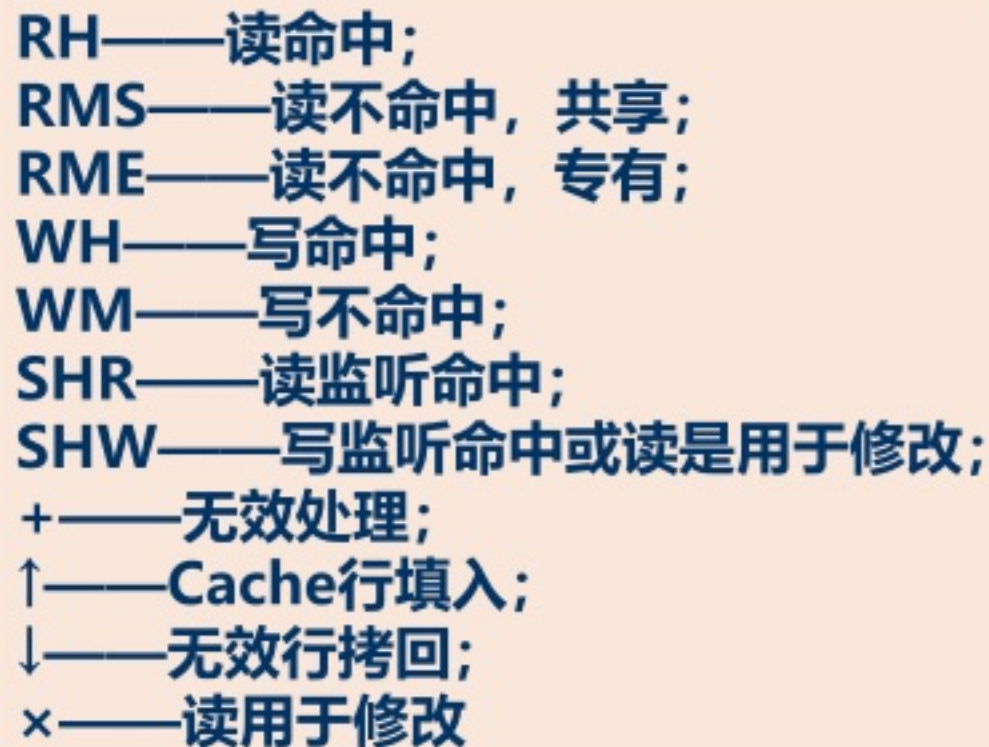
1. 统一Cache和分离Cache

- **统一Cache:** 只有一个Cache, 指令和数据混放。
- **分离Cache:** 分为指令Cache和数据Cache。
 - 可消除流水线中取指令部件和执行单元间的竞争→特别适用于超标量流水线
 - 是Cache结构发展的趋势

2. 单级Cache与多级Cache

- 一级Cache (L1) 、二级Cache (L2) 和三级Cache (L3)
- 采用多级Cache结构可以提高性能

- MIMD: 多个处理器都有自己的Cache, 那么对于主存的同一块数据, 可能在不同的Cache中有不同的副本。
- 如何保持数据一致性?



MESI协议的状态图



5.6 虚拟存储器

❖ 为什么设置虚拟存储器 (Virtual Memory) ?

- 程序设计总是受小容量主存的限制; → 不受限制
- 使多个用户程序/虚拟机能同时运行于一台计算机, 共享同一个存储器

❖ 什么叫虚拟存储器?

- 指存储体系中**主存-辅存层次**; 包含主存(内存条)、辅存(硬盘等)
- 由**存储器管理部件MMU** (**硬件**, 位于MPU内) 和**操作系统** (**存储管理软件**) 共同支持;
- 具有**自动装入和替换功能**;
- 从逻辑上为用户提供一个**具有辅存容量、接近主存速度的存储器**



5.6 虚拟存储器

❖ 虚拟存储器的容量：

- 与物理主存大小无关，受限于计算机的地址结构和可用磁盘容量

❖ 计算机中三种存储地址空间：

- 主存地址空间：存放正在运行的程序和数据；物理地址
- 虚拟地址空间：程序员编写程序时所使用的地址空间；逻辑地址/虚拟地址
- 辅存地址空间：存放暂时不用的程序和数据；



5.6 虚拟存储器

❖ 虚拟存储器的实现方式有三种：段式、页式或段页式

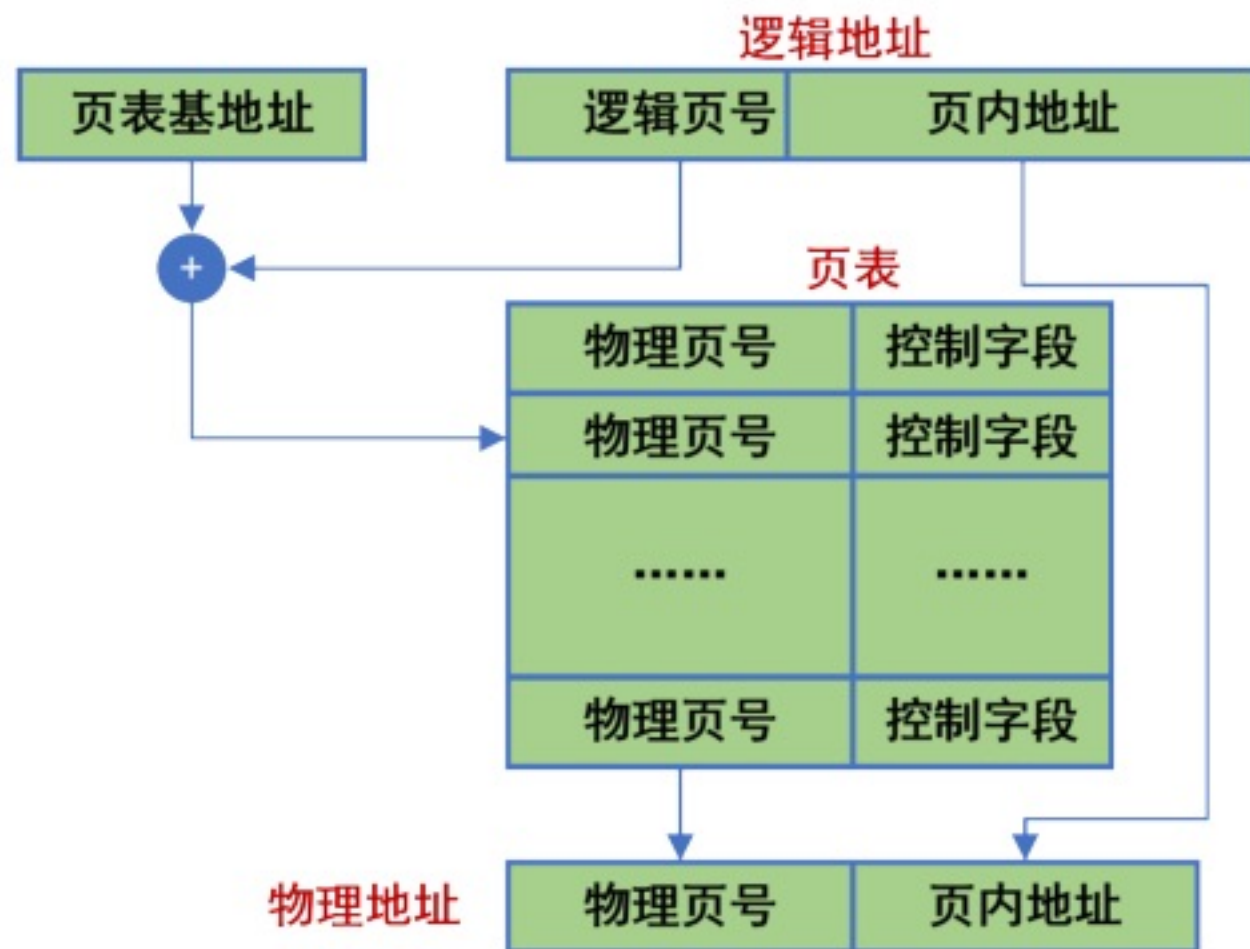
1. 页式虚拟存储器：程序、主存和辅存都以“页”为单位进行划分和调度

❖ MMU功能：

- 逻辑地址→物理地址
 - 创建、管理页表
- 访问权限管理

❖ MMU组成：

- 专用寄存器组
 - 页表基址寄存器
 - 控制寄存器
- 地址映射机构

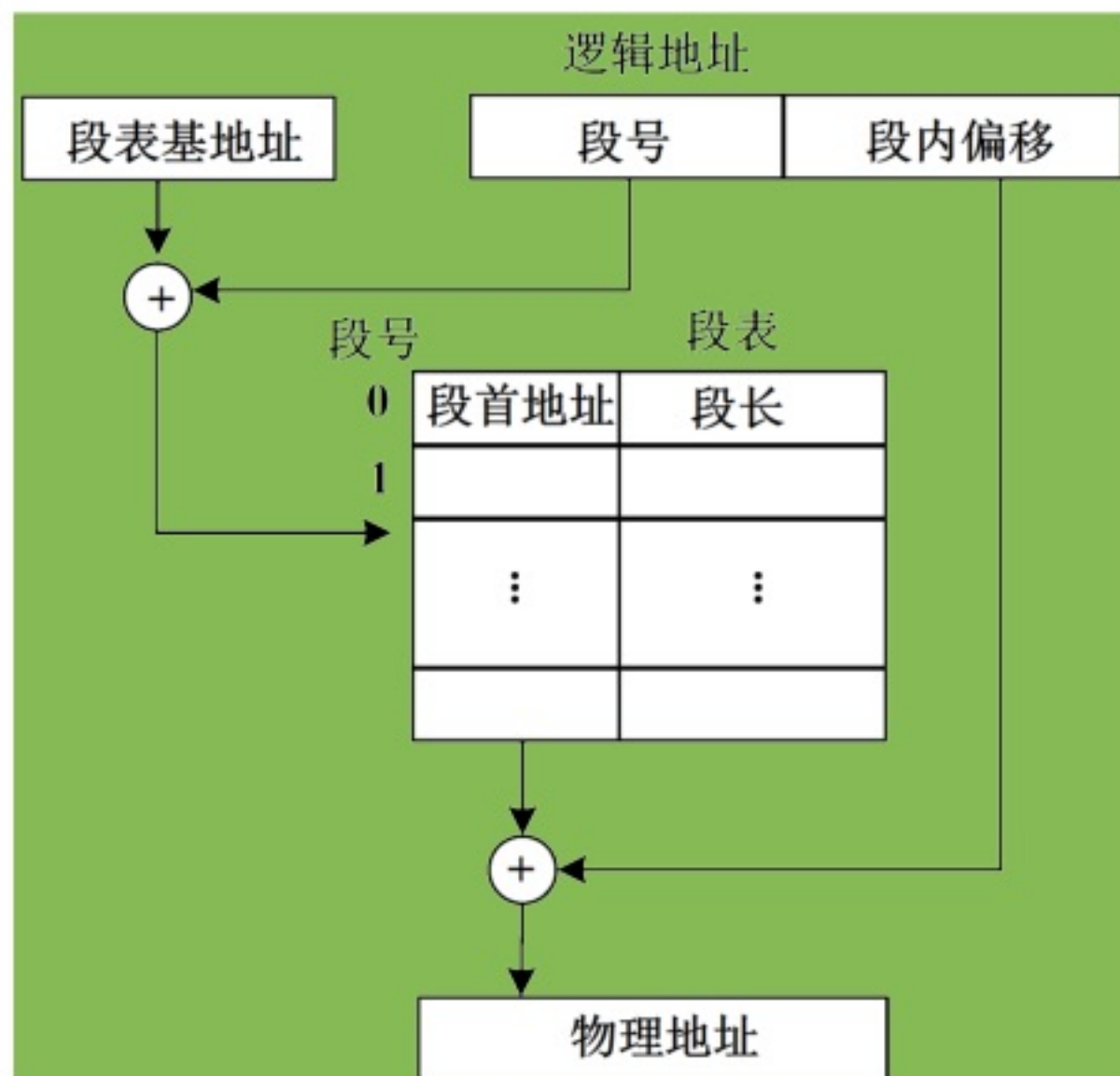




5.6 虚拟存储器

❖ 2、段式虚拟存储器

- ❖ 段的分界与程序的自然分界相对应，段长不固定，因程序而异
- ❖ 段的独立性—易于编译、管理、修改和维护，也便于多道程序共享
- ❖ 各段的长度不同，不便于主存的分配
- ❖ 段式管理容易产生碎块，浪费主存空间

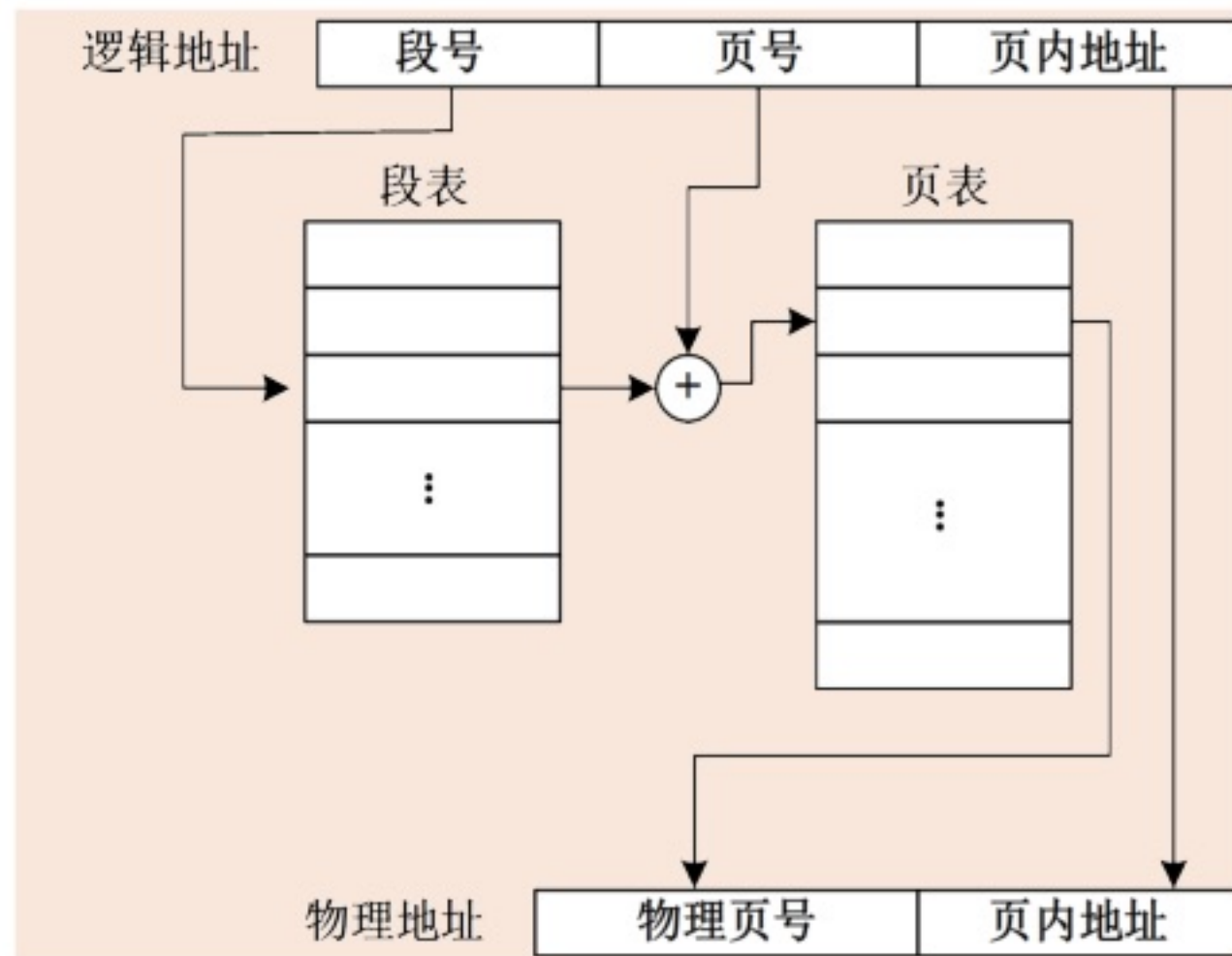




5.6 虚拟存储器

❖ 3、段页式虚拟存储器

- ❖ 程序按模块分段
- ❖ 段再分成长度固定的页。
- ❖ 程序调入调出按页面来进行
- ❖ 程序共享保护按段进行
- ❖ 兼备段式，页式管理的优点
- ❖ 在地址映像中需要多次查表





5.7 外存储器

❖ 外存储器：既是输入设备，也是输出设备

❖ 特点：

- 大都采用磁性和光学材料制成。
- 与内存相比，容量大，价格低，速度慢。
- 在断电的情况下可以长期保存信息，所以称为**永久性存储器**。
- 一般为**顺序存取**的存储器，即访问所需时间与数据所在的地址相关。

一 磁盘存储器

二 光盘存储器

三 闪存盘





一、磁盘存储器

- ❖ 大多数外存储器均采用磁记录→正渐渐为SSD取代
- ❖ 磁记录存储器：磁鼓、磁芯、软磁盘、硬磁盘、磁带
- ❖ 磁表面存储：磁性材料薄层涂在金属或塑料表面作载磁体存储信息
 - 硬磁盘存储器：基底(载磁体)是金属
 - 软磁盘存储器：基底(载磁体)是塑料（已淘汰）
- ❖ 磁盘存储器：由磁盘控制器、磁盘驱动器和磁盘盘片三部分构成。
 - ❖ 容量大，位价格低
 - ❖ 可重复使用
 - ❖ 长期保存
 - ❖ 非破坏性读出
 - ❖ 采用机械装置，机械结构复杂
 - ❖ 速度慢
 - ❖ 对工作环境要求高



1、软磁盘存储器

❖ **软盘驱动器：**简称软驱,软磁盘片的读写机构。

- 早期的微机中常配置3.5英寸驱动器一个，其容量为1.44MB，盘符为“A:”

❖ **软盘盘片：**记录信息的载体，使用塑料基底。

- **信息的存储组织方式：**是按磁道和扇区组织的
- **格式化：**格式化就是对软磁盘划分磁道和扇区。

❖ **软盘的特点：**

- **优点：**成本低，重量轻，价格便宜，便于携带
- **缺点：**存储容量小，且容易损坏。



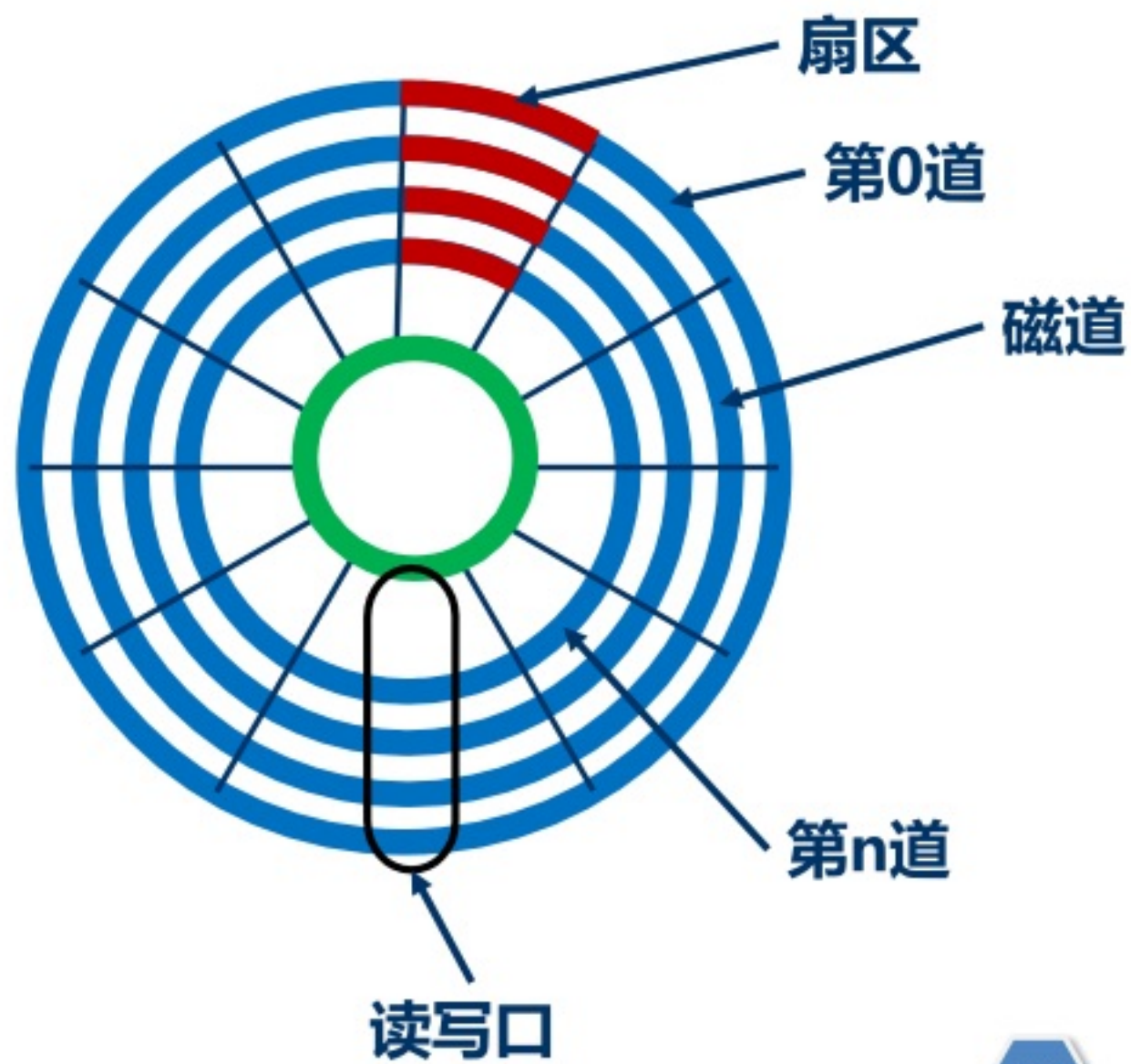
1、软磁盘存储器



软盘外壳

盘片

软盘外壳





1、软磁盘存储器

❖软盘数据定位：磁道号、记录面、扇区号

❖容量=记录面数×每面磁道数×每磁道扇区数×每扇区字节数（字节）

软盘片规格	面数	磁道/面	扇区/道	字节/扇区	容量(KB)
5.25英寸双面双密	2	40 (0--39)	9 (1--9)	512	360
5.25英寸双面高密	2	80 (0--79)	15 (1--9)	512	1200
3.5英寸双面低密	2	80 (0--79)	9 (1--9)	512	720
3.5英寸双面高密	2	80 (0--79)	18 (1--18)	512	1440
3.5英寸双面超高密	2	80 (0--79)	36 (1--36)	512	2880

$$\text{容量} = 2 \times 80 \times 18 \times 512 = 1474560(\text{B}) = 1.44(\text{MB})$$



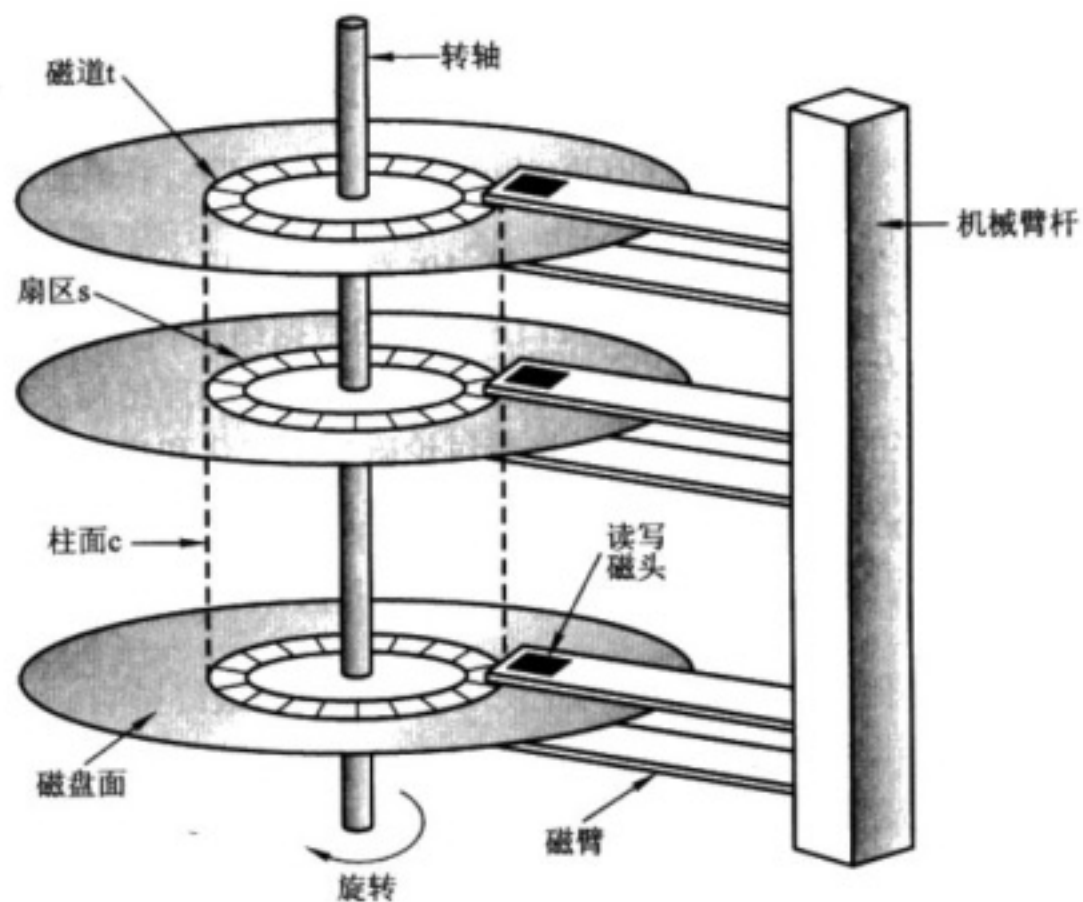
2、硬磁盘存储器

- ❖ **硬磁盘**：也称固定盘，大都采用**温盘**
- ❖ **温切斯特 (wenchester) 技术**：将盘片和驱动器**密封**在外壳内，在盘片飞速旋转时，**磁头靠空气垫浮**在盘片上。
- ❖ **硬盘特点**：
 - **优点**：可靠性高，存储容量大，读写速度快，对环境要求不高。
 - **缺点**：不便于携带，且工作时应避免振动。
- ❖ **硬盘接口**：用得较多的是IDE(ATA)、SCSI和SATA接口

2、硬磁盘存储器

❖ **硬盘盘片**：按柱面号、磁头号、扇区号的格式组织信息
3D参数CHS: Cylinder、Head、Sector (绝对扇区)

相对扇区编号：以扇区为单位线性编址



❖ **柱面号**：磁道号

- **柱面**：一组盘片的同一磁道在纵向上所形成的同心圆

❖ **磁头号**：记录面号

- 每一个记录面上均有一个磁头，所有记录面上的磁头均固定在传动臂上

❖ **扇区号**：扇区编号

- 每条磁道上有若干扇区，数目固定或不同
- **扇区**：访问的最小单位

❖ **每个扇区**：512B



2、硬盘存储器

❖ 硬盘盘符：通常为“C: ”、“D”、“E”、“F”等。

❖ 硬盘容量的计算公式为：单位字节

硬盘容量 = 磁头数 × 柱面数 × 每磁道平均扇区数 × 每扇区字节数

硬盘容量 = 记录面数 × 每面磁道数 × 每磁道平均扇区数 × 每扇区字节数

❖ 硬盘访问时间：

- 寻道时间：将磁头移动到目标磁道（柱面）上的时间
 - 固有参数（平均值），一般3-9ms，最大20ms
- 等待时间：将目标扇区旋转到磁头下面的时间
 - 与转速有关，平均时间是旋转一周所花时间的一半
- 传送时间：读取数据所花的时间
 - 与转速、位密度、数据量有关

定位时间



硬盘技术参数

- ❖ **直径**: 有5.25in、3.5in、2.5in、1.8in
- ❖ **转速**: 每分钟转数, RPM (转/分钟) , 5400 ~ 15000RPM
- ❖ **磁道密度**: 沿磁盘半径方向单位长度上的磁道数, TPI (磁道数/英寸)
 - 已高达5400TPI或更高
- ❖ **位密度**: 磁道沿周长方向单位长度上记录的二进制代码的位数, BPI(位/英寸)
- ❖ **面密度**: 道密度×位密度
- ❖ **盘片组**: 2 ~ 14片不等, 通常有2 ~ 3个盘片, 盘面号(磁头号)为0 ~ 3或0 ~ 5
- ❖ **每面磁道数**: 300 ~ 1024个, 0始编码 现代: 成千上万条
- ❖ **每道扇区数**: 9 ~ 63扇区, 1始编码 现代: 每道不同
- ❖ **每扇区字节数**: 512B SSD: 4KB





二、光盘存储器

❖ **光盘的特点：** 存储容量大,价格低;不怕电磁干扰,存储密度高,可靠性高; 存取速度不断提高

❖ **光盘分类：**

- 只读式光盘**CD-ROM** (Compact Disk Read Only Memory)
- 一次性写入光盘**WORM** (Write-Once-Read- Many) : 用户可以写入一次, 多次读取, 但无法修改其中的数据。
- 可擦除光盘**EOD** (Erasable Optical Disk) : 用户可以像用软盘一样对其进行多次读/写操作。

❖ **光盘的接口类型：** 常用的有IDE或EIDE接口





三、闪存盘 (Flash Memory)

❖ Flash Memory, 简称闪存

- 存储介质：半导体介质
- 基本单元电路：与E²PROM类似，由双层浮空栅MOS管组成

❖ 特点：非易失性、高速、高密度、价格低廉、低功耗、便于携带

❖ 应用场合：数码产品、嵌入式系统、PC系统等

U盘



存储卡



移动硬盘



固态硬盘SSD





5.8 存储保护

- ❖ 存储保护包括两方面：存储区域保护和访问方式保护。
- ❖ **存储区域保护**：当多个用户共享主存时，应防止由于一个用户程序出错而破坏其他用户的程序和系统软件，以及一个用户程序不合法地访问不是分配给它的主存区域。
 - 在虚拟存储系统中，通常采用页表保护、段表保护、键式保护和环保护方法。
- ❖ **访问方式保护**：对主存信息的使用可以有三种方式：读（R）、写（W）、执行（E）





5.9 IA32架构的存储系统举例

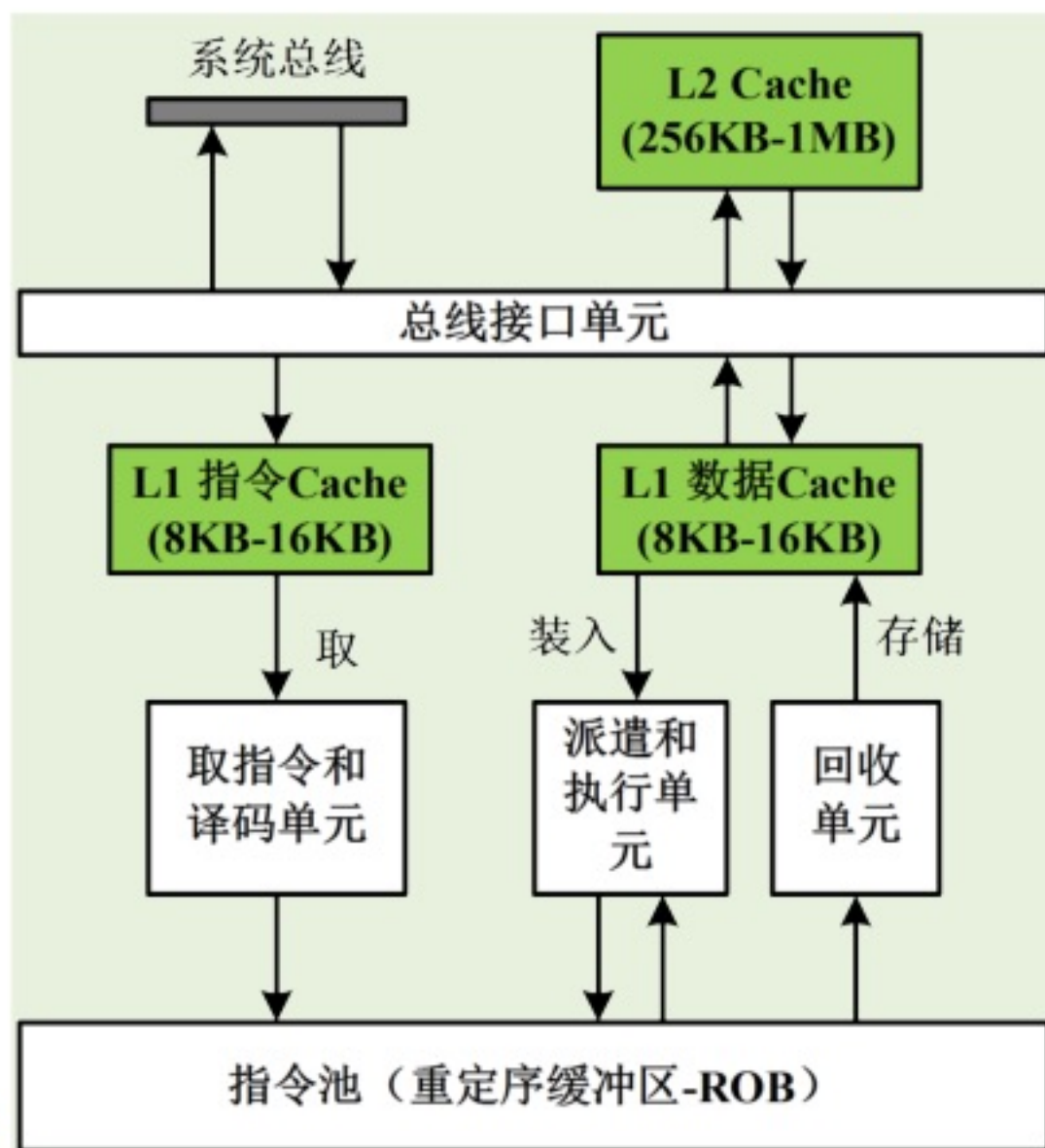
- 一 P6微架构下的Cache
- 二 Intel NetBurst微架构下的Cache
- 三 Intel Core微架构的多核高效内存管理技术





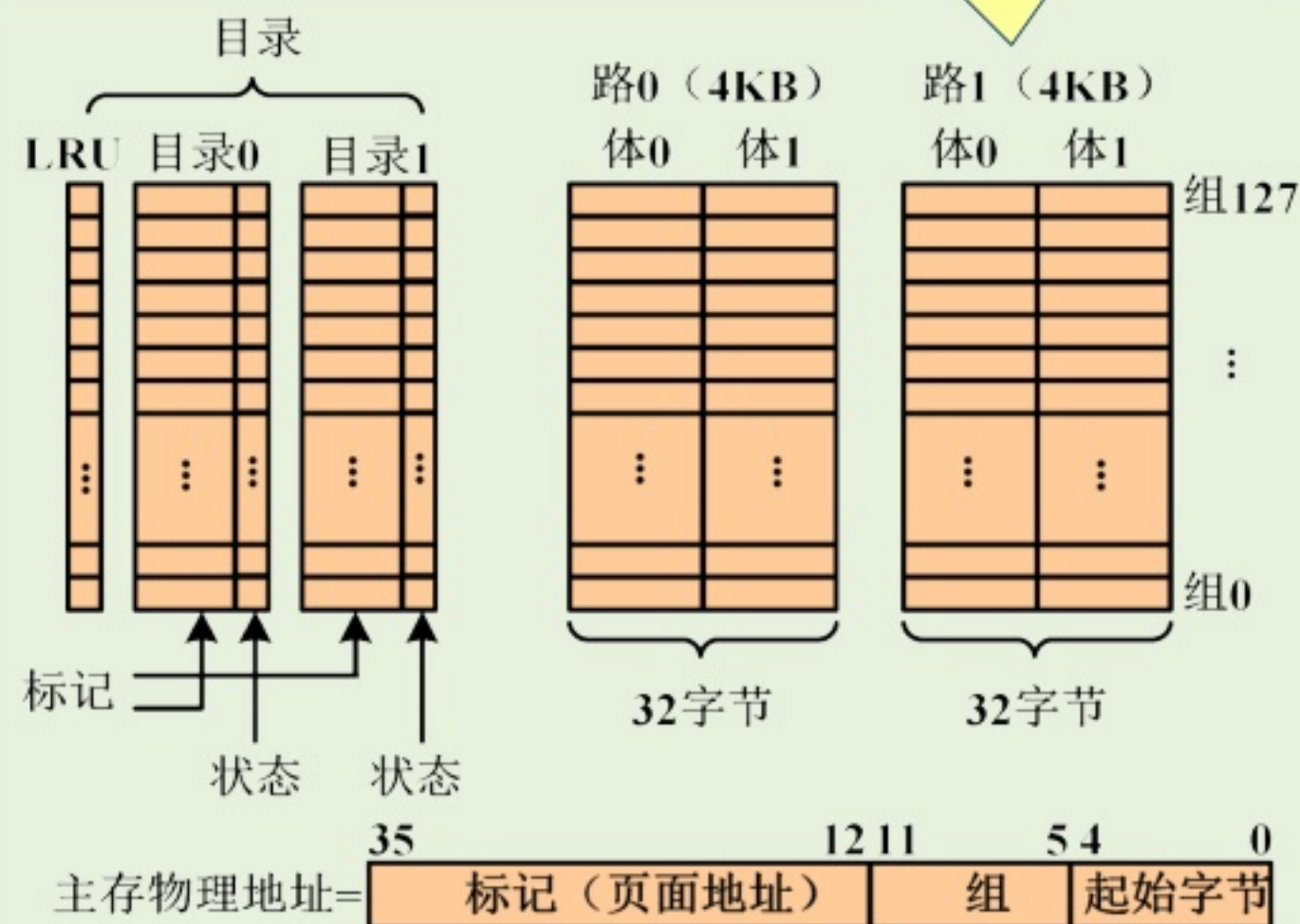
一、P6微架构下的Cache

Pentium II 处理器框图



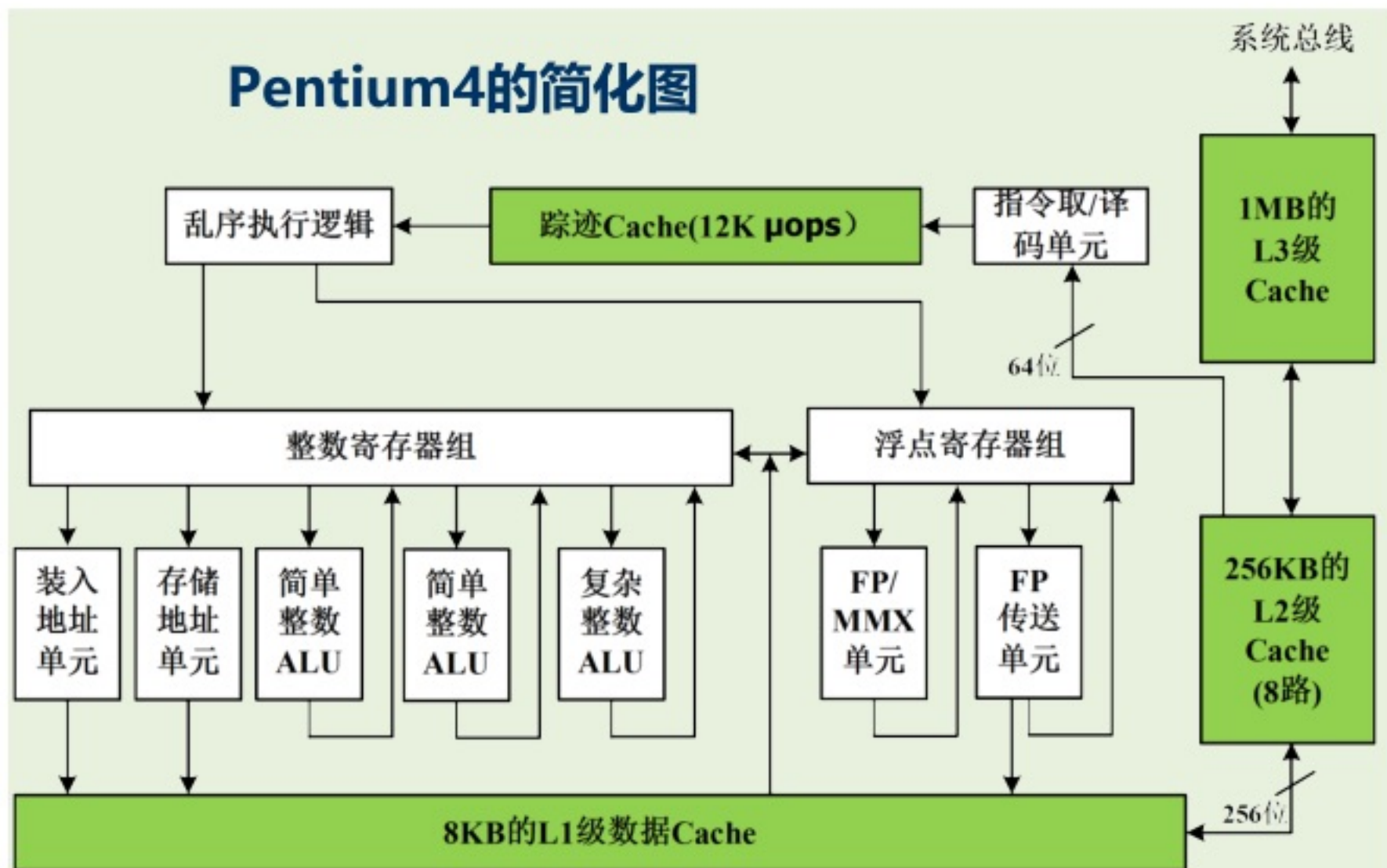
L1级数据Cache的结构

2路组相联，每块/行32B



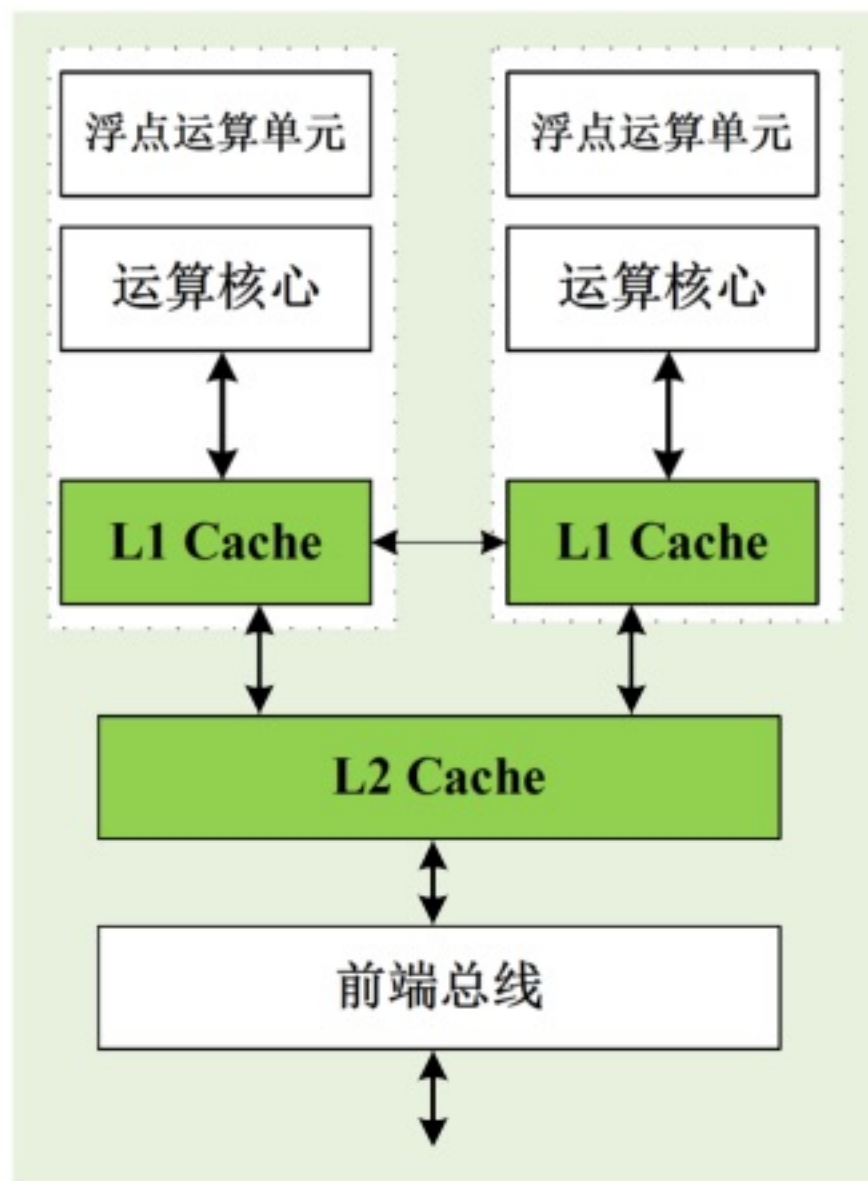


二、Intel NetBurst微架构下的Cache





三、Intel Core微架构的多核高效内存管理技术



❖ Intel Core微架构:

- 两个计算内核
- ❖ 两个核各具有**独立的L1级Cache**
 - L1级指令Cache: 8路32KB
 - L1级数据Cache: 8路32KB、双端口
- ❖ 两核**共享L2级Cache**: 16路、容量为2MB或4MB
- ❖ 集成在CPU内





本章小结

- ❖ 存储器的层次结构
- ❖ 主存的主要性能指标：容量、速度和价格
- ❖ 主存的分类
 - 随机读写存储器：SRAM、DRAM
 - 只读存储器
- ❖ 重点1：主存的扩展（SRAM）及与CPU的连接
- ❖ 重点2：提高主存速度的几种方法
 - 多端口存储器
 - 高速缓冲存储器
 - 多体交叉存储器
 - 相联存储器



❖ 重点3：高速缓存Cache

- Cache的特点和命中率
- Cache的地址映射方式：直接映射、全相联映射和组相联映射
- Cache的替换算法

❖ 虚拟存储器：概念、实现方式（页式、段式、段页式）

❖ 外存储器的特点

- 硬盘存储器、光盘存储器、闪存





The End!